

Top 100 Javascript Interview Questions





Table of Content

1. Basics	5
2. Variables & Datatypes	6
3. Operators & Conditions	8
4. Arrays	13
5. Loops	8
6. Functions	14
7. Strings	6
8. DOM	11
9. Error Handling	6
10. Objects	13
11. Events	10



Q. What is **DOM**? What is the difference between **HTML** and **DOM**? **V. IMP.**

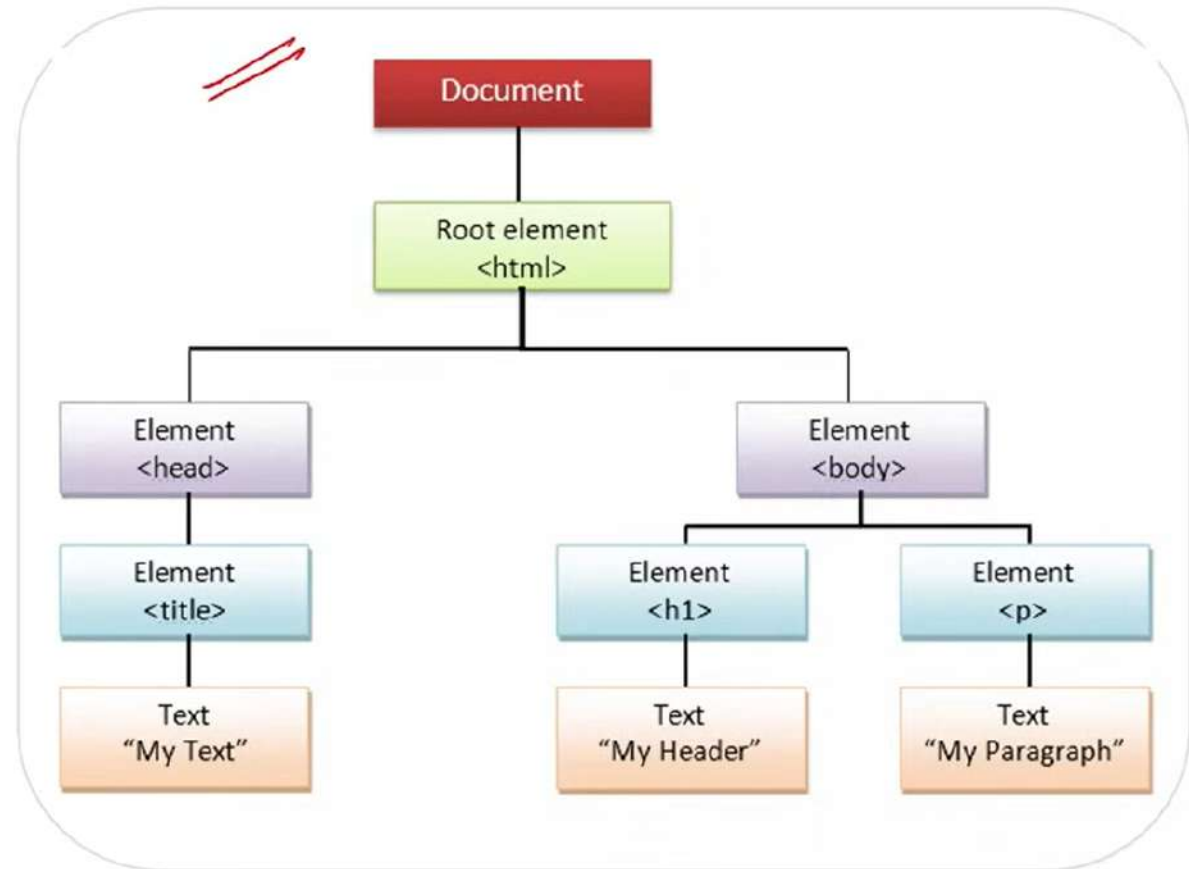


```
<!DOCTYPE html>
<html>

  <head>
    <title>My Text </title>
  </head>

  <body>
    <h1>
      My Header
    </h1>
    <p>My Paragraph </p>
  </body>

</html>
```



Q. What is **DOM**? What is the difference between **HTML** and **DOM**? **V. IMP.**



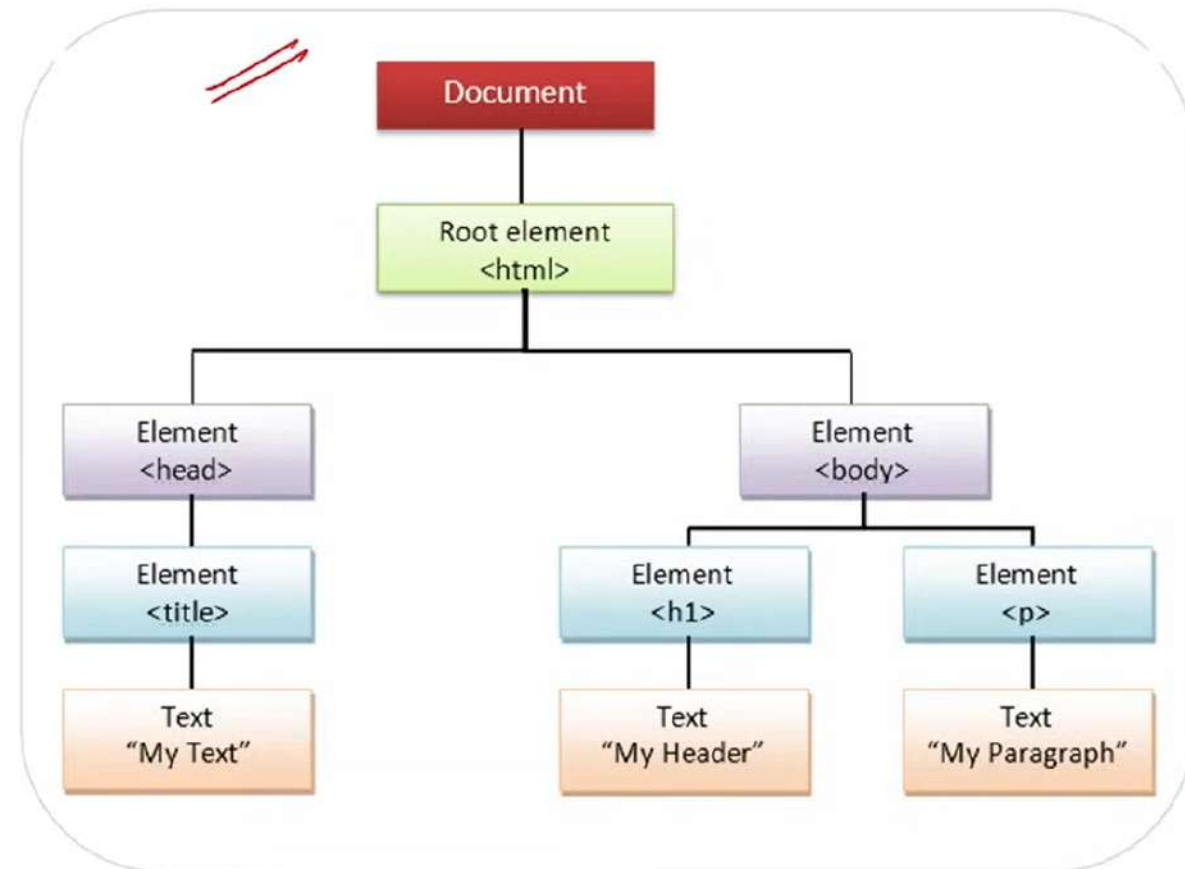
❖ The DOM represents the web page as a **tree-like structure**

```
<!DOCTYPE html>
<html>

  <head>
    <title>My Text </title>
  </head>

  <body>
    <h1>
      My Header
    </h1>
    <p> My Paragraph </p>
  </body>

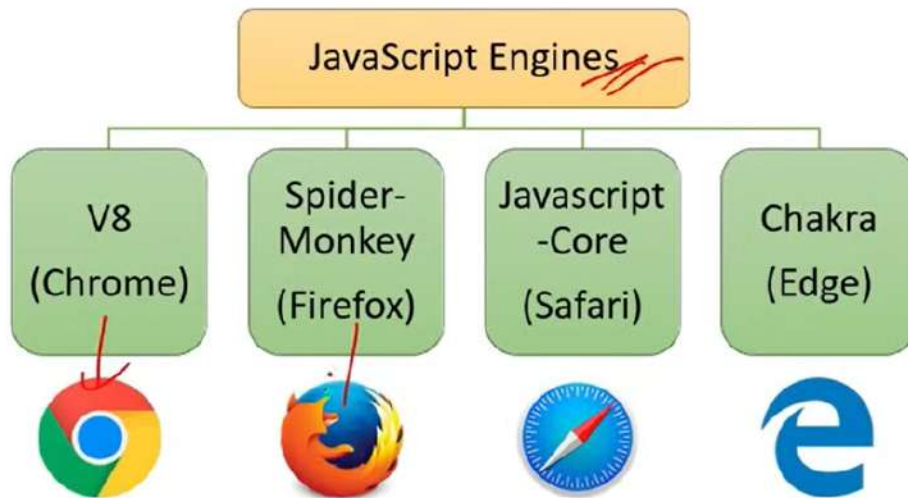
</html>
```



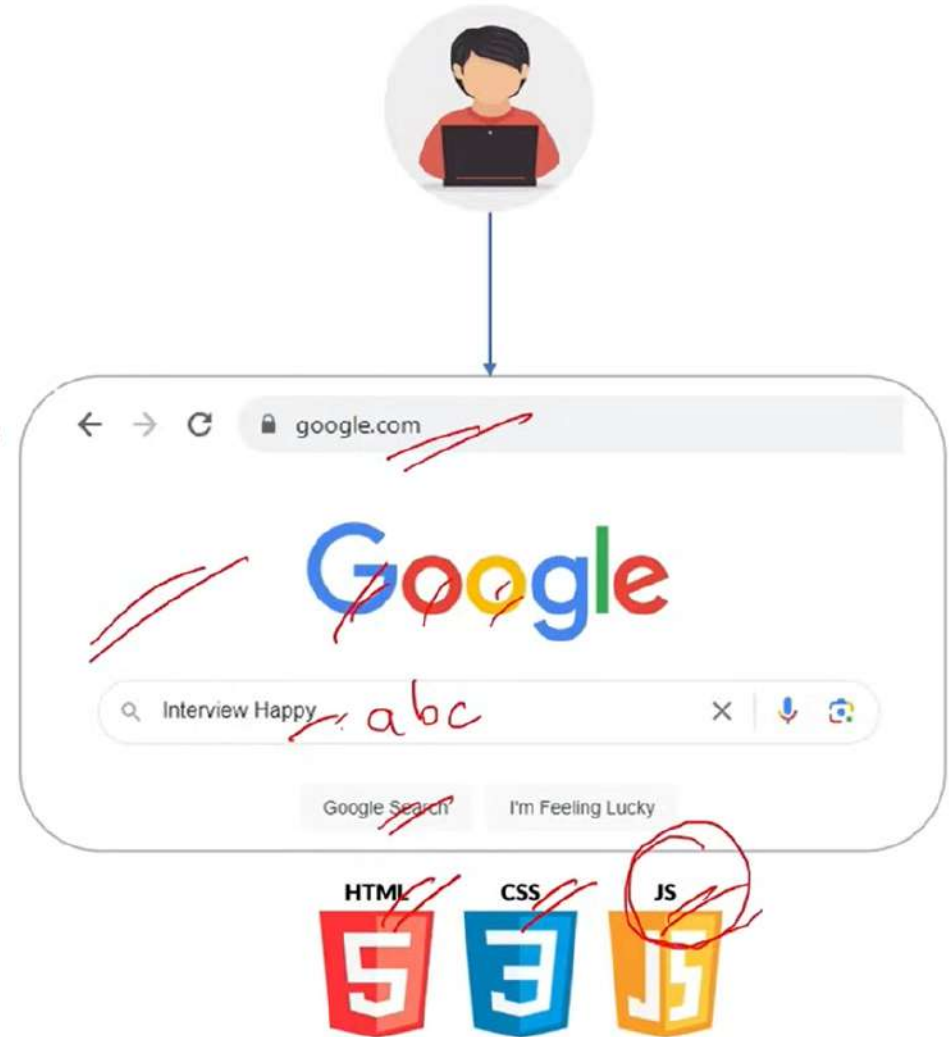


Q. What is **JavaScript**? What is the **role** of JavaScript engine? **V. IMP.**

- ❖ JavaScript is a programming language that is used for converting static web pages to **interactive and dynamic** web pages.



L7



Q. What is JavaScript? What is the role of JavaScript engine? **V. IMP.**



```
<> index.html >
1  <!DOCTYPE html>
2  <html lang="en">
3    <head>
4      <title>Document</title>
5    </head>
6    <body>
7      <h1>Interview Happy</h1>
8      <button id="myButton">Click</button>
9
10
11    </body>
12  </html>
```

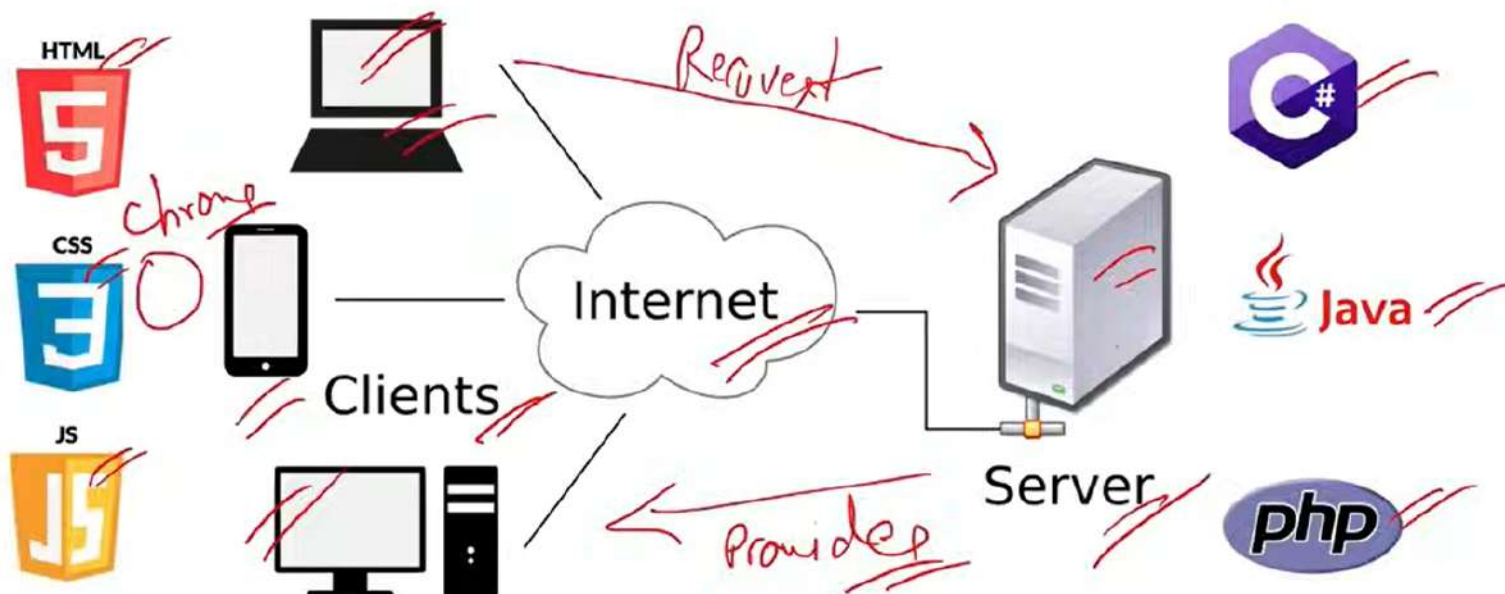


Q. What are Client side and Server side? V. IMP.



- ❖ A client is a device, application, or software component that **requests** and consumes services or resources from a server.

- ❖ A server is a device, computer, or software application that **provides** services, resources, or functions to clients.





Q. What is Scope in JavaScript? V. IMP.

- ❖ Scope determines where variables are **defined** and where they can be **accessed**.

Global Function Block

```
//Global - accessible anywhere
let globalVariable = "global";

greet();

function greet() {
  //Function - accessible inside function only
  let functionVariable = "function";
  if (true) {
    //Block - accessible inside block only
    let blockVariables = "block";
    console.log(blockVariables); //Output: block
    console.log(functionVariable); //Output: function
    console.log(globalVariable); //Output: global
  }
  console.log(functionVariable); //Output: function
  console.log(globalVariable); //Output: global
}
console.log(globalVariable); //Output: global
```



Q. What is the **type of a variable** in JS when it is declared without using the var, let, or const keywords?



- ❖ **“var”** is the implicit type of variable when a variable is declared without var, let, or const keywords.

```
if (true) {  
  var variable = 10;  
}  
  
console.log(variable);  
  
// Output: 10
```



Advanced

12. Closures

13. Asynchronous - Basics

14. Asynchronous - Promises

15. Asynchronous - Async Await

16. Browser APIs & Web Storage

17. Classes & Constructors

18. ECMAScript & Modules

19. Security & Performance

Scenario & Coding (50 Questions)

20. Tricky Short Questions



21. Feature Development



22. Coding Questions



LINK IN DESCRIPTION



Q. What is Hoisting in JavaScript? **V. IMP.**



Hoisting

```
//Function hoisting  
myFunction();  
  
function myFunction(){  
    console.log("Hello!");  
}  
//Output: Hello
```



Q. What is Hoisting in JavaScript? **V. IMP.**



Hoisting

```
//Function hoisting  
myFunction();  
  
function myFunction(){  
    console.log("Hello!");  
}  
//Output: Hello
```

```
//Variable Hoisting  
x = 10;  
console.log(x);  
//Output: 10  
  
var x;
```

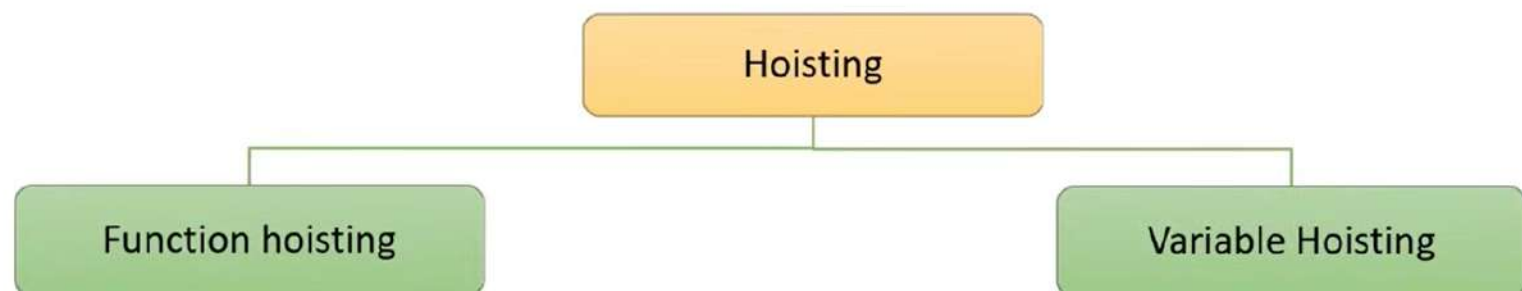


Q. What is Hoisting in JavaScript? **V. IMP.**



- ❖ Hoisting is a JavaScript behavior where functions and variable **declarations are moved to the top** of their respective scopes during the compilation phase.

Hoisting



```
//Function hoisting  
myFunction();  
  
function myFunction(){  
  console.log("Hello!");  
}  
  
//Output: Hello
```

```
//Variable Hoisting  
x = 10;  
console.log(x);  
//Output: 10  
  
var x;
```

Q. What is JSON?



- ❖ JSON (JavaScript Object Notation) is a lightweight **data interchange format**.
- ❖ JSON consists of **key-value** pairs.

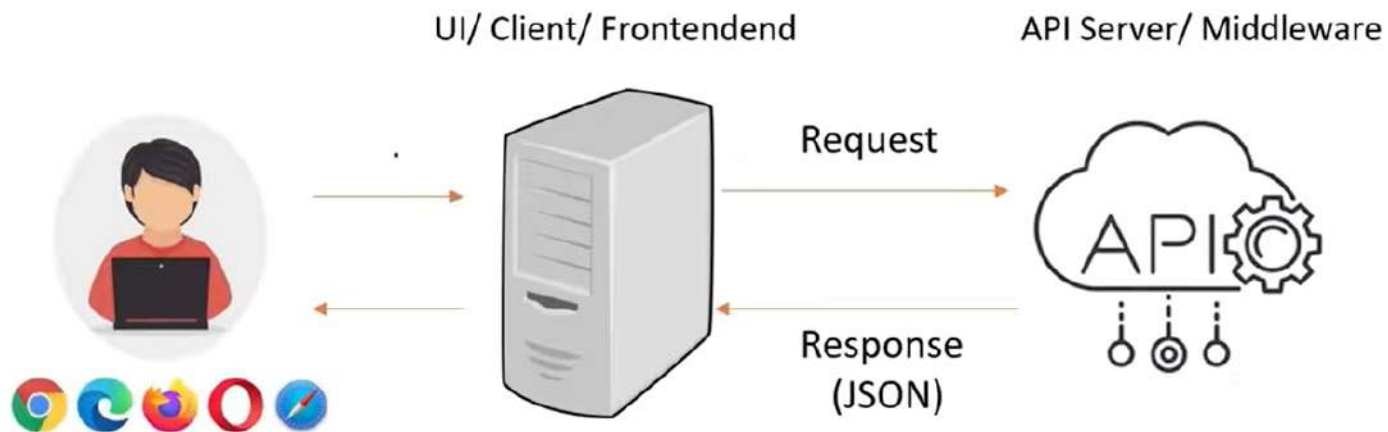
```
{  
  "name": "John Doe",  
  "age": 30,  
  "isStudent": false,  
  "address": {  
    "street": "123 Main St",  
    "city": "New York",  
    "country": "USA"  
  }  
}
```



Q. What is **JSON**?



- ❖ JSON (JavaScript Object Notation) is a lightweight **data interchange format**.
- ❖ JSON consists of **key-value** pairs.



```
{  
  "name": "John Doe",  
  "age": 30,  
  "isStudent": false,  
  "address": {  
    "street": "123 Main St",  
    "city": "New York",  
    "country": "USA"  
  }  
}
```

Chapter 2: Variables & Datatypes

JS

Q. What are variables? What is the difference between var, let, and const ?

basics

Q. What are data types in JS?

basics

Q. What is the difference between primitive and non-primitive data types?

Q. What is the difference between null and undefined in JS?

Q. What is the use of typeof operator?

Q. What is type coercion in JS?

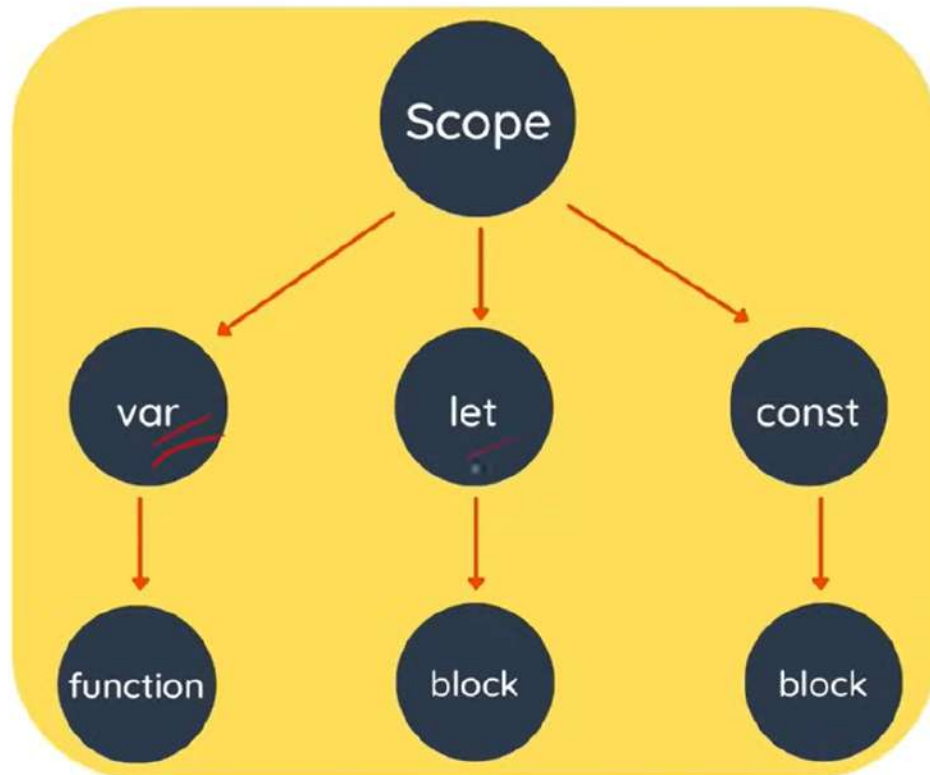


Q. What are **variables**? What is the difference between **var**, **let**, and **const**? **V. IMP.**



❖ Variables are used to **store** data.

```
var count = 10;
```



Q. What are **variables**? What is the difference between **var**, **let**, and **const**? **V. IMP.**



- ❖ **var** creates a **function-scoped** variable.

```
//using var
function example() {
  if (true) {
    var count = 10;
    console.log(count);
    //output: 10
  }
  console.log(count);
  //Output: 10
}
```

- ❖ **let** creates a **block-scoped** variable

```
//using let
function example() {
  if (true) {
    let count = 10;
    console.log(count);
    //Output: 10
  }
  console.log(count);
  //Output: Uncaught
  //Reference Error:
  //count is not defined
}
```

Q. What are **variables**? What is the difference between **var**, **let**, and **const**? **V. IMP.**



- ❖ **var** creates a **function-scoped** variable.

```
//using var
function example() {
  if (true) {
    var count = 10;
    console.log(count);
    //output: 10
  }
  console.log(count);
  //Output: 10
}
```

- ❖ **let** creates a **block-scoped** variable

```
//using let
function example() {
  if (true) {
    let count = 10;
    console.log(count);
    //Output: 10
  }
  console.log(count);
  //Output: Uncaught
  //Reference Error:
  //count is not defined
}
```

- ❖ **const** can be assigned only once, and its value **cannot be changed** afterwards.

```
// Using constant
const z = 10;
z = 20;

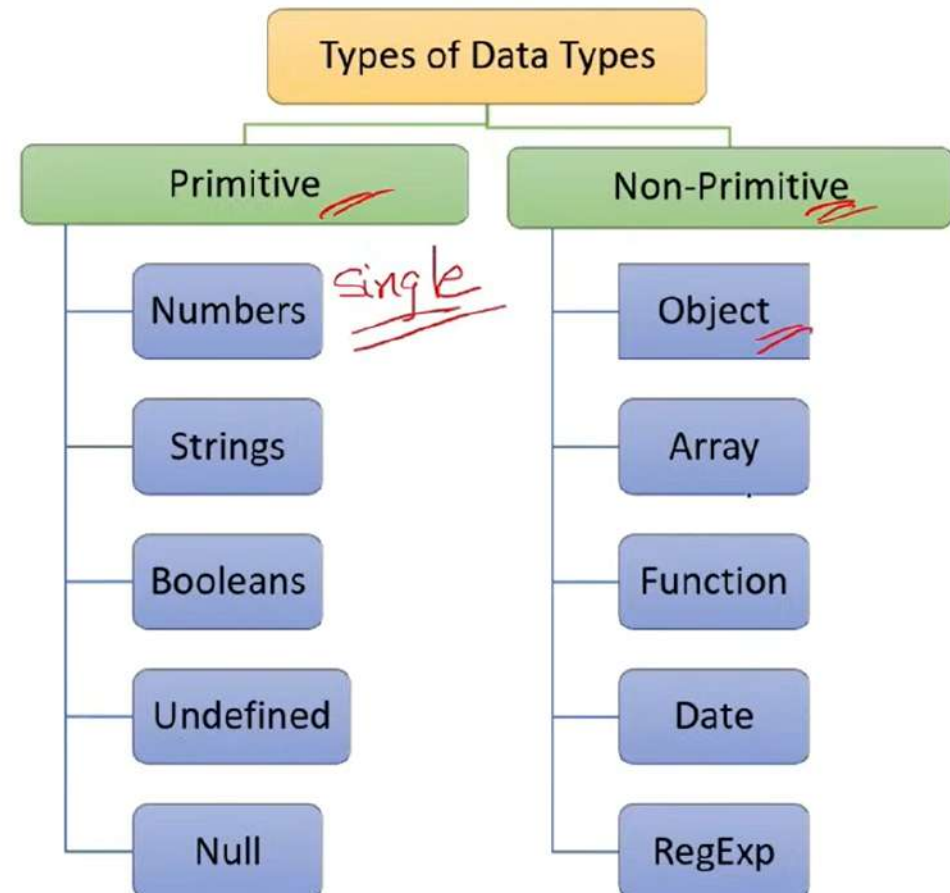
// This will result
//in an error
console.log(z);
```


Q. What are data types in JS?



```
//Number  
let age = 25;
```

```
//String  
let message = 'Hello!';  
  
//Boolean  
let isTrue = true;  
  
//Undefined  
let x;  
console.log(x);  
// Output: undefined  
  
//Null  
let y = null;  
console.log(y);  
// Output: null
```

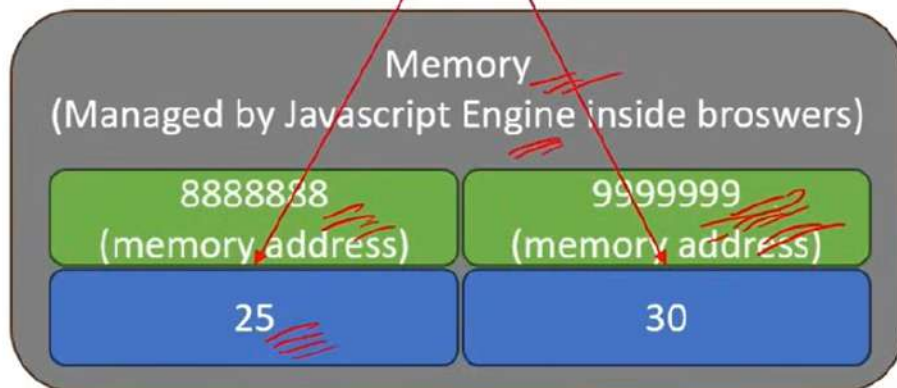


Q. What is the difference between **primitive** and **non-primitive** data types? **V. IMP.**



- ❖ Primitive data types can hold only **single** value.
- ❖ Primitive data types are **immutable**, meaning their values, **once assigned, cannot be changed.**

```
//Number  
let age = 25;  
age = 30; // new
```

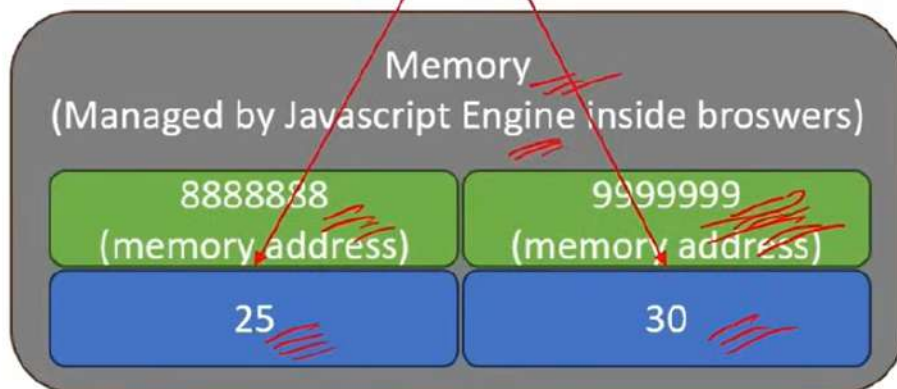


Q. What is the difference between **primitive** and **non-primitive** data types? **V. IMP.**



- ❖ Primitive data types can hold only **single** value.
- ❖ Primitive data types are **immutable**, meaning their values, once assigned, cannot be changed.

```
//Number  
let age = 25;  
age = 30; new
```



- ❖ Non primitive data types can hold **multiple** value.
- ❖ They are **mutable** and their values can be changed.

```
//Non primitive data types  
  
//Array  
let oddNumbers = [1, 3, 5]  
  
//Object  
let person = {  
  name: "John",  
  age: 30,  
  grades: ["A", "B", "C"],  
  greet: function() {  
    console.log(this.name);  
  }  
};
```



Q. What is the difference between **primitive** and **non-primitive** data types? **V. IMP.**



Primitive Data Types	Non-primitive Data Types
1. Number, string, Boolean, undefined, null are primitive data types.	Object, array, function, date, RegExp are non-primitive data types.
2. Primitive data types can hold only single value.	Non-primitive data types can hold multiple values and methods.
3. Primitive data types are immutable and their values cannot be changed.	Non-primitive data types are mutable and their values can be changed.



Q. What is the difference between **primitive** and **non-primitive** data types? **V. IMP.**



Primitive Data Types	Non-primitive Data Types
1. Number, string, Boolean, undefined, null are primitive data types.	Object, array, function, date, RegExp are non-primitive data types.
2. Primitive data types can hold only single value.	Non-primitive data types can hold multiple values and methods.
3. Primitive data types are immutable and their values cannot be changed.	Non-primitive data types are mutable and their values can be changed.
4. Primitive data types are simple data types.	Non-primitive data types are complex data types.



Q. What is the difference between null and undefined in JS?



```
let value1 = 0;  
let value2 = '';
```



- ❖ (A stand on the wall with also a paper holder)
Means there is a **valid variable** with also a value of **data type number**.

string

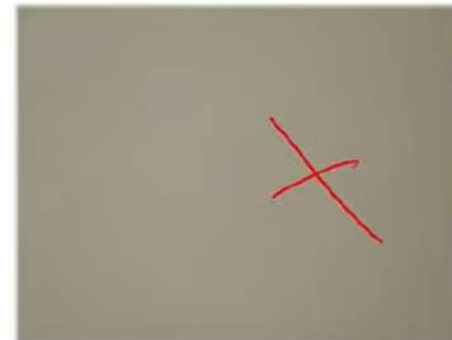
```
let value3 = null;
```

any data type



- ❖ (There is just a stand on the wall)
Means there is a **valid variable** with a value of **no data type**.

```
let value4;
```



- ❖ (There is nothing on the wall)
Means variable is **incomplete variable** and not assigned anything.

Q. What is the difference between **null** and **undefined** in JS?



```
let undefinedVariable; //no value assigned  
console.log(undefinedVariable);  
// Output: undefined
```

- ❖ **undefined**: When a variable is declared but has **not been assigned a value**, it is automatically initialized with **undefined**.
- ❖ **Undefined** can be used when you don't have the value right now, but you will get it after some logic or operation.

```
let nullVariable = null; //null assigned  
console.log(nullVariable);  
// Output: null
```

- ❖ **null**: null variables are intentionally assigned the **null value**.
- ❖ **Null** can be used, when you are sure you do not have any value for the particular variable.





Q. What is the use of **typeof** operator?

- ❖ **typeof** operator is used to determine the **type** of each variable.
- ❖ Real application use -> **typeof** operator can be used to **validate the data** received from external sources (**api**).

api → Number
→ validate
~~String~~

```
let num = 42;  
let str = "Hello, world!";  
let bool = true;  
let obj = { key: "value" };  
let arr = [1, 2, 3];  
let func = function() {};
```

```
//using typeof  
console.log(typeof num); // Output: "number"  
console.log(typeof str); // Output: "string"  
console.log(typeof bool); // Output: "boolean"  
console.log(typeof obj); // Output: "object"  
console.log(typeof arr); // Output: "object"  
console.log(typeof func); // Output: "function"  
console.log(typeof undefinedVariable);  
// Output: "undefined"
```

Q. What is **type coercion** in JS?



```
let string = "42";  
let number = 42;  
let boolean = true;  
let nullValue = null;
```

```
//Type coercion - automatic conversion  
console.log(string + number); // Output: "4242"
```



Q. What is type coercion in JS?



- ❖ Type coercion is the automatic conversion of values from one data type to another during certain operations or comparisons.

```
let string = "42";  
let number = 42;  
let boolean = true;  
let nullValue = null;
```

```
//Type coercion - automatic conversion  
console.log(string + number); // Output: "4242"
```



Q. What is type coercion in JS?



- ❖ Type coercion is the automatic conversion of values from one data type to another during certain operations or comparisons.

```
let string = "42";  
let number = 42;  
let boolean = true;  
let nullValue = null;
```

```
//Type coercion - automatic conversion  
console.log(string + number); // Output: "4242"  
  
console.log(number + boolean); // Output: 43  
  
console.log(number == string); // Output: true  
console.log(boolean == 1); // Output: true  
console.log(boolean + nullValue); // Output: 1
```


Chapter 3: Operators & Conditions

JS

Q. What are operators? What are the types of operators in JS? **basics**

Q. What is the difference between unary, binary, and ternary operators?

Q. What is short-circuit evaluation in JS?

Q. What is operator precedence?

Q. What are the types of conditions statements in JS? **basics**

Q. When to use which type of conditions statements in real applications?

Q. What is the difference between `==` and `===`?

Q. What is the difference between Spread and Rest operator in JS?



Q. What is type coercion in JS?



- ❖ Type coercion is the automatic conversion of values from one data type to another during certain operations or comparisons.

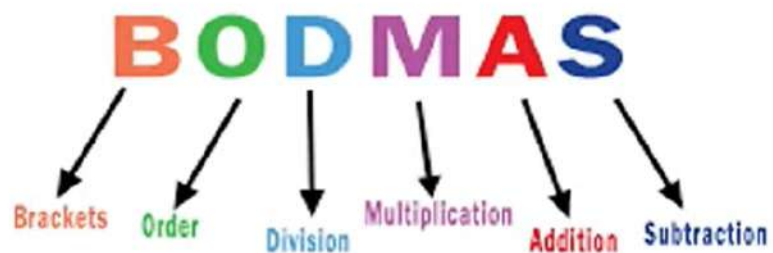
```
let string = "42";  
let number = 42;  
let boolean = true;  
let nullValue = null;
```

- ❖ Uses of type coercion:

1. Type coercion can be used during **String and Number concatenation**.
2. Type coercion can be used while using **Comparison operators**.

```
//Type coercion - automatic conversion  
console.log(string + number); // Output: "4242"  
  
console.log(number + boolean); // Output: 43  
  
console.log(number == string); // Output: true  
console.log(boolean == 1); // Output: true  
console.log(boolean + nullValue); // Output: 1
```

Q. What is **operator precedence**?



```
let a = 6;
let b = 3;
let c = 2;
```

```
//BracketOf-Division-Multiplication-Add-Sub
```

```
let result = a + b * c + (a - b);
```

```
console.log(result);
```

```
// Output: 15
```



Q. What is the difference between **unary**, **binary**, and **ternary** operators?



Operators based on number of operands

Unary Operator

```
let a = 5;

// Unary operator
// Single operand
let b = -a;
console.log(b);
// Output: -5
console.log(++a);
// Output: 6
```

Binary Operator

```
let x = 10;
let y = 3;

// Binary operator
// Two operands
let z = x + y;
console.log(z);
// Output: 13
```

Ternary Operator

```
// Ternary operator
// Three operands
let result = condition ? 'Yes' : 'No';

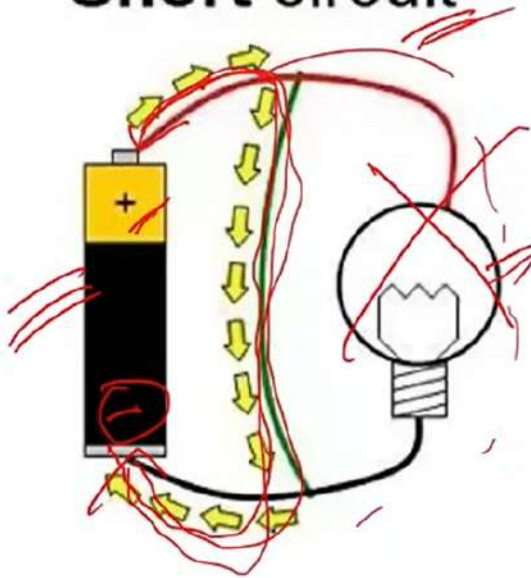
console.log(result);
// Output: 'Yes'
```



Q. What is **short-circuit** evaluation in JS?



Short circuit



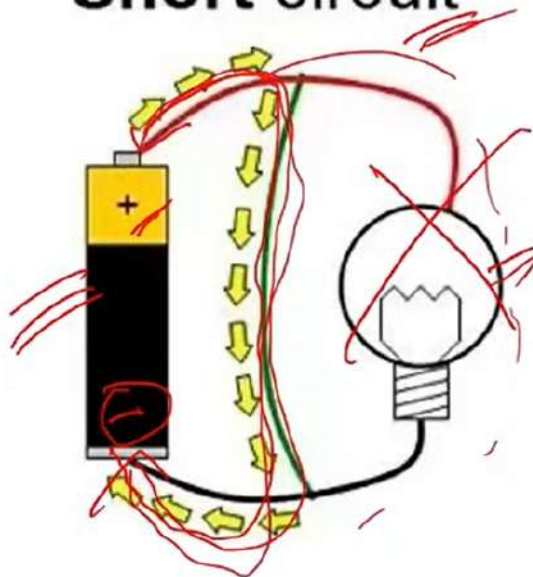
```
// Short-circuit evaluation with logical AND  
let result1 = false && someFunction();  
console.log(result1);  
// Output: false
```



Q. What is **short-circuit** evaluation in JS?



Short circuit



```
// Short-circuit evaluation with logical AND  
let result1 = false && someFunction();  
console.log(result1);  
// Output: false
```

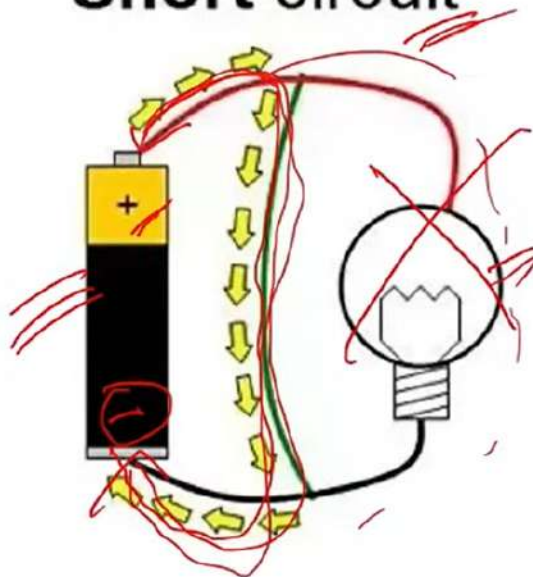
```
// Short-circuit evaluation with logical OR  
let result2 = true || someFunction();  
console.log(result2);  
// Output: true
```

Q. What is **short-circuit** evaluation in JS?



- ❖ Short-circuit evaluation stops the execution as soon as the **result can be determined without evaluating** the remaining sub-expressions.

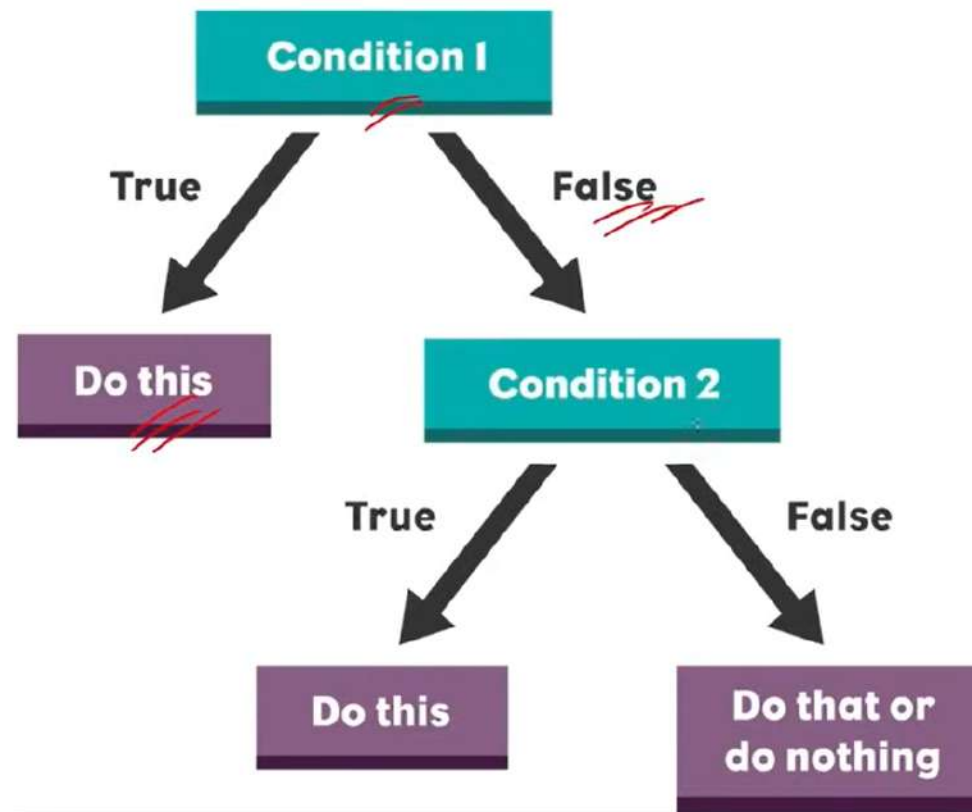
Short circuit



```
// Short-circuit evaluation with logical AND  
let result1 = false && someFunction();  
console.log(result1);  
// Output: false
```

```
// Short-circuit evaluation with logical OR  
let result2 = true || someFunction();  
console.log(result2);  
// Output: true
```

Q. What are the types of **conditions statements** in JS? **V. IMP.**



Q. What are the types of **conditions statements** in JS? **V. IMP.**



Types of condition statements

1. If/ else statements

```
let x = 5;

if (x > 10) {
  console.log("1");
} else if (x < 5) {
  console.log("2");
} else {
  console.log("3");
}

// Output: '3'
```

2. Ternary operator

```
let y = 20;
let z = y > 10 ? "1" : "0";
console.log(z);
// Output: '1'
```

3. Switch statement

```
let a = 5;
switch (a) {
  case 1:
    console.log("1");
    break;
  case 5:
    console.log("2");
    break;
  default:
    console.log("3");
}

// Output: '2'
```



Q. What is the difference between == and ===? **V. IMP.**



type coercion

```
//Loose Equality  
console.log(1 == '1');  
console.log(true == 1);  
// Output: true
```

```
//Strict Equality  
console.log(1 === '1');  
console.log(true === 1);  
// Output: false
```

- ❖ Loose Equality (==) operator compares two values for equality after performing **type coercion**



Q. What is the difference between == and ===? **V. IMP.**



type coercion

```
//Loose Equality  
console.log(1 == '1');  
console.log(true == 1);  
// Output: true
```

- ❖ Loose Equality (==) operator compares two values for equality after performing **type coercion**

```
//Strict Equality  
console.log(1 === '1');  
console.log(true === 1);  
// Output: false
```

- ❖ Strict Equality (===) operator compares two values for equality **without** performing type coercion.



Q. What is the difference between == and ===? **V. IMP.**



type coercion

```
//Loose Equality  
console.log(1 == '1');  
console.log(true == 1);  
// Output: true
```

- ❖ Loose Equality (==) operator compares two values for equality after performing **type coercion**.

```
//Strict Equality  
console.log(1 === '1');  
console.log(true === 1);  
// Output: false
```

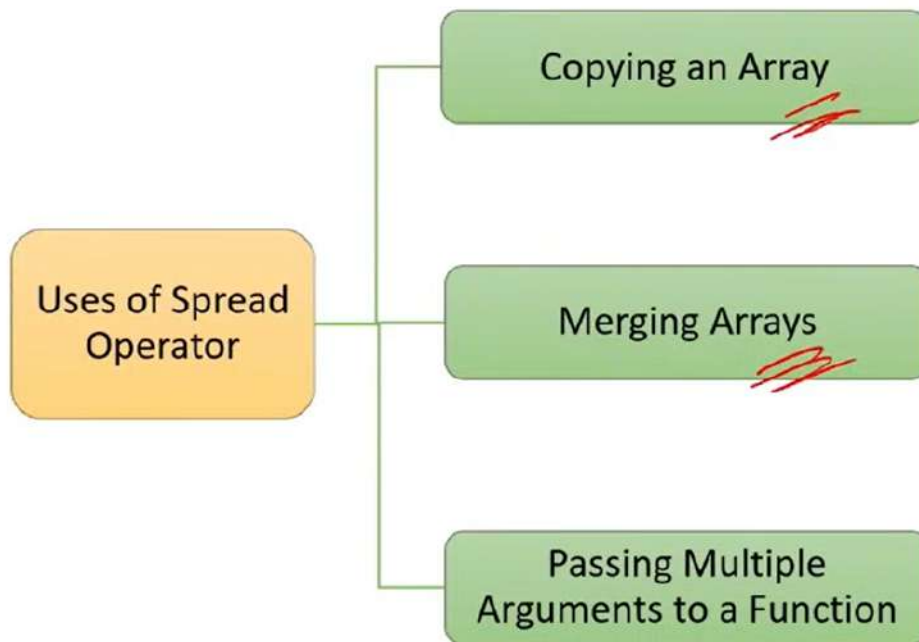
- ❖ Strict Equality (===) operator compares two values for equality **without** performing type coercion.
- ❖ Normally **=== is preferred** in use to get more accurate comparisons.





Q. What is the difference between **Spread** and **Rest** operator in JS?

- ❖ The spread operator(...) is used to **expand or spread elements** from an iterable (such as an array, string, or object) into individual elements.



```
// Spread Operator Examples
const array = [1, 2, 3];
console.log(...array); // Output: 1, 2, 3
```

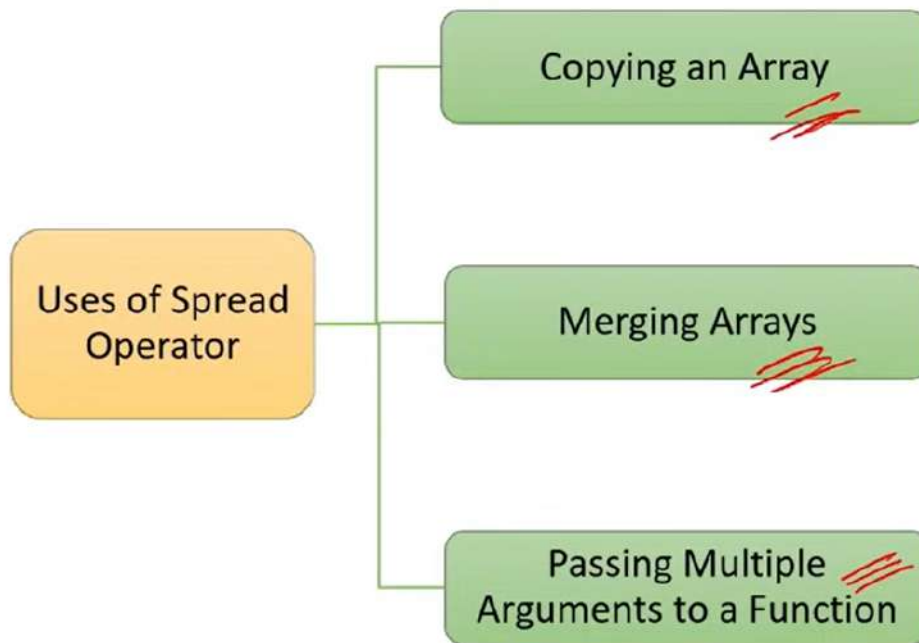
```
// Copying an array
const originalArray = [1, 2, 3];
const copiedArray = [...originalArray];
console.log(copiedArray); // Output: [1, 2, 3]
```

```
// Merging arrays
const array1 = [1, 2, 3];
const array2 = [4, 5];
const mergedArray = [...array1, ...array2];
console.log(mergedArray); // Output: [1, 2, 3, 4, 5]
```



Q. What is the difference between **Spread** and **Rest** operator in JS?

- ❖ The spread operator(...) is used to **expand or spread elements** from an iterable (such as an array, string, or object) into individual elements.



```
// Spread Operator Examples
const array = [1, 2, 3];
console.log(...array); // Output: 1, 2, 3
```

```
// Copying an array
const originalArray = [1, 2, 3];
const copiedArray = [...originalArray];
console.log(copiedArray); // Output: [1, 2, 3]
```

```
// Merging arrays
const array1 = [1, 2, 3];
const array2 = [4, 5];
const mergedArray = [...array1, ...array2];
console.log(mergedArray); // Output: [1, 2, 3, 4, 5]
```

```
// Passing multiple arguments to a function
const numbers = [1, 2, 3, 4, 5];
sum(...numbers);
function sum(a, b, c, d, e) {
  console.log(a + b + c + d + e); //Output: 15
}
```



Q. What is the difference between **Spread** and **Rest** operator in JS?



```
// Rest Operator Example
display(1, 2, 3, 4, 5);

function display(first, second, ...restArguments) {
  console.log(first); // Output: 1
  console.log(second); // Output: 2

  console.log(remaining); // Output: [3, 4, 5]
}
```



Q. What is the difference between **Spread** and **Rest** operator in JS?



- ❖ The rest operator is used in function parameters to collect all **remaining arguments** into an array.

```
// Rest Operator Example
display(1, 2, 3, 4, 5);

function display(first, second, ...restArguments) {
  console.log(first); // Output: 1
  console.log(second); // Output: 2

  console.log(remaining); // Output: [3, 4, 5]
}
```

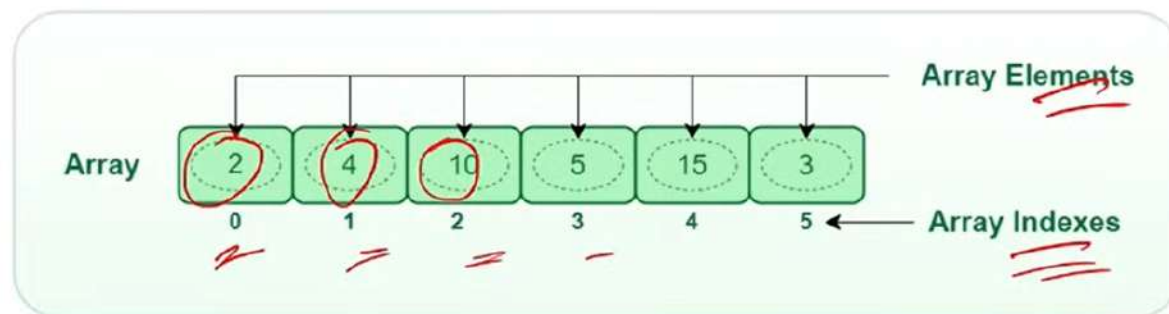


Q. What are **Arrays** in JS? How to **get**, **add** & **remove** elements from arrays? **V. IMP.**

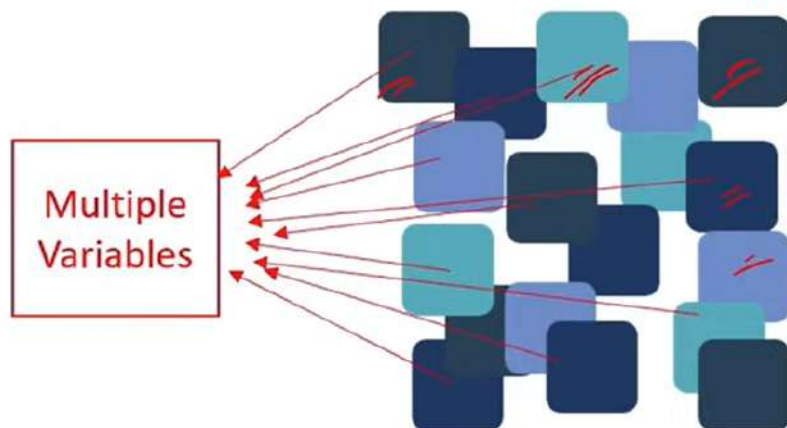


- ❖ An array is a data type that allows you to **store multiple values** in a single variable.

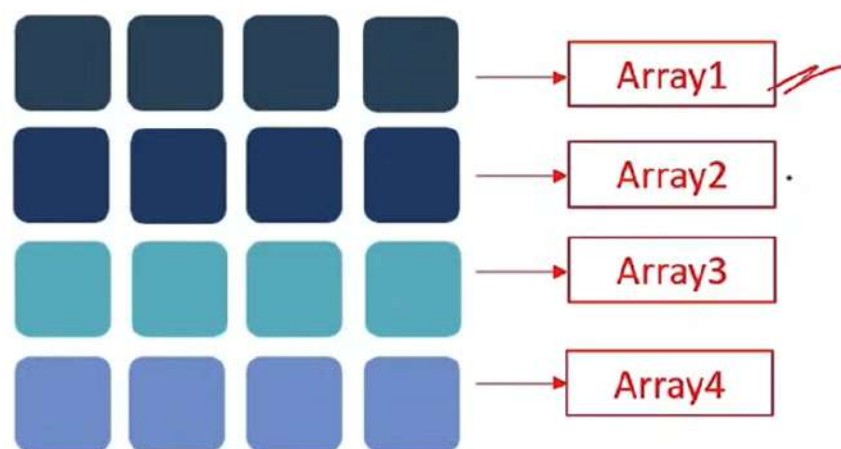
```
//Array  
let fruits = ["apple", "banana", "orange"];
```



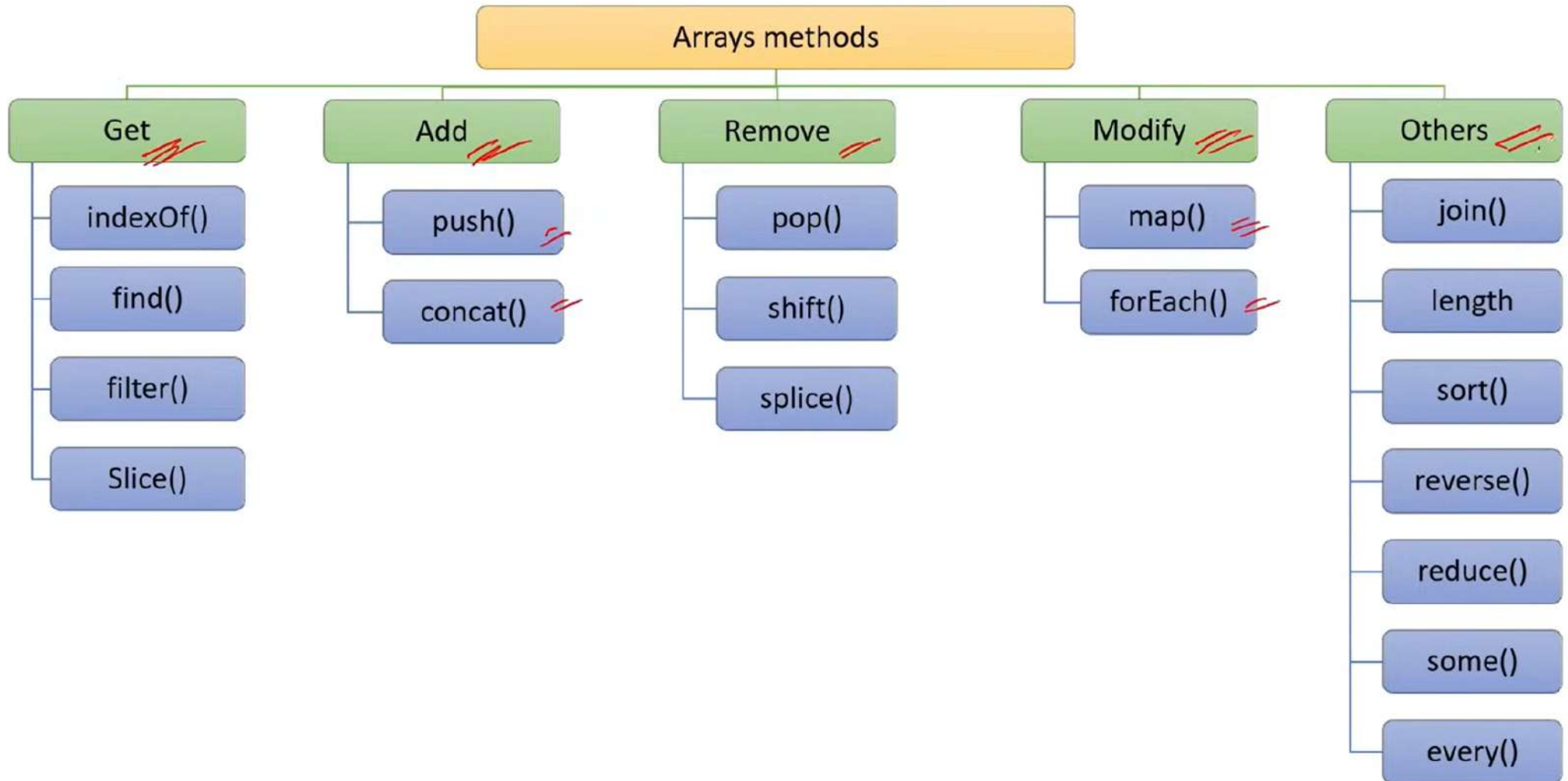
UNSTRUCTURED DATA



STRUCTURED DATA



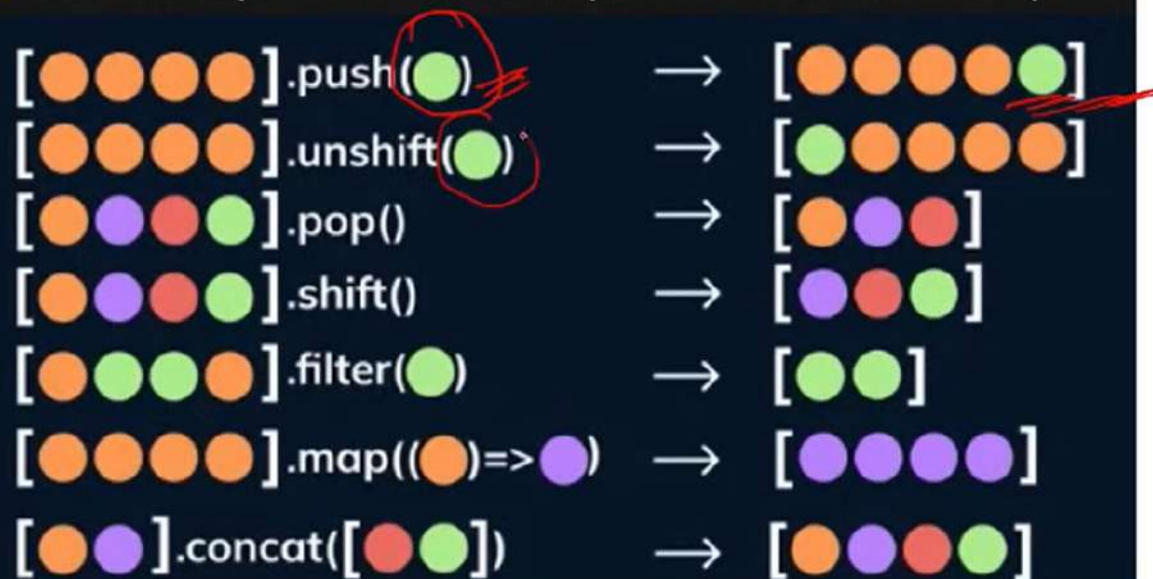
Q. What are **Arrays** in JS? How to **get**, **add** & **remove** elements from arrays? **V. IMP.**



Q. What are **Arrays** in JS? How to **get**, **add** & **remove** elements from arrays? **V. IMP.**



❖ Pictorial representation of important method of arrays



Q. What is the `indexOf()` method of an Array?



```
// Example array
const array = [1, 2, 3, 4, 5];
let a = array.indexOf(3);
console.log(a);
// Output: 2
```

Handwritten annotations in red: The number 1 above the first element is circled. The number 2 above the second element is circled. An arrow points from the word 'element' to the value 3 in the `indexOf(3)` call. The output `2` is underlined.





Q. What is the `indexOf()` method of an Array?

- ❖ `indexOf()` method **gets the index** of a specified element in the array.

```
// Example array
const array = [1, 2, 3, 4, 5];
let a = array.indexOf(3);
console.log(a);
// Output: 2
```

Handwritten annotations: Red circles around the numbers 1, 2, and 3 in the array. Red arrows pointing from the word 'element' to the number 3 in the `indexOf(3)` call. Red arrows pointing from the word 'index' to the number 2 in the `console.log(a)` call.

Q. What is the difference between **find()** and **filter()** methods of an Array? **V. IMP.**



Array methods for **getting** elements

find()

filter()

slice()

```
// Example array
const array = [1, 2, 3, 4, 5];
let c = array.find((num) =>
  num % 2 === 0);
console.log(c);
// Output: 2
```

❖ **find()** method get the **first element** that satisfies a condition.



Q. What is the difference between **find()** and **filter()** methods of an Array? **V. IMP.**



Array methods for **getting** elements

find()

filter()

slice()

```
// Example array
const array = [1, 2, 3, 4, 5];
let c = array.find((num) =>
  num % 2 === 0);
console.log(c);
// Output: 2
```

```
// Example array
const array = [1, 2, 3, 4, 5];
let d = array.filter((num) =>
  num % 2 === 0);
console.log(d);
// Output: [2, 4]
```

❖ **find()** method get the **first element** that satisfies a condition.

❖ **filter()** method **get an array of elements** that satisfies a condition.

Q. What is the `slice()` method of an Array? **V. IMP.**



Array methods for **getting** elements

`find()`

`filter()`

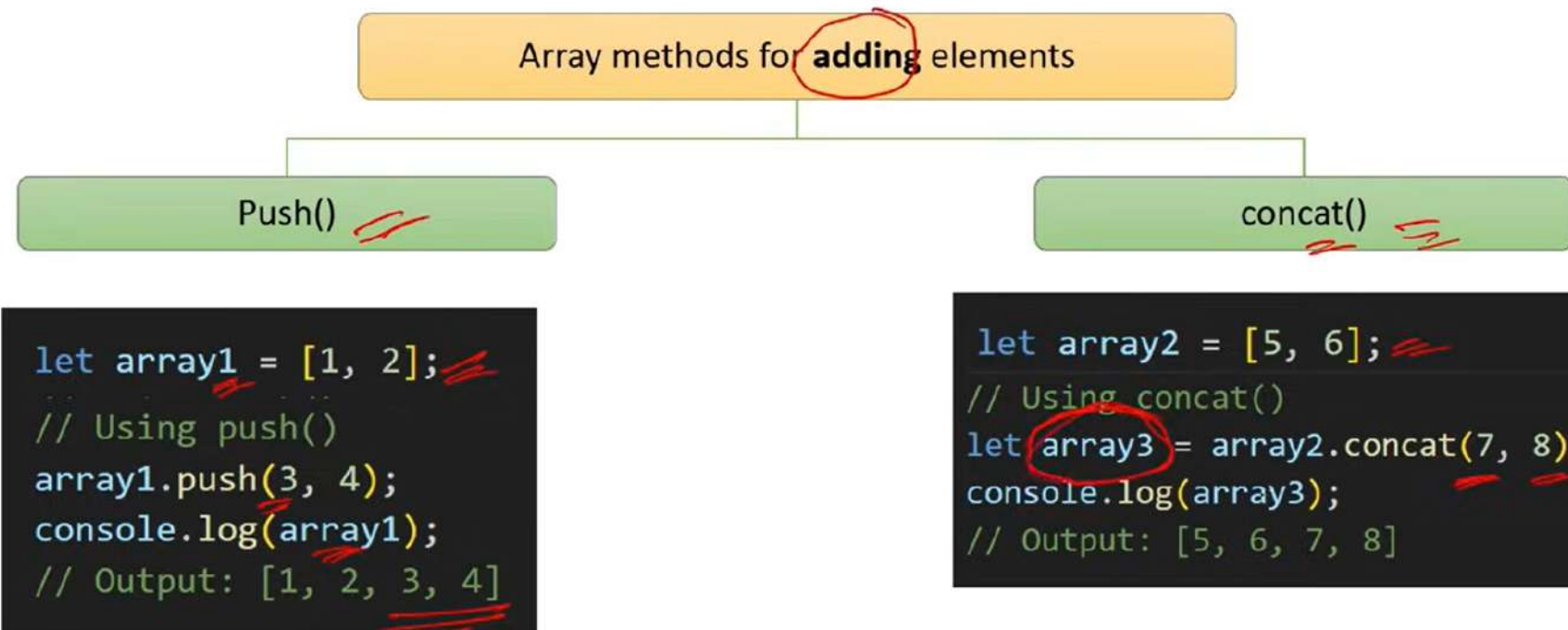
`slice()`

```
const array = ["a", "b", "c", "d", "e"];  
let e = array.slice(1, 4);  
console.log(e);  
  
// Output: ['b', 'c', 'd']
```

❖ `Slice()` method get a **subset of the array** from start index to end index (end not included).

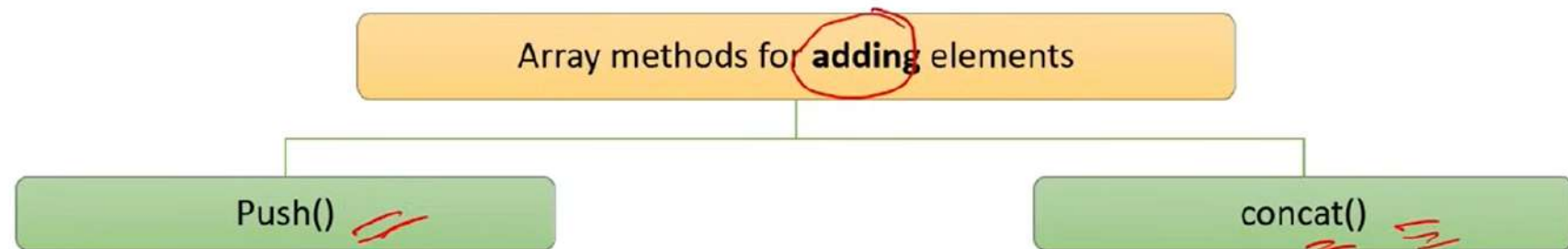


Q. What is the difference between push() and concat() methods of an Array?



- ❖ Push() will **modify the original array** itself.

Q. What is the difference between push() and concat() methods of an Array?



```
let array1 = [1, 2];  
// Using push()  
array1.push(3, 4);  
console.log(array1);  
// Output: [1, 2, 3, 4]
```

- ❖ Push() will **modify the original array** itself.

```
let array2 = [5, 6];  
// Using concat()  
let array3 = array2.concat(7, 8);  
console.log(array3);  
// Output: [5, 6, 7, 8]  
  
console.log(array2);  
// original array is not modified  
// Output: [5, 6]
```

- ❖ Concat() method will **create the new array** and not modify the original array.

Q. What is the difference between pop() and shift() methods of an Array?



Array methods for removing elements

pop()

```
// Using pop()
let arr1 = [1, 2, 3, 4];
let popped = arr1.pop();
console.log(popped);
// Output: 4
console.log(arr1);
// Output: [1, 2, 3]
```

❖ pop() will remove the last element of the array

shift()

```
// Using shift()
let arr2 = [1, 2, 3, 4];
let shifted = arr2.shift();
console.log(shifted);
// Output: 1
console.log(arr2);
// Output: [2, 3, 4]
```

❖ Shift() will remove the first element of the array





Q. What is the splice() method of an Array? **V. IMP.**

- ❖ The splice() method is used to add, remove, or replace elements in an array.

```
array.splice(startIndex, deleteCount, ...3itemsToAdd);
```





Q. What is the splice() method of an Array? **V. IMP.**

- ❖ The splice() method is used to add, remove, or replace elements in an array.

`array.splice(startIndex, deleteCount, ...itemsToAdd);`

```
let letters = ['a', 'b', 'c'];
```

```
// Add 'x' and 'y' at index 1  
letters.splice(1, 0, 'x', 'y');  
console.log(letters);
```

```
// Output: ['a', 'x', 'y', 'b', 'c']
```

```
// Removes 1 element starting from index 1
```

```
letters.splice(1, 1);
```

```
console.log(letters);
```

```
// Output: ['a', 'y', 'b', 'c']
```





Q. What is the splice() method of an Array? **V. IMP.**

- ❖ The splice() method is used to add, remove, or replace elements in an array.

array.splice(startIndex, deleteCount, ...itemsToAdd);

```
let letters = ['a', 'b', 'c'];
```

```
// Add 'x' and 'y' at index 1  
letters.splice(1, 0, 'x', 'y');  
console.log(letters);
```

```
// Output: ['a', 'x', 'y', 'b', 'c']
```

```
// Removes 1 element starting from index 1  
letters.splice(1, 1);  
console.log(letters);
```

```
// Output: ['a', 'y', 'b', 'c']
```

```
// Replaces the element at index 2 with 'q'  
letters.splice(2, 1, 'q');  
console.log(letters);  
// Output: ['a', 'y', 'q', 'c']
```





Q. What is the difference between the `slice()` and `splice()` methods of an Array?

- ❖ The `slice()` method is used to get a subset of the array from the start index to the end index (end not included).
- ❖ The `splice()` method is used to **add, remove, or replace** elements in an array.



Q. What is the difference map() and forEach() array methods of an Array?



Array methods for modification and iteration

map()

```
// Using map()  
let arr1 = [1, 2, 3];  
let mapArray = arr1.map((e) => e * 2);  
console.log(mapArray);  
//map return a new array  
// Output: [2, 4, 6]
```

forEach()

```
// Using forEach()  
let arr2 = [1, 2, 3];  
arr2.forEach((e) => {  
  console.log(e * 2);  
});  
//Does not return anything  
// Output: 2 4 6
```

❖ The map() method is used when you want to modify each element of an array and create a **new array** with the modified values.

❖ The forEach() method is used when you want to perform some operation on each element of an array **without creating a new array**.



Q. How to **sort** and **reverse** an array?



- ❖ Array can be sorted or reversed by using **sort()** and **reverse()** methods of array.

```
let array = ['c', 'e', 'a', 't'];
```

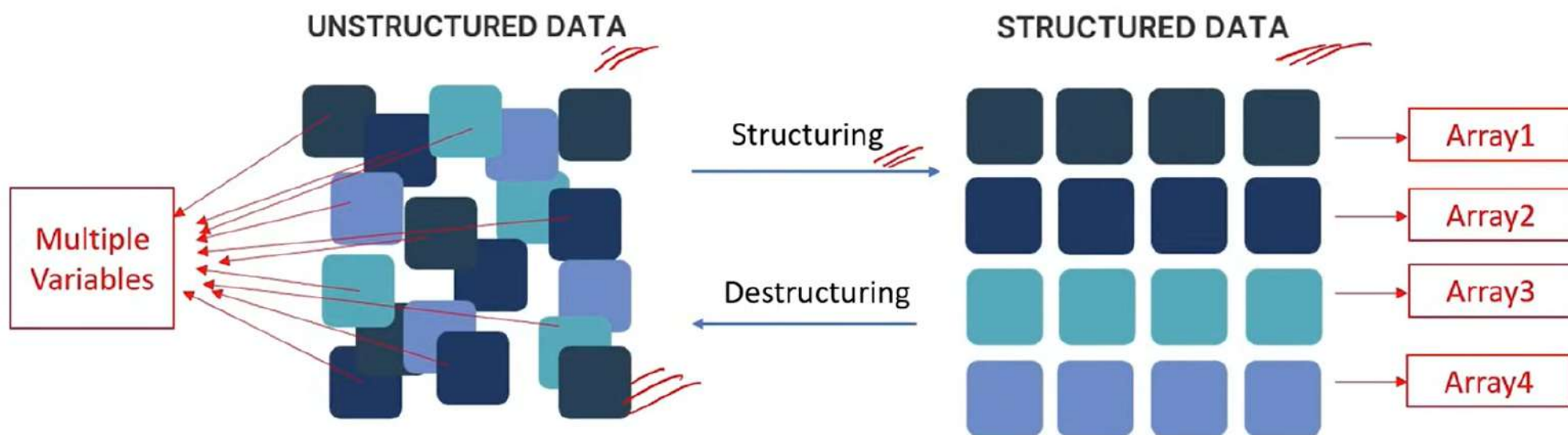
```
//sort the array  
array.sort();  
console.log(array);  
// Output: ['a', 'c', 'e', 't']
```

```
//reverse the array  
array.reverse();  
console.log(array);  
// Output: ['t', 'e', 'c', 'a']
```

Q. What is **Array Destructuring** in JS? **V. IMP.**



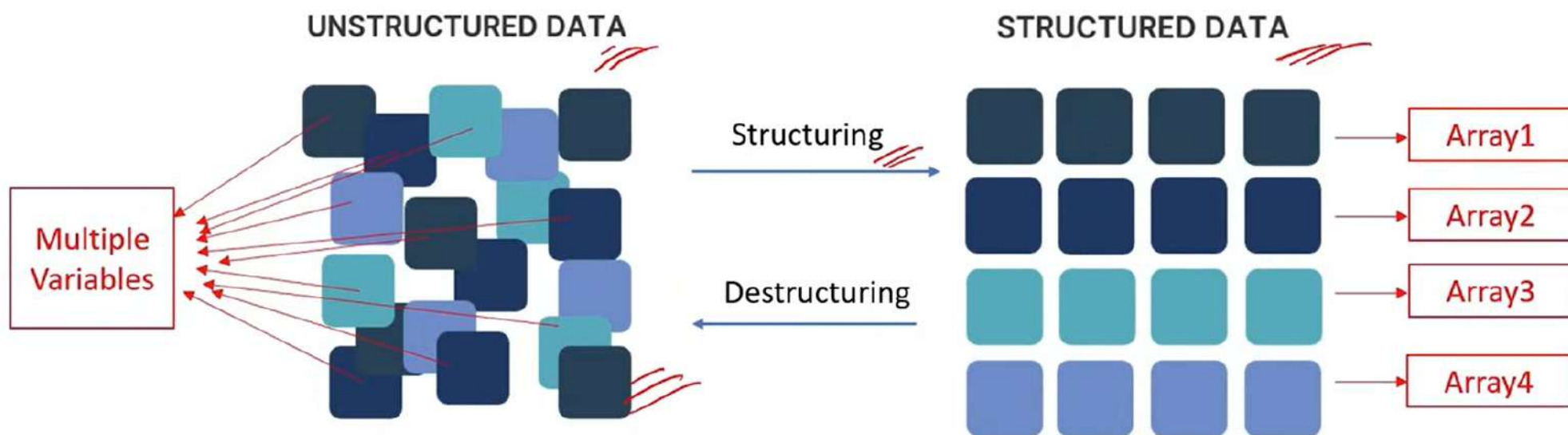
- ❖ Array destructuring allows you to extract elements from an array and assign them to **individual variables** in a single statement.



Q. What is **Array Destructuring** in JS? **V. IMP.**



- ❖ Array destructuring allows you to extract elements from an array and assign them to **individual variables** in a single statement.
- ❖ Array destructuring is introduced in **ECMAScript 6 (ES6)**.



Q. What is **array destructuring** in JS?



```
// Example array  
const fruits = ['apple', 'banana', 'orange'];
```

```
// Array destructuring  
const [firstFruit, secondFruit, thirdFruit] = fruits;
```

```
// Output  
console.log(firstFruit); // Output: "apple"  
console.log(secondFruit); // Output: "banana"  
console.log(thirdFruit); // Output: "orange"
```



Q. What are **array-like objects** In JS? **V. IMP.**



- ❖ Array-like objects are objects that have indexed elements and a length property, **similar to arrays**, but they may not have all the methods of arrays like `push()`, `pop()` & others.

```
//String
const str = "Hello";
console.log(str); // Output: Hello
console.log(str.length); // Output: 5
console.log(str[0]); // Output: H
```

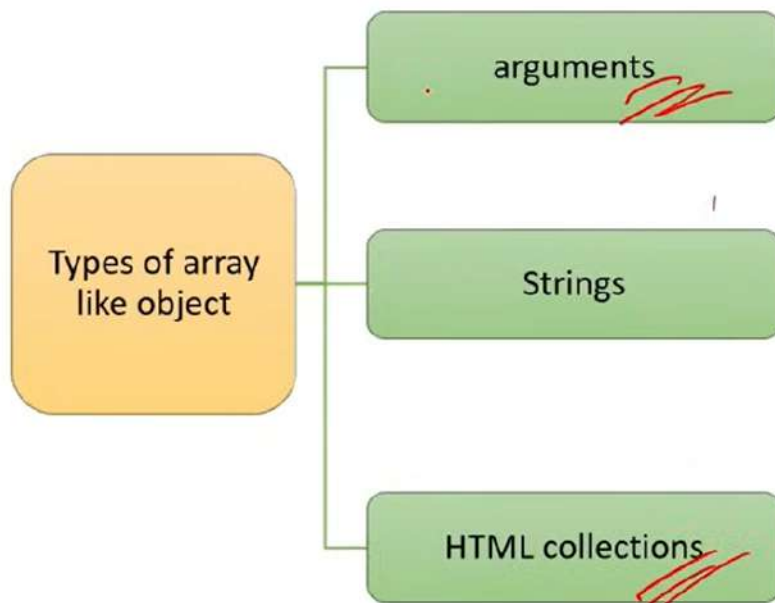
array like



Q. What are array-like objects In JS? **V. IMP.**



- ❖ Array-like objects are objects that have indexed elements and a length property, **similar to arrays**, but they may not have all the methods of arrays like push(), pop() & others.



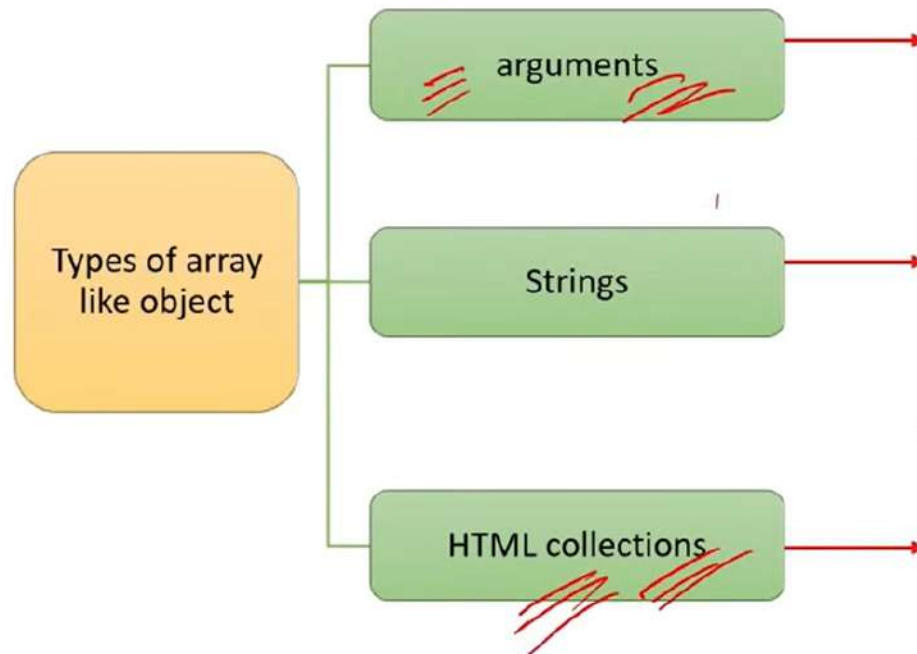
```
//String  
const str = "Hello";  
console.log(str); // Output: Hello  
console.log(str.length); // Output: 5  
console.log(str[0]); // Output: H
```

array like



Q. What are array-like objects In JS? **V. IMP.**

- ❖ Array-like objects are objects that have indexed elements and a length property, **similar to arrays**, but they may not have all the methods of arrays like push(), pop() & others.



```
sum(1, 2, 3);  
//Arguments Object  
function sum() {  
  console.log(arguments); // Output: [1, 2, 3]  
  console.log(arguments.length); // Output: 3  
  console.log(arguments[0]); // Output: 1  
}
```

Handwritten notes: "array like" with an arrow pointing to the function call, and "arguments" circled in red.

```
//String  
const str = "Hello";  
console.log(str); // Output: Hello  
console.log(str.length); // Output: 5  
console.log(str[0]); // Output: H
```

Handwritten notes: "array like" with an arrow pointing to the string variable, and "str", "str.length", and "str[0]" circled in red.

```
// Accessing HTML collection  
var boxes = document.getElementsByClassName('box');  
// Accessing elements in HTML collection using index  
console.log(boxes[0]);  
// Accessing length property of HTML collection  
console.log(boxes.length);
```

Handwritten notes: "boxes" circled in red.

Q. How to **convert** an array-like object into an array?



Methods to
convert an
array-like object
into an array

Array.from()

Spread Syntax (...)

Array.prototype.slice.call()

```
// Example array-like object  
var arrayLike = {0: 'a', 1: 'b', 2: 'c', length: 3};
```

```
// Using Array.from()  
var array1 = Array.from(arrayLike);  
console.log(array1);  
// Output: ['a', 'b', 'c']
```

```
// Using spread syntax (...)  
var array2 = [...arrayLike];  
console.log(array2);  
// Output: ['a', 'b', 'c']
```

```
// Using Array.prototype.slice.call()  
var array3 = Array.prototype.slice.call(arrayLike);  
console.log(array3);  
// Output: ['a', 'b', 'c']
```



Chapter 5: Loops

JS

basics

Q. What is a loop? What are the types of loops in JS?

Q. What is the difference between while and for loops?

Q. What is the difference between while and do-while loops?

Q. What is the difference between break and continue statement?

Q. What is the difference between for and for...of loop in JS?

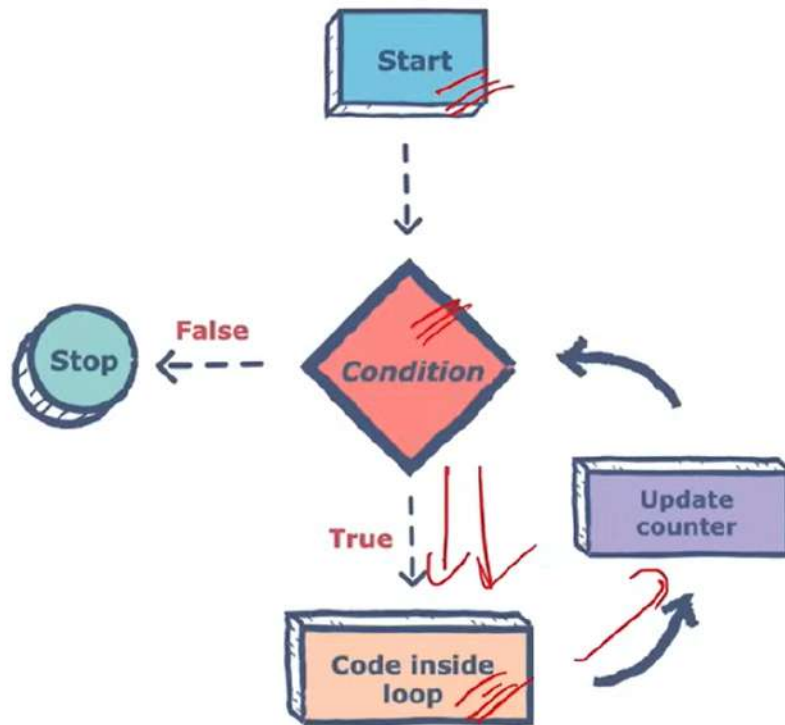
Q. What is the difference between for...of and for...in loop?

Q. What is forEach method? Compare it with for...of and for...in loop?

Q. When to use for...of loop and when to use forEach method in applications?



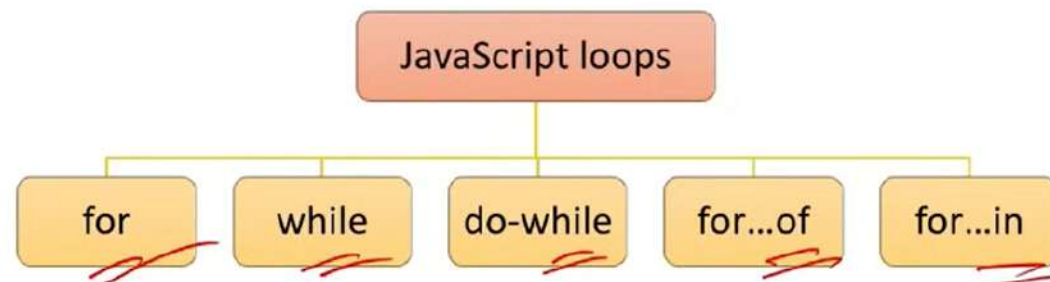
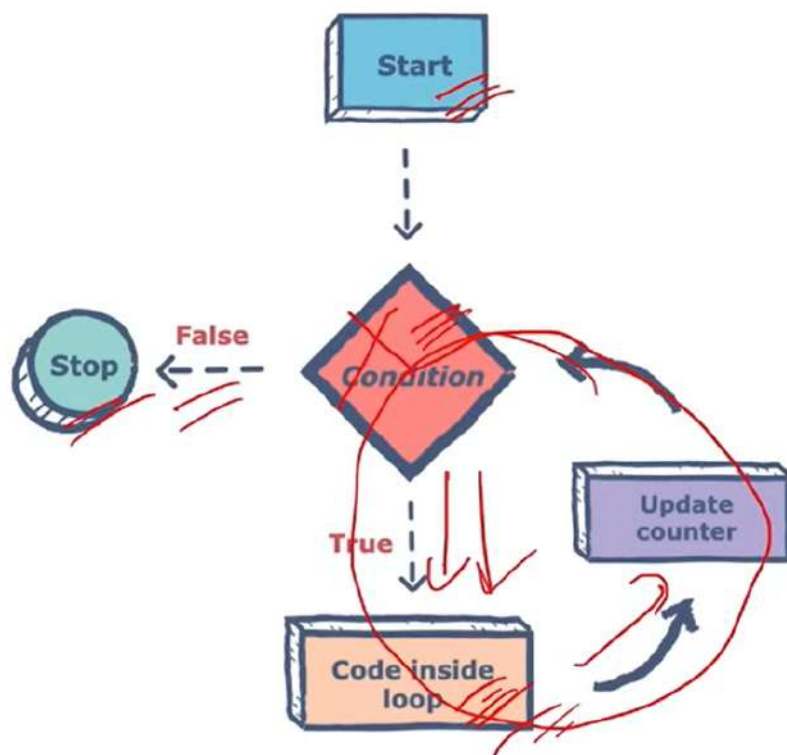
Q. What is a **loop**? What are the **types** of loops in JS? **V. IMP.**



Q. What is a **loop**? What are the **types** of loops in JS? **V. IMP.**



❖ A loop is a programming way to run a piece of **code repeatedly** until a certain condition is met.



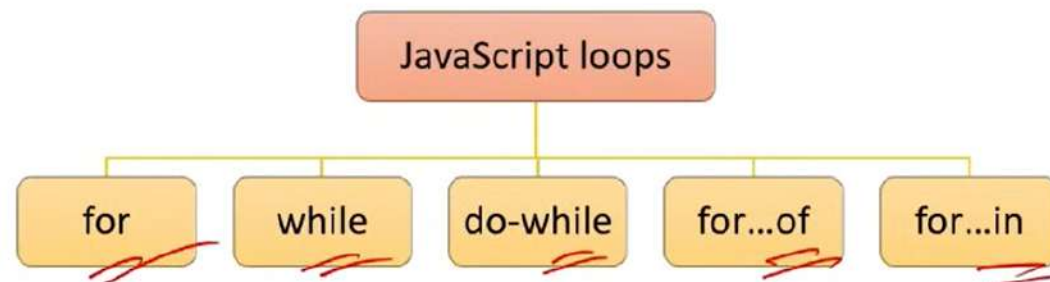
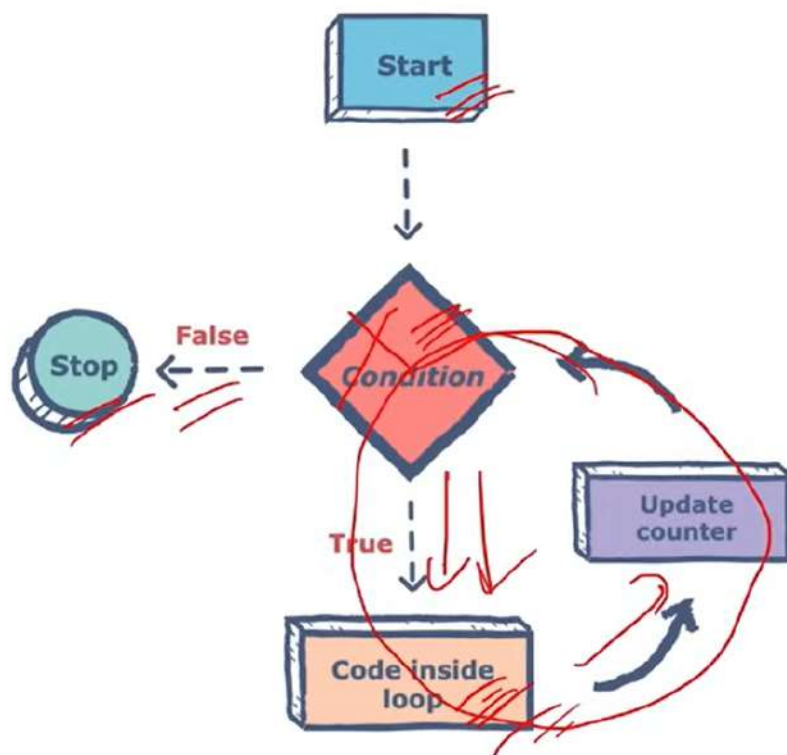
```
//for-loop
for (let i = 0; i < 5; i++) {
    console.log(i);
}
//Output: 0 1 2 3 4
```



Q. What is a **loop**? What are the **types** of loops in JS? **V. IMP.**



❖ A loop is a programming way to run a piece of **code repeatedly** until a certain condition is met.



1. Initialization

2. Condition

3. Increment

```
//for-loop
for (let i = 0; i < 5; i++) {
  console.log(i);
}
//Output: 0 1 2 3 4
```

Red arrows point from the labels '1. Initialization', '2. Condition', and '3. Increment' to the corresponding parts of the for-loop code: 'let i = 0', 'i < 5', and 'i++' respectively.



Q. What is the difference between **while** and **for** loops?



- ❖ For loop allows to iterate a block of code a **specific number** of times.

```
// for loop
for (let i = 0; i < 5; i++) {
  console.log(i);
}
// Output: 0 1 2 3 4
```

```
// while loop
let j = 0;
while (j < 5) {
  console.log(j);
  j++;
}
// Output: 0 1 2 3 4
```

```
// condition only
while ("a" == "a") {
  console.log("Happy");
} break;
//Output: Happy(infinity)
```





Q. What is the difference between **while** and **for** loops?

- ❖ For loop allows to iterate a block of code a **specific number** of times.

```
// for loop
for (let i = 0; i < 5; i++) {
  console.log(i);
}
// Output: 0 1 2 3 4
```

- ❖ While loop execute a block of code while a certain **condition** is true.

```
// while loop
let j = 0;
while (j < 5) {
  console.log(j);
  j++;
}
// Output: 0 1 2 3 4
```

```
// condition only
while ("a" == "a") {
  console.log("Happy");
} break;
//Output: Happy(infinity)
```



Q. What is the difference between **while** and **for** loops?

- ❖ For loop allows to iterate a block of code a **specific number** of times.
- ❖ for loop is better for condition with initialization and with increment because all can be set in just **one line of code**.

```
// for loop
for (let i = 0; i < 5; i++) {
  console.log(i);
}
// Output: 0 1 2 3 4
```

- ❖ While loop execute a block of code while a certain **condition** is true.
- ❖ While loop is better when there is **only condition**, no initialization, no increment.

```
// while loop
let j = 0;
while (j < 5) {
  console.log(j);
  j++;
}
// Output: 0 1 2 3 4
```

```
// condition only
while ("a" == "a") {
  console.log("Happy");
} break;
//Output: Happy(infinity)
```

Q. What is the difference between **while** and **do-while** loops? **V. IMP.**



```
// while loop
let j = 0;

while (j < 5) {
    console.log(j);
    j++;
}
// Output: 0 1 2 3 4
```

```
// do-while loop
let k = 0;

do {
    console.log(k);
    k++;
} while (k > 1);
```



Q. What is the difference between **while** and **do-while** loops? **V. IMP.**



```
// while loop
let j = 0;

while (j < 5) {
    console.log(j);
    j++;
}
// Output: 0 1 2 3 4
```

```
// do-while loop
let k = 0;

do {
    console.log(k);
    k++;
} while (k > 1);
```



Q. What is the difference between **break** and **continue** statement? **V. IMP.**



- ❖ The "break" statement is used to **terminate** the loop.

```
//break statment
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    break;
  }
  console.log(i);
}
//Output: 1 2
```

- ❖ The "continue" statement is used to **skip the current iteration** of the loop and move on to the **next iteration**.

```
//continue statment
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    continue;
  }
  console.log(i);
}
//Output: 1 2 4 5
```


Q. What is the difference between for and for...of loop in JS?



```
let arr = [1, 2, 3];
```

```
// for loop has more code
for (let i = 0; i < arr.length; i++) {
  console.log(arr[i]);
}
//Output: 1 2 3
```

```
// for of is much simpler
for (let val of arr) {
  console.log(val);
}
//Output: 1 2 3
```



Q. What is the difference between for...of and for...in loop? **V. IMP.**



- ❖ for...of loop is used to loop through the values of an object like arrays, strings.
- ❖ It allows you to access each value directly, without having to use an index.
- ❖ for...in loop is used to loop through the properties of an object.
- ❖ It allows you to iterate **over the keys of an object** and access the values associated by using keys as the index.

```
let arr = [1, 2, 3];  
for (let val of arr) {  
  console.log(val);  
}  
//Output: 1 2 3
```

```
// for-in loop  
const person = {  
  name: 'Happy',  
  role: 'Developer'  
};  
  
for (let key in person) {  
  console.log(person[key]);  
}  
//Output: Happy Developer
```



Q. What is forEach method? Compare it with for...of and for...in loop? **V. IMP.**



```
const array = [1, 2, 3];
```

```
//for...of loop
for (let item of array) {
  console.log(item);
}
//Output: 1 2 3
```

```
//forEach method
array.forEach(function(item) {
  console.log(item);
});
//Output: 1 2 3
```



Q. What is forEach method? Compare it with for...of and for...in loop? **V. IMP.**



```
const array = [1, 2, 3];
```

```
//for...of loop
for (let item of array) {
  console.log(item);
}
//Output: 1 2 3
```

```
//forEach method
array.forEach(function(item) {
  console.log(item);
});
//Output: 1 2 3
```



Q. When to use for...of loop and when to use forEach method in applications? **V. IMP.**



```
const array = [1, 2, 3];
```

```
// for-of loop
for (let item of array) {
  console.log(item);

  if (item === 2) {
    break;
  }
}

//Output: 1 2
```

```
//forEach method
array.forEach(function (item) {
  console.log(item);
  if (item === 2) {
    break;
  }
});
// Error: Illegal break statement
```



Q. When to use for...of loop and when to use forEach method in applications? **V. IMP.**



- ❖ for...of loop is suitable when you need **more control over the loop**, such as using break statement or continue statement inside.

```
const array = [1, 2, 3];
```

```
// for-of loop
for (let item of array) {
  console.log(item);

  if (item === 2) {
    break;
  }
}

//Output: 1 2
```

```
//forEach method
array.forEach(function (item) {
  console.log(item);
  if (item === 2) {
    break;
  }
});

// Error: Illegal break statement
```



Q. When to use for...of loop and when to use forEach method in applications? **V. IMP.**



- ❖ for...of loop is suitable when you need **more control over the loop**, such as using break statement or continue statement inside.

- ❖ forEach method **iterate over each element** of the array and perform some action on each element.

```
const array = [1, 2, 3];
```

```
// for-of loop
for (let item of array) {
  console.log(item);

  if (item === 2) {
    break;
  }
}

//Output: 1 2
```

```
//forEach method
array.forEach(function (item) {
  console.log(item);
  if (item === 2) {
    break;
  }
});

// Error: Illegal break statement
```



Chapter 6: Functions



Q. What are Functions in JS? What are the types of function?

basics

Q. What is the difference between named and anonymous functions? When to use what in applications?

Q. What is function expression in JS?

Q. What are Arrow Functions in JS? What is it use?

basics

Q. What are Callback Functions? What is it use?

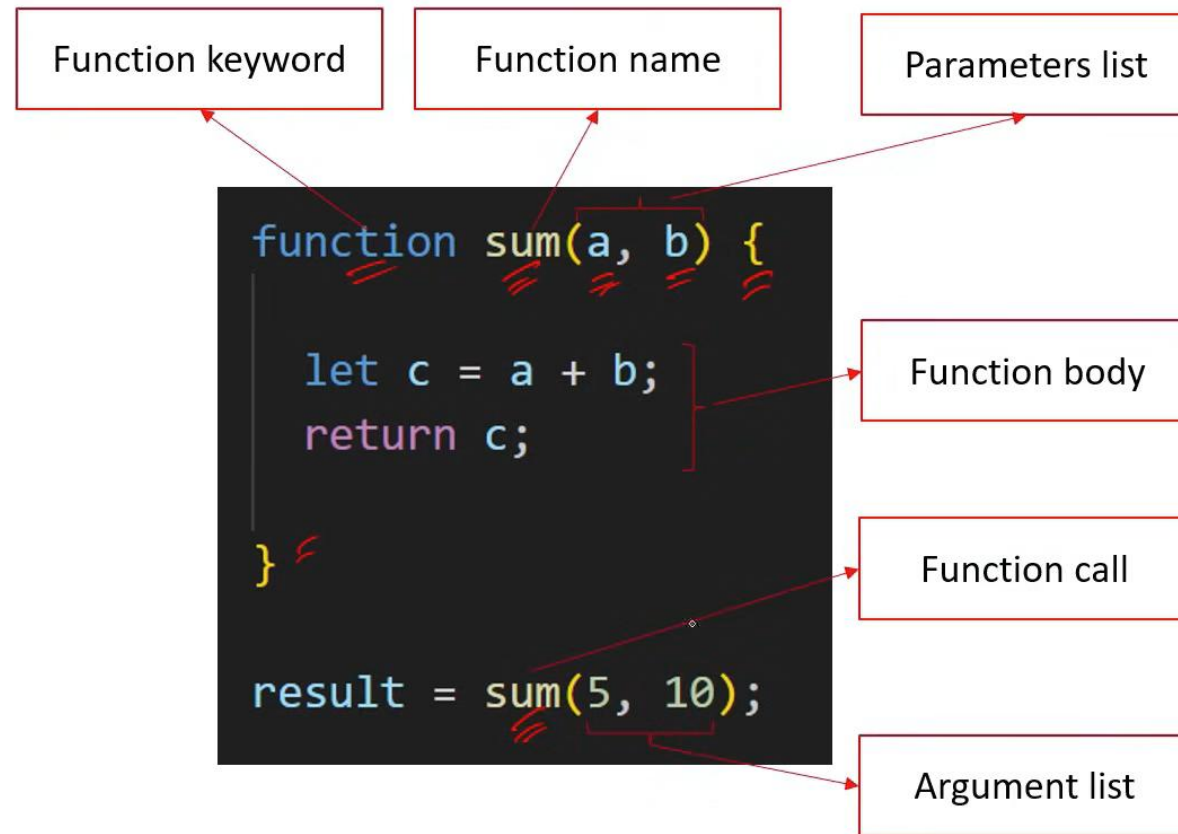
Q. What is Higher-order function In JS?

Q. What is the difference between arguments and parameters?

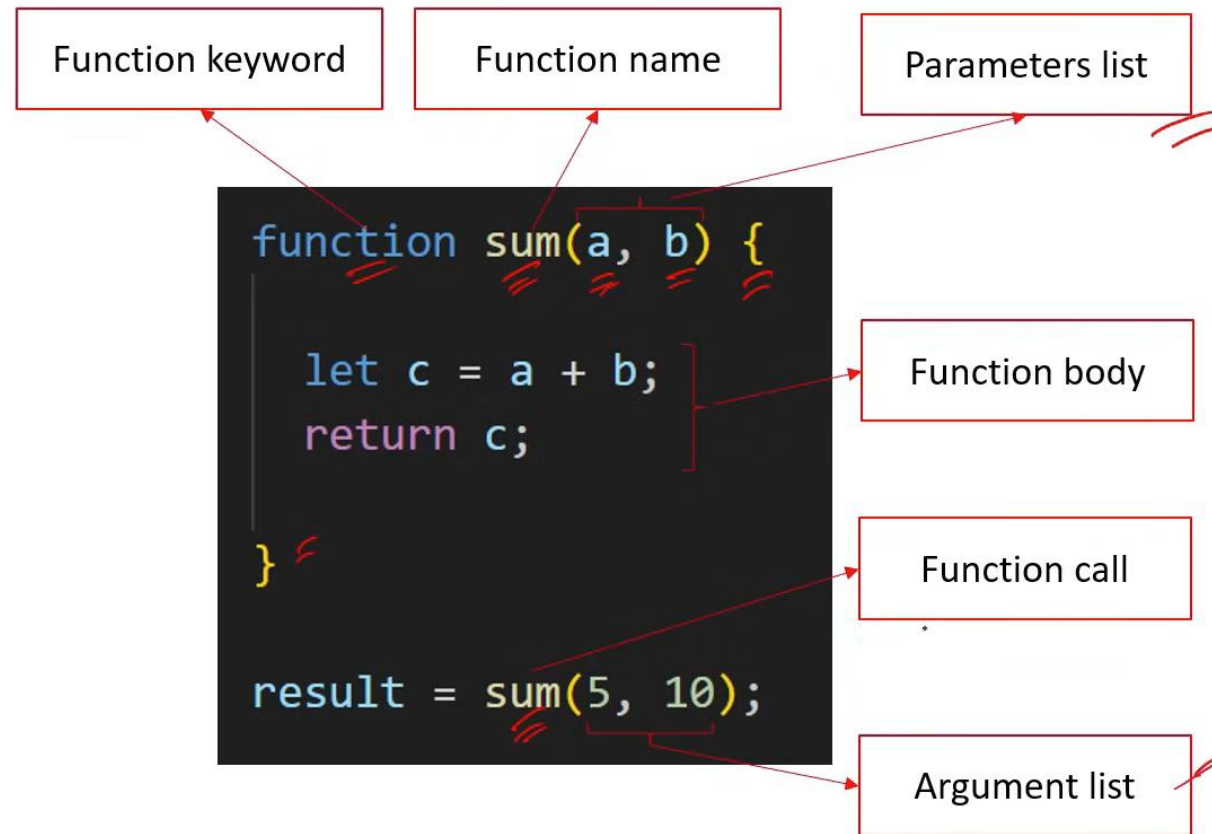
Q. In how many ways can you pass arguments to a function?



Q. What are **Functions** in JS? What are the **types** of function? **V. IMP.**



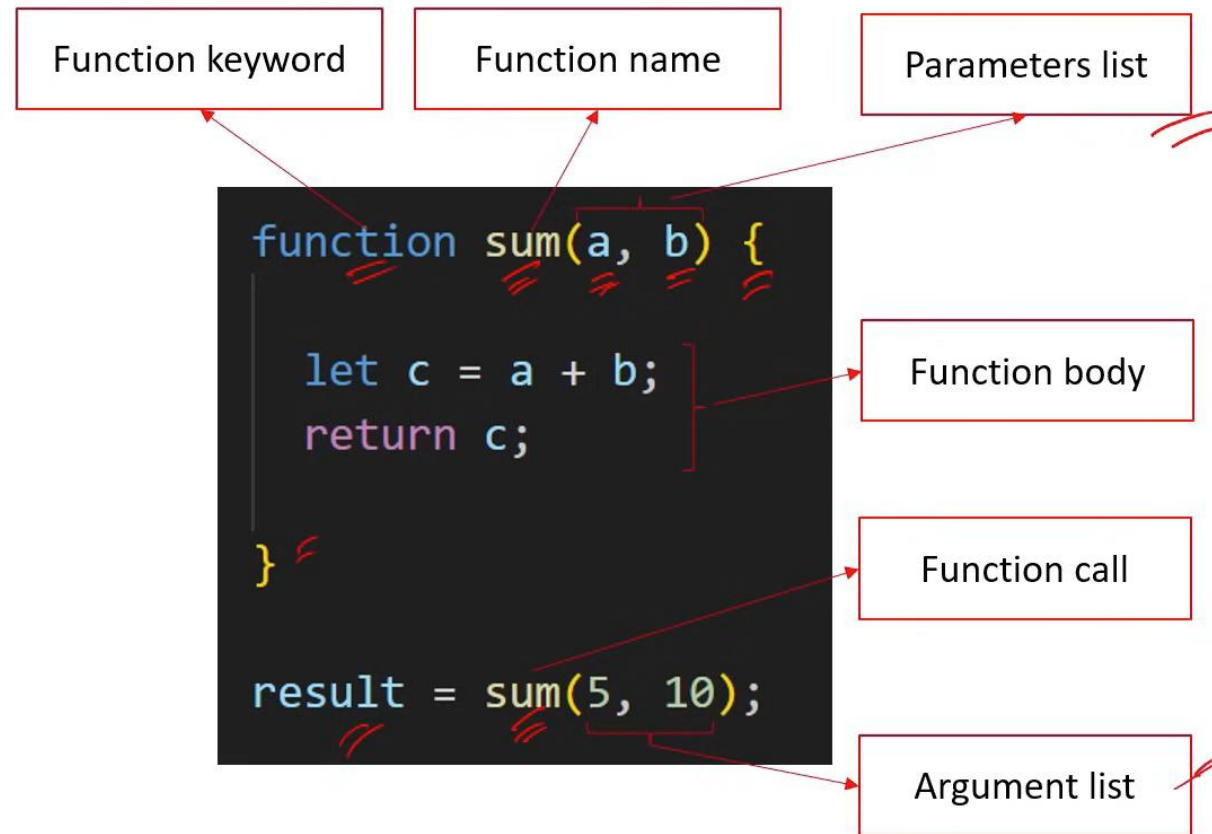
Q. What are **Functions** in JS? What are the **types** of function? **V. IMP.**



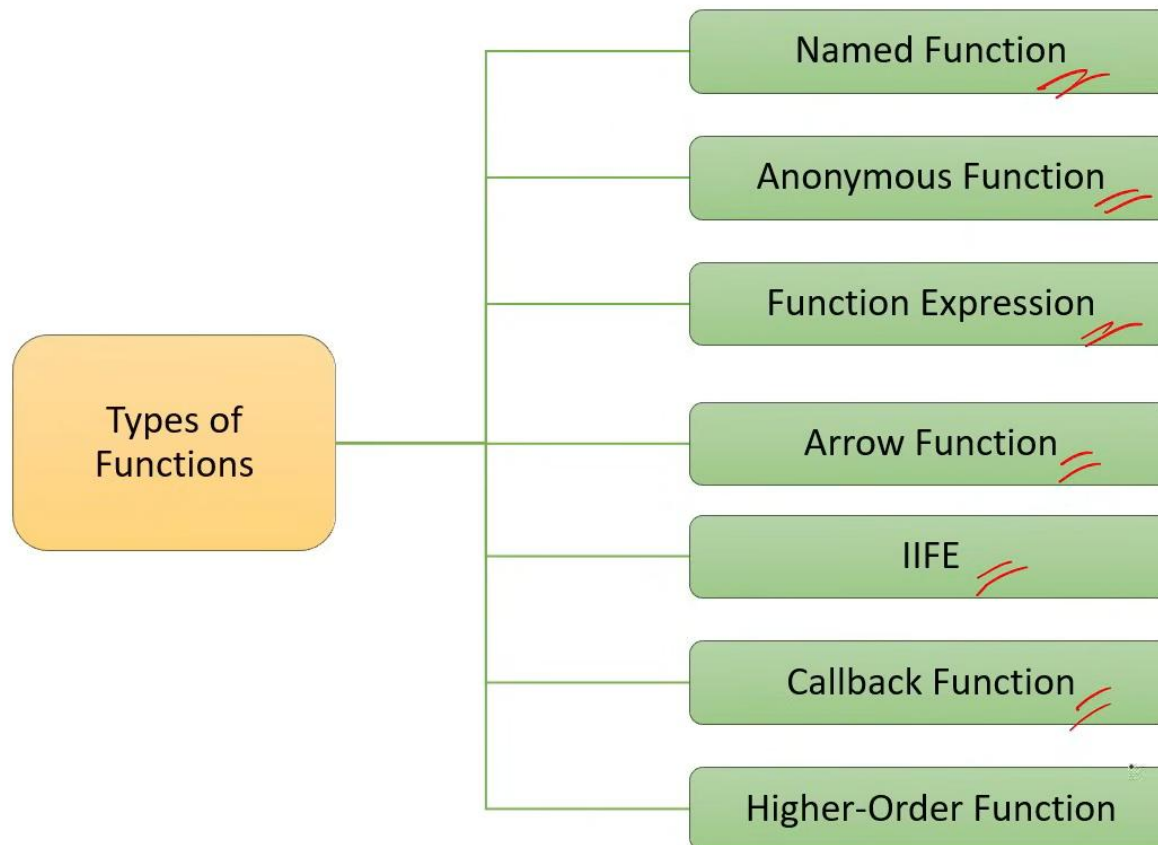
Q. What are **Functions** in JS? What are the **types** of function? **V. IMP.**



- ❖ A function is a **reusable block of code** that performs a specific task.



Q. What are **Functions** in JS? What are the **types** of function? **V. IMP.**



Q. What is the difference between named and anonymous functions?
When to use what in applications?



❖ Named functions have a **name identifier**

```
//Named Function
//Function Declaration

function sum(a, b) {
  return a + b;
};

console.log(add(5, 3));
//Output: 8
```

❖ Anonymous functions **do not have a name identifier** and cannot be referenced directly by name.

```
// Anonymous function

console.log(function(a, b) {
  return a * b;
})(4, 5);
// Output: 20
```



Q. What is the difference between named and anonymous functions? When to use what in applications?



- ❖ Named functions have a **name identifier**

```
//Named Function
//Function Declaration

function sum(a, b) {
  return a + b;
};

console.log(add(5, 3));
//Output: 8
```

- ❖ Use named functions for **big and complex** logics.
- ❖ Use when you want to **reuse** one function at multiple places.

- ❖ Anonymous functions **do not have a name identifier** and cannot be referenced directly by name.

```
// Anonymous function

console.log(function(a, b) {
  return a * b;
})(4, 5);
// Output: 20
```

- ❖ Use anonymous functions for **small logics**.
- ❖ Use when want to use a function in a **single place**.



Q. What is function expression in JS?



- ❖ A function expression is a way to define a function by assigning it to a variable.

//Anonymous Function Expression

```
const add = function(a, b) {  
  return a + b;  
};
```

```
console.log(add(5, 3));  
//Output: 8
```

//Named Function Expression

```
const add = function sum(a, b) {  
  return a + b;  
};
```

```
console.log(sumadd(5, 3));  
//Output: 8
```



Q. What are Arrow Functions in JS? What is it use? **V. IMP.**



Parameters list

Function body

```
//Traditional approach
```

```
function add(x, y)
{
  return x + y;
}
```

```
console.log(add(5, 3));
//output : 8
```

```
//Arrow function
```

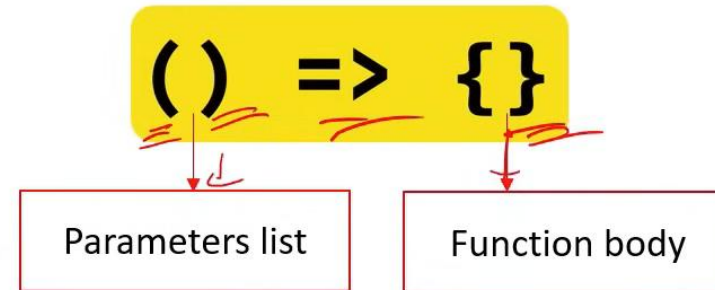
```
const add = (x, y) => x + y;
console.log(add(5, 3));
//output : 8
```



Q. What are Arrow Functions in JS? What is it use? **V. IMP.**



- ❖ Arrow functions, also known as fat arrow functions, is a **simpler and shorter** way for defining functions in JavaScript.



//Traditional approach

```
function add(x, y)
{
  return x + y;
}
```

```
console.log(add(5, 3));
//output : 8
```

//Arrow function

```
const add = (x, y) => x + y;
console.log(add(5, 3));
//output : 8
```

Q. What are Callback Functions? What is it use? **V. IMP.**



Callback function

```
function add(x, y) {  
  return x + y;  
}  
  
let a = 3, b = 5;  
let result = add(a, b)  
  
console.log(result);  
//Output: 8
```

```
function display(x, y, operation) {  
  var result = operation(x, y);  
  console.log(result);  
}  
  
display(10, 5, add);
```



Q. What are Callback Functions? What is it use? **V. IMP.**



- ❖ A callback function is a function that is passed as an argument to another function.

```
function add(x, y) {  
  return x + y;  
}  
  
let a = 3, b = 5;  
let result = add(a, b)  
  
console.log(result);  
//Output: 8
```

```
function display(x, y, operation) {  
  var result = operation(x, y);  
  console.log(result);  
}  
  
display(10, 5, add);
```

Callback function



Q. What are Callback Functions? What is it use? **V. IMP.**



- ❖ A callback function is a function that is **passed as an argument** to another function.

```
function add(x, y) {  
  return x + y;  
}  
  
let a = 3, b = 5;  
let result = add(a, b);  
  
console.log(result);  
//Output: 8
```

Higher-order function

Callback function

```
function display(x, y, operation) {  
  var result = operation(x, y);  
  console.log(result);  
}  
  
display(10, 5, add);
```


Q. What are Callback Functions? What is it use? **V. IMP.**



- ❖ A callback function is a function that is **passed as an argument** to another function.

```
function add(x, y) {  
  return x + y;  
}  
  
let a = 3, b = 5;  
let result = add(a, b);  
  
console.log(result);  
//Output: 8
```

Higher-order function

Callback function

```
function display(x, y, operation) {  
  var result = operation(x, y);  
  console.log(result);  
}  
  
display(10, 5, add);  
display(10, 5, multiply);  
display(10, 5, subtract);  
display(10, 5, divide);
```

Q. What is Higher-order function In JS?



- ❖ A Higher order function:
- 1. Take one or more functions as **arguments**(callback function) OR
- 2. **Return** a function as a result

```
//Take one or more functions
//as arguments
function hof(func) {
  func();
}

hof(sayHello);

function sayHello() {
  console.log("Hello!");
}

// Output: "Hello!"
```

Callback function

High-order function

```
//Return a function as a result
function createAdder(number) {
  return function(value) {
    return value + number;
  };
}
```



Q. What is Higher-order function In JS?



- ❖ A Higher order function:
- 1. Take one or more functions as **arguments**(callback function) OR
- 2. **Return** a function as a result

```
//Take one or more functions
//as arguments
function hof(func) {
  func();
}

hof(sayHello);

function sayHello() {
  console.log("Hello!");
}

// Output: "Hello!"
```

Callback function

High-order function

```
//Return a function as a result
function createAdder(number) {
  return function(value) {
    return value + number;
  };
}

const addFive = createAdder(5);

console.log(addFive(2));

// Output: 7
```



Q. What is the difference between arguments and parameters?



- ❖ Parameters are the **placeholders** defined in the function declaration.

```
//a and b are parameters  
function add(a, b) {  
  console.log(a + b);  
}
```

- ❖ Arguments are the **actual values passed** to a function when it is invoked or called.

```
add(3, 4);  
// 3 and 4 are arguments
```



Q. What is the difference between arguments and parameters?



- ❖ Parameters are the **placeholders** defined in the function declaration.

```
//a and b are parameters  
function add(a, b) {  
  console.log(a + b);  
}
```

- ❖ Arguments are the **actual values passed** to a function when it is invoked or called.

```
add(3, 4);  
// 3 and 4 are arguments
```



Q. In how many ways can you pass arguments to a function?



Ways you can pass arguments to a function

Positional Arguments

```
//Positional Arguments
function add(a, b) {
  console.log(a + b);
}
```

```
add(3, 4);
// Output: 7
```

Named Arguments

```
//Named Arguments
var person = {
  name: "Happy",
  role: "Developer"
};

function greet(person) {
  console.log(person.name +
    " " + person.role);
}

greet(person);
// Output: Happy Developer
```

Arguments Object



Q. In how many ways can you pass arguments to a function?



Ways you can pass arguments to a function

Positional Arguments

```
//Positional Arguments
function add(a, b) {
  console.log(a + b);
}

add(3, 4);
// Output: 7
```

Named Arguments

```
//Named Arguments
var person = {
  name: "Happy",
  role: "Developer"
};

function greet(person) {
  console.log(person.name +
    " " + person.role);
}

greet(person);
// Output: Happy Developer
```

Arguments Object

```
sum(1, 2, 3);

//Argument Object
function sum(){
  console.log(arguments[0]);
  // Output: 1
  console.log(arguments[1]);
  // Output: 2
  console.log(arguments[2]);
  // Output: 3
  console.log(arguments.length);
  // Output: 3
}
```



Q. How do you use **default parameters** in a function?



- ❖ In JavaScript, default parameters allow you to specify **default values** for function parameters.

```
//Function with default parameter value  
function greet(name = "Happy") {  
    console.log("Hello, " + name + "!");  
}
```

```
greet();  
// Output: Hello, Happy!
```

```
greet("Amit");  
// Output: Hello, Amit!
```



Q. What is the use of event handling in JS? **V. IMP.**



```
<button id="myButton">Click me</button>

// Get a reference to the button element
const button = document.getElementById('myButton');

// Add an event listener for the 'click' event
button.addEventListener('click', function() {
    alert('Button clicked!');
});
```



Q. What is the use of event handling in JS? **V. IMP.**



```
<button id="myButton">Click me</button>

// Get a reference to the button element
const button = document.getElementById('myButton');

// Add an event listener for the 'click' event
button.addEventListener('click', function() {
    alert('Button clicked!');
});
```

Event

Callback function



Q. What is the use of event handling in JS? **V. IMP.**



- ❖ Event handling is the process of responding to user actions in a web page.
- ❖ The **addEventListener** method of Javascript allows to attach an **event name** and with the **function** you want to perform on that event.

```
<button id="myButton">Click me</button>

// Get a reference to the button element
const button = document.getElementById('myButton');

// Add an event listener for the 'click' event
button.addEventListener('click', function() {
    alert('Button clicked!');
});
```

Event

Callback function

Click Event: addEventListener('click', handler)

Mouseover Event: addEventListener('mouseover', handler)

Keydown Event: addEventListener('keydown', handler)

Keyup Event: addEventListener('keyup', handler)

Submit Event: addEventListener('submit', handler)

Focus Event: addEventListener('focus', handler)

Blur Event: addEventListener('blur', handler)

Change Event: addEventListener('change', handler)

Load Event: addEventListener('load', handler)

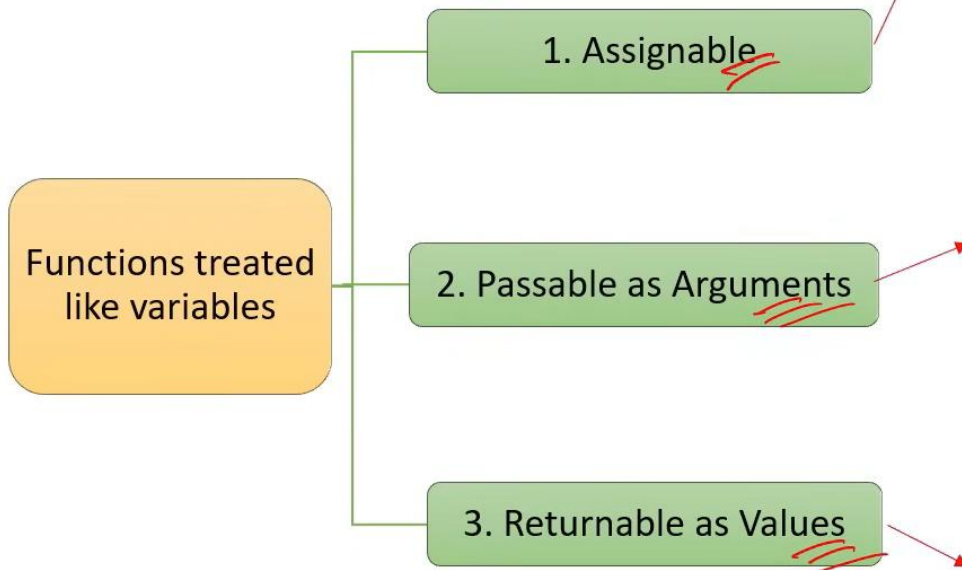
Resize Event: addEventListener('resize', handler)



Q. What are First-Class functions in JS?



- ❖ A programming language is said to have First-class functions if functions in that language are treated like **other variables**.



```
// 1. Assigning function like a variable
const myFunction = function () {
  console.log("Interview, Happy!");
};
myFunction(); // Output: "Interview, Happy!"
```

```
function double(number) {
  return number * 2;
}

// 2. Passing function as an argument like a variable
function performOperation(double, value) {
  return double(value);
}
console.log(performOperation(double, 5)); // Output: 10
```

```
// 3. A function that returns another function
function createSimpleFunction() {
  return function () {
    console.log("I am from return function.");
  };
}

const simpleFunction = createSimpleFunction();
simpleFunction(); // Output: "I am from return function."
```



Q. What are Pure and Impure functions in JS?



1. A pure function is a function that always produces the same output for the same input.

```
// Pure function
function add(a, b) {
  return a + b;
}

console.log(add(3, 5));
// Output: 8

console.log(add(3, 5));
// Same Output: 8
```

1. An impure function, can produce different outputs for the same input.

```
// Impure function
let total = 0; → 5

function addToTotal(value) {
  total += value;
  return total;
}

console.log(addToTotal(5));
// Output: 5

console.log(addToTotal(5));
// Not same output: 10
```

Handwritten notes: A red arrow points from the initial `total = 0` to a circled `5`. Another red arrow points from the `total += value` line to the calculation `0 + 5 = 5`. A third red arrow points from the `return total` line to the calculation `5 + 5 = 10`.

Q. What are Pure and Impure functions in JS?



1. A pure function is a function that always produces the same output for the same input.
2. Pure functions cannot modify the state.

```
// Pure function
function add(a, b) {
  return a + b;
}

console.log(add(3, 5));
// Output: 8

console.log(add(3, 5));
// Same Output: 8
```

1. An impure function, can produce different outputs for the same input.
2. Impure functions can modify the state.

```
// Impure function
let total = 0; → 5

function addToTotal(value) {
  total += value;
  return total;
}

console.log(addToTotal(5));
// Output: 5

console.log(addToTotal(5));
// Not same output: 10
```

Handwritten notes:
 - A red arrow points from the initial `total = 0` to a circled `5` above the function call.
 - Next to the function definition, `total += value;` is annotated with `0 + 5 = 5`.
 - Below the second function call, `5 + 5 = 10` is written, indicating the state change.

Q. What are Pure and Impure functions in JS?



1. A pure function is a function that always produces the same output for the same input.
2. Pure functions cannot modify the state.
3. Pure functions cannot have side effects.

```
// Pure function
function add(a, b) {
  return a + b;
}

console.log(add(3, 5));
// Output: 8

console.log(add(3, 5));
// Same Output: 8
```

1. An impure function, can produce different outputs for the same input.
2. Impure functions can modify the state.
3. Impure functions can have side effects.

```
// Impure function
let total = 0;

function addToTotal(value) {
  total += value;
  return total;
}

console.log(addToTotal(5));
// Output: 5

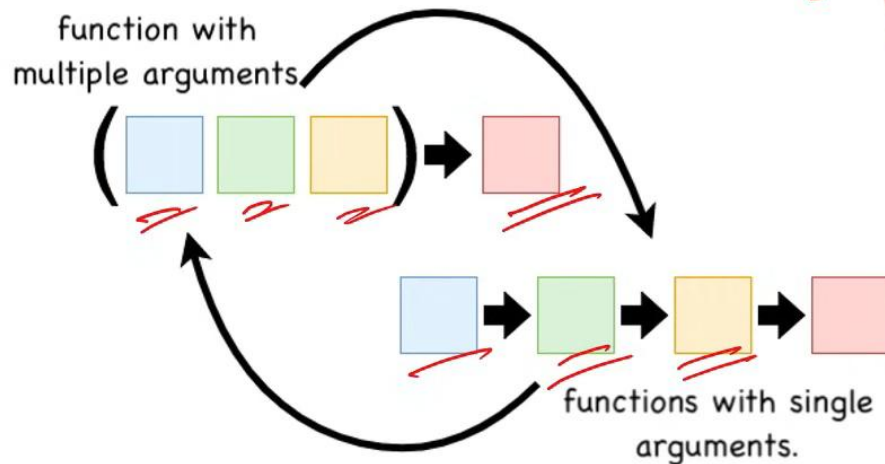
console.log(addToTotal(5));
// Not same output: 10
```

Handwritten annotations: A red circle around `total = 0` with an arrow pointing to a circled `5`. Another red circle around `addToTotal(5)` in the first call, with an arrow pointing to `total += value`. A third red circle around `addToTotal(5)` in the second call, with an arrow pointing to `total`. Red text next to the second call shows $0 + 5 = 5$ and $5 + 5 = 10$.

Q. What is Function Currying in JS?



- ❖ Currying in JavaScript transforms a function with multiple arguments into a **nested series of functions**, each taking a single argument.



```
// Regular function that takes two arguments  
// and returns their product  
function multiply(a, b) {  
  return a * b;  
}
```

```
// Curried version of the multiply function  
function curriedMultiply(a) {  
  return function (b) {  
    return a * b;  
  };  
}
```

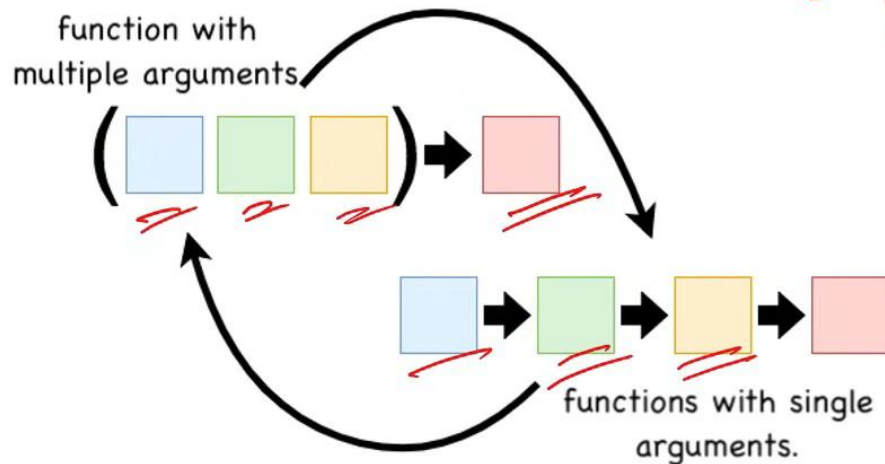
```
// Create a specialized function for doubling a number  
const double = curriedMultiply(2);  
console.log(double(5));  
// Output: 10 (2 * 5)
```



Q. What is Function Currying in JS?



- ❖ Currying in JavaScript transforms a function with multiple arguments into **a nested series of functions**, each taking a single argument.
- ❖ Advantage : **Reusability, modularity, and specialization**. Big, complex functions with multiple arguments can be broken down into small, reusable functions with fewer arguments.



```
// Regular function that takes two arguments
// and returns their product
function multiply(a, b) {
  return a * b;
}

// Curried version of the multiply function
function curriedMultiply(a) {
  return function (b) {
    return a * b;
  };
}

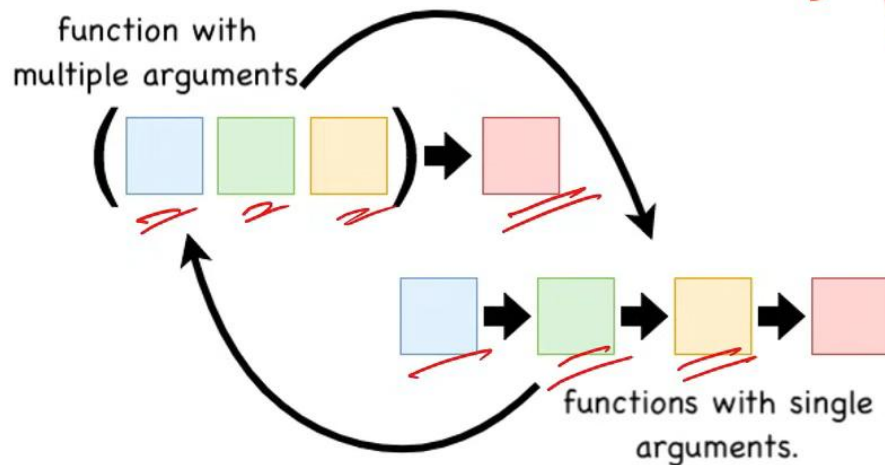
// Create a specialized function for doubling a number
const double = curriedMultiply(2);
console.log(double(5));
// Output: 10 (2 * 5)
```



Q. What is Function Currying in JS?



- ❖ Currying in JavaScript transforms a function with multiple arguments into a nested series of functions, each taking a single argument.
- ❖ Advantage : **Reusability, modularity, and specialization**. Big, complex functions with multiple arguments can be broken down into small, reusable functions with fewer arguments.



```
// Regular function that takes two arguments
// and returns their product
function multiply(a, b) {
  return a * b;
}

// Curried version of the multiply function
function curriedMultiply(a) {
  return function (b) {
    return a * b;
  };
}

// Create a specialized function for doubling a number
const double = curriedMultiply(2);
console.log(double(5));
// Output: 10 (2 * 5)

// Create a specialized function for tripling a number
const triple = curriedMultiply(3);
console.log(triple(5));
// Output: 15 (3 * 5)
```





Sponsored ⓘ • 1 of 2 officemaster.in

⏮

⏪

🔊

0:00 / 1:13

⌵

⏩

⏭

⚙

⛶

Q. What are **call**, **apply** and **bind** methods in JS?



- ❖ call, apply, and bind are three methods in JavaScript that are used to work with functions and **control how they are invoked** and what context they operate in.

- ❖ These methods provide a way to manipulate the **this value** and pass arguments to functions.

```
// Defining a function that uses the "this" context and an argument
function sayHello(message) {
  console.log(`${message}, ${this.name}!`);
}
const person = { name: 'Happy' };
```

```
// 1. call - Using the "call" method to invoke the function
// with a specific context and argument
sayHello.call(person, 'Hello');
// Output: "Hello, Happy!"
```

```
// 2. apply - Using the "apply" method to invoke the function
// with a specific context and an array of arguments
sayHello.apply(person, ['Hi']);
// Output: "Hi, Happy!"
```

```
// 3. bind - Using the "bind" method to create a new function
// with a specific context (not invoking it immediately)
const greetPerson = sayHello.bind(person);
greetPerson('Greetings');
// Output: "Greetings, Happy!"
```



Chapter 7: Strings



Q. What is a String?

Q. What are template literals and string interpolation in strings?

Q. What is the difference between single quotes ('), double quotes (") & backticks (`)?

Q. What are some important string operations in JS?

basics

Q. What is string immutability?

Q. In how many ways you can concatenate strings?



Q. What is a **String**?



❖ A string is a **data type** used to **store and manipulate data**.

```
// Single quotes ('')  
var str1 = 'Hello';
```



Q. What are template literals and string interpolation in strings? **V. IMP.**



``${Template} Literal``

variable fixed
Run

```
// Backticks (`)
//Template literals with string interpolation
var myname = "Happy";
var str3 = `Hello ${myname}!`;
console.log(str3);
// Output: Hello Happy!
```



Q. What are template literals and string interpolation in strings? **V. IMP.**



- ❖ A template literal, also known as a template string, is a feature introduced in ECMAScript 2015 (ES6) for **string interpolation** and **multiline strings** in JavaScript.

``${Template} Literal``

variable fixed
Run

```
// Backticks (`)
//Template literals with string interpolation
var myname = "Happy";
var str3 = `Hello ${myname}!`;
console.log(str3);
// Output: Hello Happy!
```

$\$$ ← variable

```
// Backticks (`)
//Template literals for multiline strings
var multilineStr = `
This is a
multiline string.
`;
```



Q. What are template literals and string interpolation in strings? **V. IMP.**



- ❖ A template literal, also known as a template string, is a feature introduced in ECMAScript 2015 (ES6) for **string interpolation** and **multiline strings** in JavaScript.

``${Template} Literal``

variable fixed
Run

```
// Backticks (`)
//Template literals with string interpolation
var myname = "Happy";
var str3 = `Hello ${myname}!`;
console.log(str3);
// Output: Hello Happy!
```

$\$ \leftarrow$ variable }

```
// Backticks (`)
//Template literals for multiline strings
var multilineStr = `
This is a
multiline string.
`;
```



Q. What is the difference between single quotes ('), double quotes (") & backticks (`)?



```
// Single quotes ('')  
var str1 = 'Hello';
```

```
// Double quotes (")  
var str2 = "World";
```

```
// Backticks (`)  
// Template literals with string interpolation  
var myname = "Happy";  
var str3 = `Hello ${myname}!`;  
console.log(str3);  
// Output: Hello Happy!
```



Q. What is the difference between single quotes (`'`), double quotes (`"`) & backticks (```)?



```
// Single quotes ('')  
var str1 = 'Hello';
```

```
// Double quotes (")  
var str2 = "World";
```

```
// Backticks (`)  
//Template literals with string interpolation  
var myname = "Happy";  
var str3 = `Hello ${myname}!`;   
console.log(str3);  
// Output: Hello Happy!
```

```
// Backticks (`)  
//Template literals for multiline strings  
var multilineStr = `  
This is a  
multiline string.  
`;  
`;
```



Q. What are some important **string operations** in JS?



JavaScript String Methods

JS

substr()	indexOf()	trim()
substring()	includes()	charAt()
replace()	slice()	valueOf()
search()	concat()	split()
toLocaleLowerCase()	lastIndexOf()	toString()
toLocaleUpperCase()	charCodeAt()	match()



Q. What are some important **string operations** in JS?



```
//Add multiple string
let str1 = "Hello";
let str2 = "World";
let result = str1 + " " + str2;
console.log(result);
// Output: Hello World
```

```
// Using concat() method
let result2 = str1.concat(" ", str2);
console.log(result2);
// Output: Hello World
```

```
//Extract a portion of a string
let subString = result.substring(6, 11);
console.log(subString);
// Output: World
```

```
//Retrieve the length of a string
console.log(result.length);
// Output: 11
```



Q. What are some important **string operations** in JS?



```
//Add multiple string
let str1 = "Hello";
let str2 = "World";
let result = str1 + " " + str2;
console.log(result);
// Output: Hello World
```

```
// Using concat() method
let result2 = str1.concat(" ", str2);
console.log(result2);
// Output: Hello World
```

```
//Extract a portion of a string
let subString = result.substring(6, 11);
console.log(subString);
// Output: World
```

```
//Retrieve the length of a string
console.log(result.length);
// Output: 11
```

```
//Convert a string to uppercase or lowercase
console.log(result.toUpperCase());
// Output: HELLO WORLD
console.log(result.toLowerCase());
// Output: hello world
```

```
//Split a string into an array of substrings
//based on a delimiter
let arr = result.split(" ");
console.log(arr);
// Output: ["Hello", "World"]
```

```
//Replace occurrences of a substring within a string
console.log(result.replace("World", "JavaScript"));
// Output: Hello JavaScript
```



Q. What are some important **string operations** in JS?



```
//Add multiple string
let str1 = "Hello";
let str2 = "World";
let result = str1 + " " + str2;
console.log(result);
// Output: Hello World
```

```
// Using concat() method
let result2 = str1.concat(" ", str2);
console.log(result2);
// Output: Hello World
```

```
//Extract a portion of a string
let subString = result.substring(6, 11);
console.log(subString);
// Output: World
```

```
//Retrieve the length of a string
console.log(result.length);
// Output: 11
```

```
//Convert a string to uppercase or lowercase
console.log(result.toUpperCase());
// Output: HELLO WORLD
console.log(result.toLowerCase());
// Output: hello world
```

```
//Split a string into an array of substrings
//based on a delimiter
let arr = result.split(" ");
console.log(arr);
// Output: ["Hello", "World"]
```

```
//Replace occurrences of a substring within a string
console.log(result.replace("World", "JavaScript"));
// Output: Hello JavaScript
```

```
//Remove leading and trailing whitespace
let str = " Hello World ";
let trimmedStr = str.trim();
console.log(trimmedStr);
// Output: Hello World
```



Q. What are some important **string operations** in JS?



```
//Add multiple string
let str1 = "Hello";
let str2 = "World";
let result = str1 + " " + str2;
console.log(result);
// Output: Hello World
```

```
// Using concat() method
let result2 = str1.concat(" ", str2);
console.log(result2);
// Output: Hello World
```

```
//Extract a portion of a string
let subString = result.substring(6, 11);
console.log(subString);
// Output: World
```

```
//Retrieve the length of a string
console.log(result.length);
// Output: 11
```

```
//Convert a string to uppercase or lowercase
console.log(result.toUpperCase());
// Output: HELLO WORLD
console.log(result.toLowerCase());
// Output: hello world
```

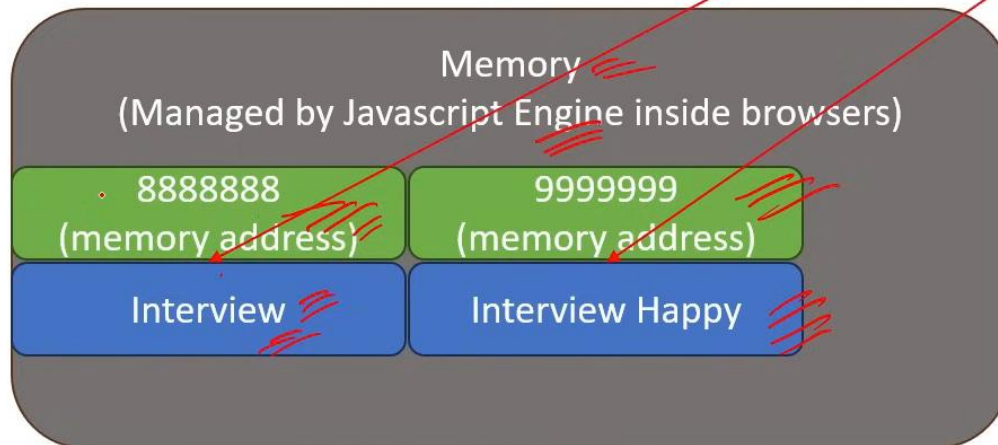
```
//Split a string into an array of substrings
//based on a delimiter
let arr = result.split(" ");
console.log(arr);
// Output: ["Hello", "World"]
```

```
//Replace occurrences of a substring within a string
console.log(result.replace("World", "JavaScript"));
// Output: Hello JavaScript
```

```
//Remove leading and trailing whitespace
let str = " Hello World ";
let trimmedStr = str.trim();
console.log(trimmedStr);
// Output: Hello World
```



Q. What is **string immutability**? **V. IMP.**



```
var str = 'Interview';  
// Creates a new string  
str = str + ' Happy';
```



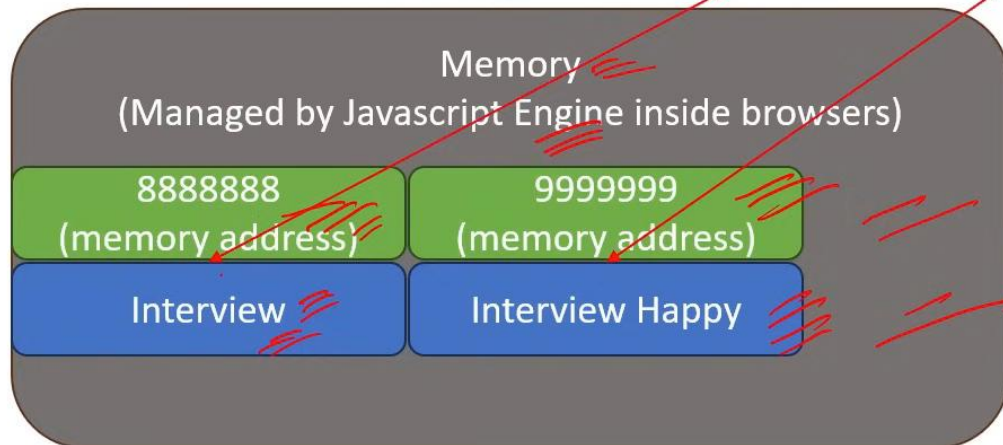
Q. What is **string immutability**? V. IMP.



- ❖ Strings in JavaScript are considered **immutable** because you **cannot modify** the contents of an existing string directly.



```
var str = 'Interview';  
// Creates a new string  
str = str + ' Happy';
```



Q. In how many ways you can concatenate strings?



Ways to concatenate strings

+ Operator

Concat() method

Template literals

Join() method

```
let s1 = 'Hello';  
let s2 = 'World';
```

```
// + operator  
let r1 = s1 + s2;  
console.log(r1);  
// Output: HelloWorld
```

```
// concat() method  
let r2 = s1.concat(s2);  
console.log(r2);  
// Output: HelloWorld
```

```
// template literals  
let r3 = `${s1} ${s2}`;  
console.log(r3);  
// Output: Hello World
```

```
// join() method  
let strings = [s1, s2];  
let r4 = strings.join(' ');  
console.log(r4);  
// Output: Hello World
```



Chapter 8: DOM



Q. What is DOM? What is the difference between HTML and DOM?

basics

Q. How do you select, modify, create and remove DOM elements?

Q. What are selectors in JS?

basics

Q. What is the difference between getElementById, getElementsByClassName and getElementsByTagName?

Q. What is the difference between querySelector() and querySelectorAll()?

Q. What are the methods to modify elements properties and attributes?

Q. What is the difference between innerHTML and textContent?

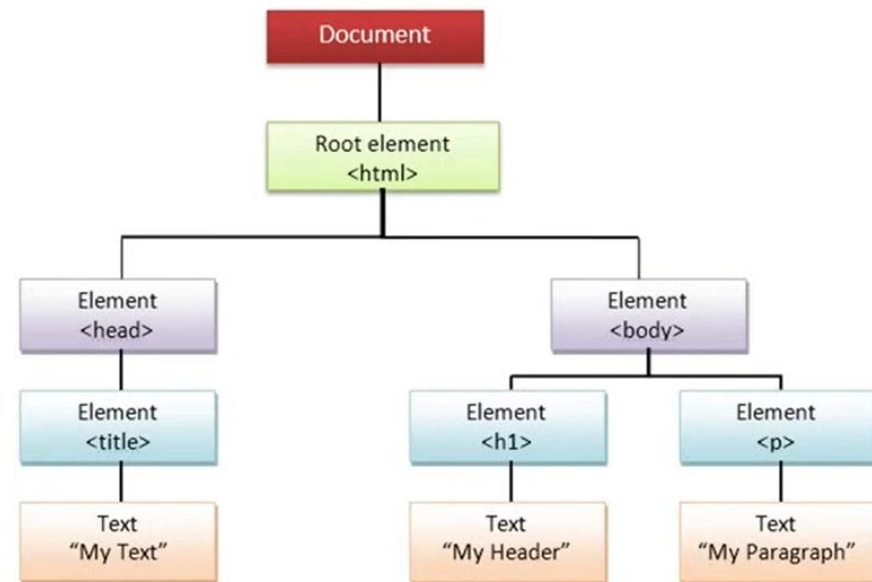


Q. What is **DOM**? What is the difference between **HTML** and the **DOM**? **V. IMP.**



```
<!DOCTYPE html>
<html>
  <head>
    <title>My Text </title>
  </head>
  <body>
    <h1>
      My Header
    </h1>
    <p>My Paragraph </p>
  </body>
</html>
```

Static HTML



DOM Tree(Real)

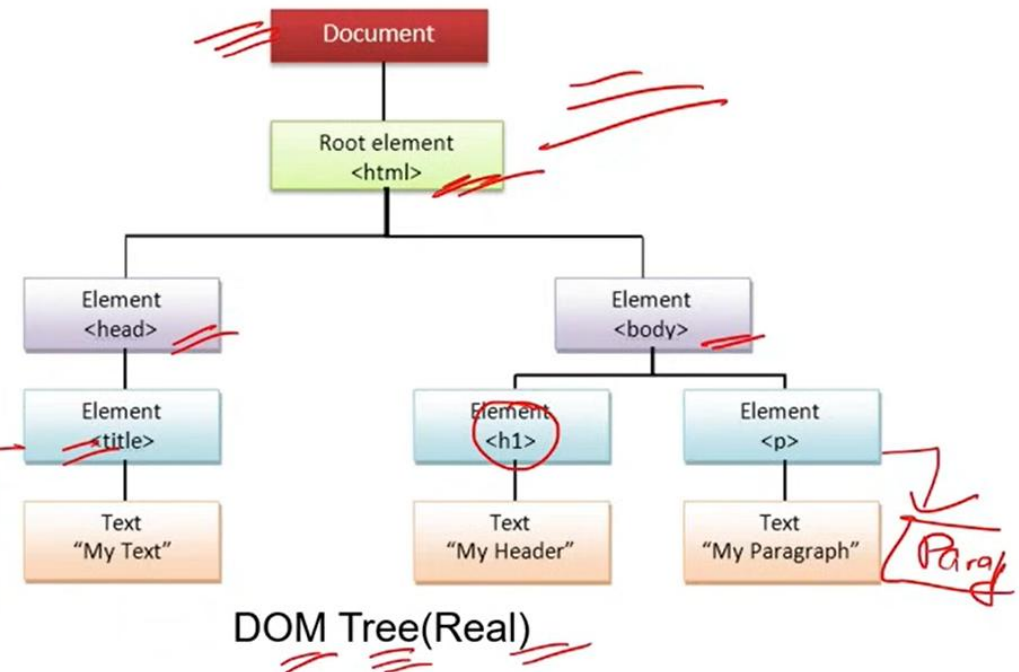


Q. What is **DOM**? What is the difference between **HTML** and the **DOM**? **V. IMP.**



```
<!DOCTYPE html>
<html>
  <head>
    <title>My Text </title>
  </head>
  <body>
    <h1>My Header
    </h1>
    <p>My Paragraph </p>
  </body>
</html>
```

Static HTML

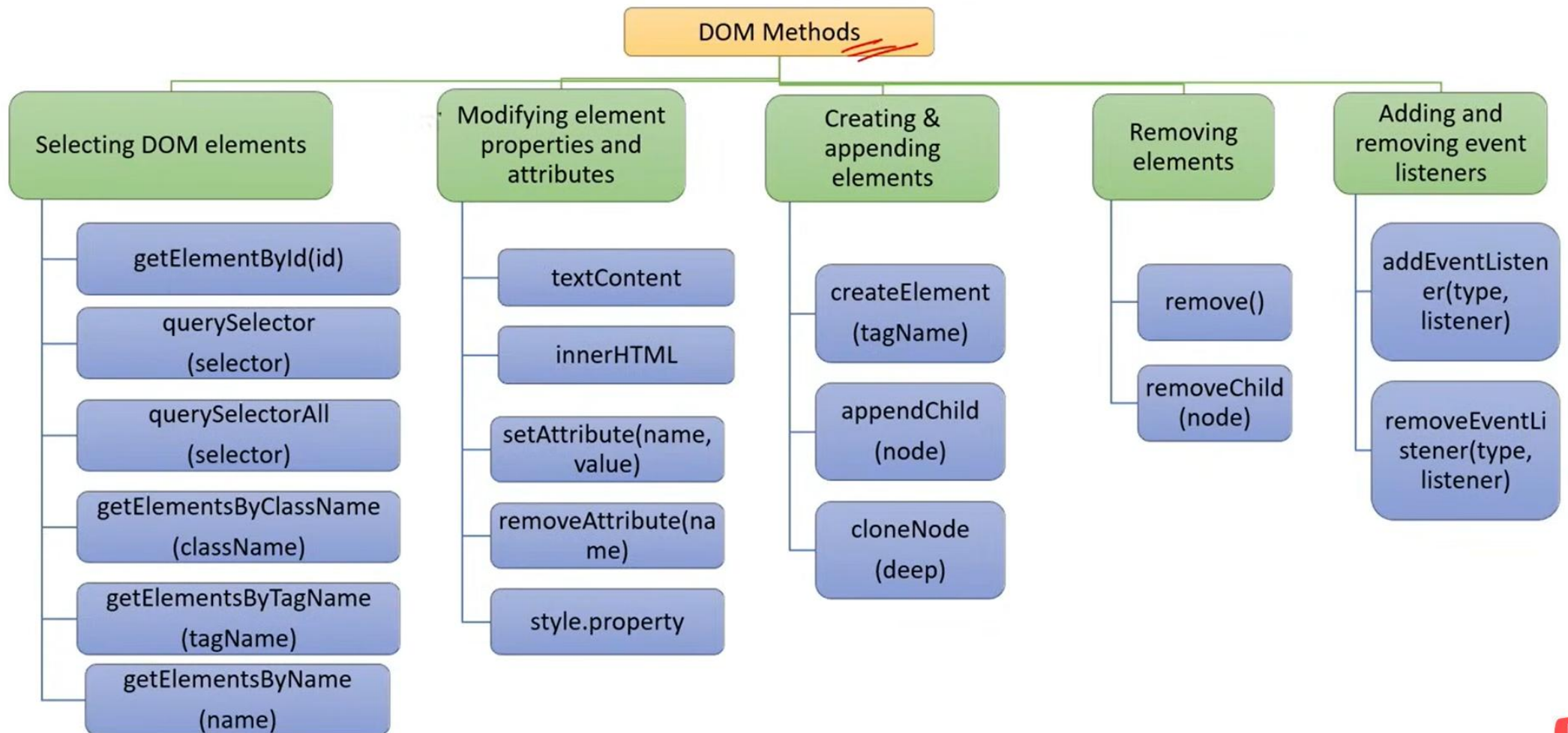


DOM Tree(Real)

- ❖ The DOM(Document Object Model) represents the web page as a **tree-like structure** that allows JavaScript to dynamically access and manipulate the content and structure of a web page.



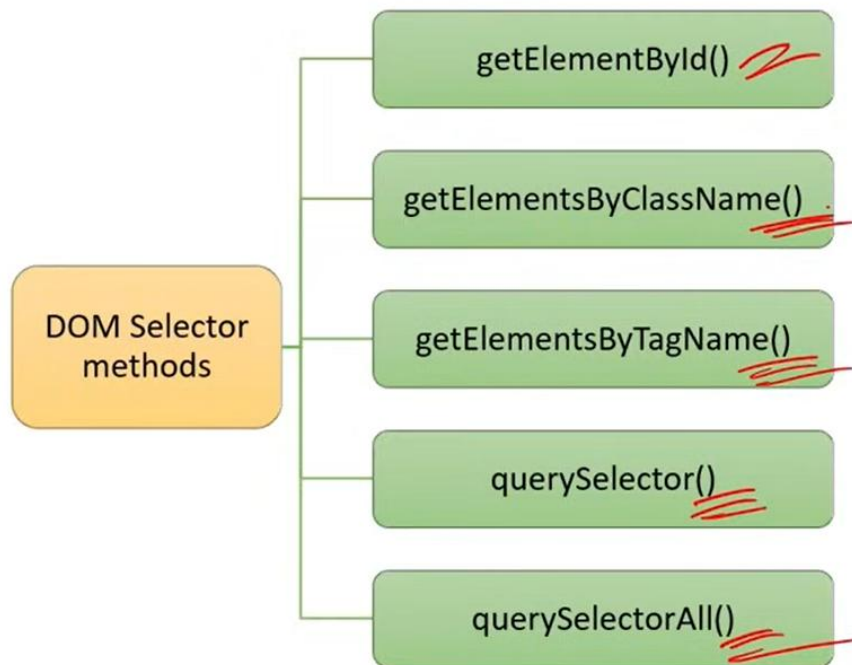
Q. How do you select, modify, create and remove DOM elements?



Q. What are selectors in JS? **V. IMP.**



- ❖ Selectors in JS help to get specific elements from **DOM** based on IDs, class names, tag names.



```
<div id="myDiv">Test</div>
```

```
//getElementById - select a single element  
const elementById = document.getElementById("myDiv");  
console.log(elementById.innerHTML);
```



Q. What is the difference between getElementById, getElementsByClassName and getElementsByTagName? **V. IMP.**



```
<!DOCTYPE html>
<html>
  <head>
    <title>DOM Methods</title>
  </head>
  <body>
    <div id="myDiv" class="myClass">1</div>
    <div class="myClass">2</div>
    <p class="myClass">3</p>
    <script src="index.js"></script>
  </body>
</html>
```

```
//getElementById - select a single element
const elementById = document.getElementById("myDiv");
console.log(elementById.innerHTML);
// Output: 1
```

Single

```
//getElementsByClassName - select multiple elements that
//share the same class name
const elements = document.getElementsByClassName("myClass");

for (let i = 0; i < elements.length; i++) {
  console.log(elements[i].textContent);
}
// Output: 1 2 3
```

```
//getElementsByTagName - select multiple elements based
//on their tag name
const elementsTag = document.getElementsByTagName("div");

for (let i = 0; i < elementsTag.length; i++) {
  console.log(elementsTag[i].textContent);
}
// Output: 1 2
```



Q. What is the difference between `querySelector()` and `querySelectorAll()`?



```
<html>
  <head>
    <title></title>
  </head>
  <body>
    <div class="myClass">Element 1</div>
    <div class="myClass">Element 2</div>
    <div class="myClass">Element 3</div>
    <script src="index.js"></script>
  </body>
</html>
```

```
// Using querySelector() - returns the first element
var element = document.querySelector('.myClass');
console.log(element.textContent);
// Output: Element 1
```

```
// Using querySelectorAll() - returns all the elements
var elements = document.querySelectorAll('.myClass');
elements.forEach(function(element) {
  console.log(element.textContent);
});
// Output: Element 1, Element 2, Element 3
```



Q. What are the methods to modify elements properties and attributes?



DOM methods
for modifying
elements and
their properties

textContent

innerHTML

setAttribute()

removeAttribute()

style.setProperty

classList.add()

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Document</title>
  </head>
  <body>
    <div id="myElement">Hello, World!</div>
    <script src="index.js"></script>
  </body>
</html>
```



Q. What is the difference between innerHTML and textContent? **V. IMP.**



DOM methods
for modifying
elements and
their
properties

textContent

innerHTML

setAttribute()

removeAttribute()

style.setProperty

classList.add()

```
<div id="myElement1">Hello</div>  
<div id="myElement2">World</div>
```

```
// textContent property used to assign plain text to element  
var element1 = document.getElementById("myElement1");  
element1.textContent = "<strong>Happy</strong>";
```

```
// innerHTML property, the browser interprets the content  
// as HTML and renders it accordingly  
var element1 = document.getElementById("myElement2");  
element1.innerHTML = "<strong>Happy</strong>";
```



Q. What is the difference between innerHTML and textContent? **V. IMP.**



DOM methods
for modifying
elements and
their
properties

textContent

innerHTML

setAttribute()

removeAttribute()

style.setProperty

classList.add()

```
<div id="myElement1">Hello</div>  
<div id="myElement2">World</div>
```

```
// textContent property used to assign plain text to element  
var element1 = document.getElementById("myElement1");  
element1.textContent = "<strong>Happy</strong>";
```

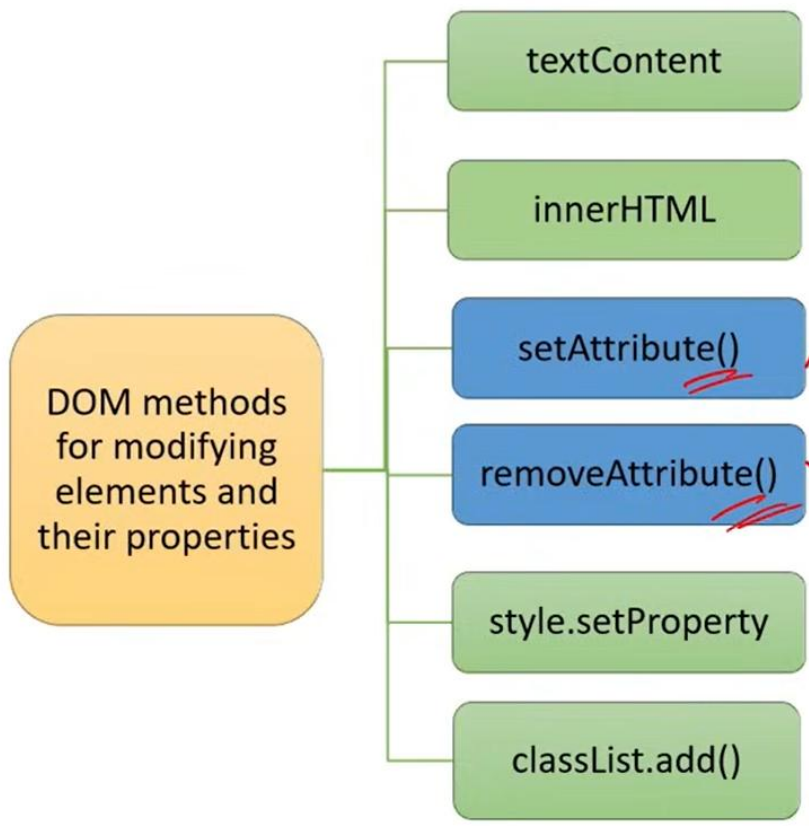
```
// innerHTML property, the browser interprets the content  
// as HTML and renders it accordingly  
var element1 = document.getElementById("myElement2");  
element1.innerHTML = "<strong>Happy</strong>"; HTML
```

→ 127.0.0.1:5500/index.html

Happy
Happy



Q. How to add and remove properties of HTML elements in the DOM using JS?



```
<div id="myElement">Hello, World!</div>
```

```
var element = document.getElementById("myElement");  
// setAttribute method is used to add property  
element.setAttribute("data-info", "new value");
```

```
<body>  
  <div id="myElement" data-info="new value">Hello, World!</div>
```

```
// removeAttribute method is used to remove property  
element.removeAttribute("data-info");
```

```
<body> == $0  
  <div id="myElement">Hello</div>
```



Q. How to add and remove style from HTML elements in DOM using JS?



DOM methods
for modifying
elements and
their
properties

textContent

innerHTML

setAttribute()

removeAttribute()

style.setProperty

classList.add()

```
<div id="myElement">Hello, World!</div>
```

```
var element = document.getElementById("myElement");  
// style.setProperty is used to modify element style  
element.style.setProperty("color", "blue");
```

```
// add the new class to element  
element.classList.add("highlight");  
element.classList.remove("highlight");  
element.classList.toggle("highlight");
```

```
<div id="myElement" class="highlight" style="color: blue;">Hello, World!</div>
```



Q. How to add and remove style from HTML elements in DOM using JS?



DOM methods
for modifying
elements and
their
properties

textContent

innerHTML

setAttribute()

removeAttribute()

style.setProperty

classList.add()

```
<div id="myElement">Hello, World!</div>
```

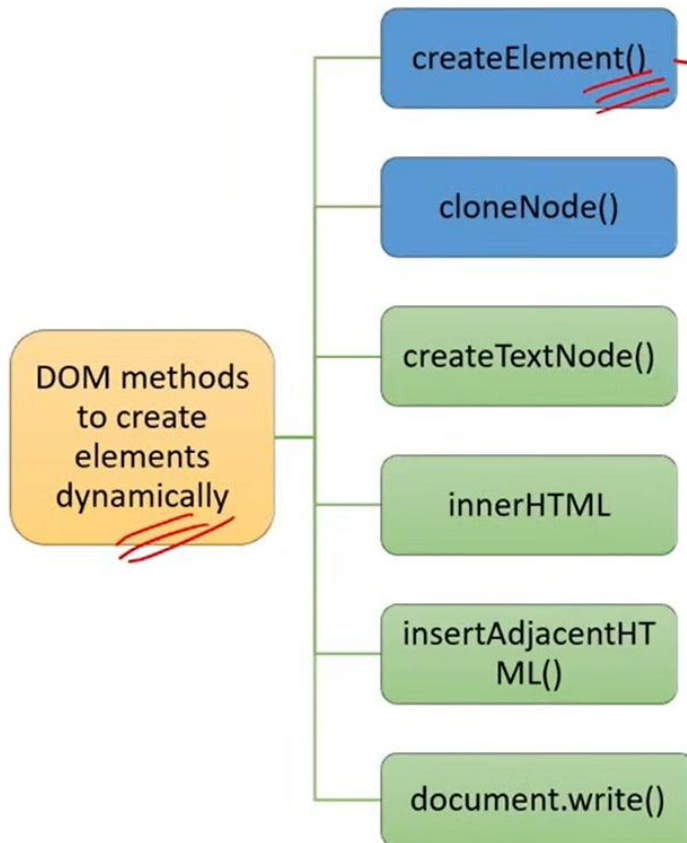
```
var element = document.getElementById("myElement");  
// style.setProperty is used to modify element style  
element.style.setProperty("color", "blue");
```

```
// add the new class to element  
element.classList.add("highlight");  
element.classList.remove("highlight");  
element.classList.toggle("highlight");
```

```
<div id="myElement" class="highlight" style="color: blue;">Hello, World!</div>
```



Q. How to create new elements in DOM using JS? What is the difference between createElement() and cloneNode()?



```
// createElement() - create new element
var newDiv = document.createElement('div');
newDiv.textContent = 'Newly created div';
document.body.appendChild(newDiv);
```

<div>Newly created div</div>



Q. How to create new elements in DOM using JS? What is the difference between createElement() and cloneNode()?



DOM methods
to create
elements
dynamically

createElement()

cloneNode()

createTextNode()

innerHTML

insertAdjacentHT
ML()

document.write()

```
// createElement() - create new element  
var newDiv = document.createElement('div');  
newDiv.textContent = 'Newly created div';  
document.body.appendChild(newDiv);
```

<h1>
<h2>

<div>Newly created div</div>

```
<div id="parentElement">  
  <h1>Existing Element</h1>  
</div>
```

```
// cloneNode() - copy the existing element with all attributes  
var existingElement = document.getElementById('parentElement');  
var clonedElement = existingElement.cloneNode(true);  
clonedElement.textContent = 'Cloned element';  
document.body.appendChild(clonedElement);
```



Q. How to create new elements in DOM using JS? What is the difference between createElement() and cloneNode()?



DOM methods
to create
elements
dynamically

createElement()

cloneNode()

createTextNode()

innerHTML

insertAdjacentHT
ML()

document.write()

```
// createElement() - create new element
var newDiv = document.createElement('div');
newDiv.textContent = 'Newly created div';
document.body.appendChild(newDiv);
```

<h1>
<h2>

<div>Newly created div</div>

```
<div id="parentElement">
  <h1>Existing Element</h1>
</div>
```

```
// cloneNode() - copy the existing element with all attributes
var existingElement = document.getElementById('parentElement');
var clonedElement = existingElement.cloneNode(true);
clonedElement.textContent = 'Cloned element';
document.body.appendChild(clonedElement);
```

<div id="parentElement">Cloned element</div>



Q. What is the difference between `createElement()` and `createTextNode()`?



DOM methods to
create elements
dynamically

`createElement()`

`cloneNode()`

`createTextNode()`

`innerHTML`

`insertAdjacentHT
ML()`

`document.write()`

```
// createElement() - create new element
var newDiv = document.createElement('div');
newDiv.textContent = 'Newly created div';
document.body.appendChild(newDiv);
```

`<div>Newly created div</div>`

```
<div id="parentElement">
  <h1>Existing Element</h1>
</div>
```

```
var parentElement = document.getElementById("parentElement");
// Create a new text node
var newText = document.createTextNode(" This is some text.");
// Append the text node to the parent
parentElement.appendChild(newText);
```



Q. What is the difference between `createElement()` and `createTextNode()`?



DOM methods to
create elements
dynamically

`createElement()`

`cloneNode()`

`createTextNode()`

`innerHTML`

`insertAdjacentHTML()`

`document.write()`

```
// createElement() - create new element
var newDiv = document.createElement('div');
newDiv.textContent = 'Newly created div';
document.body.appendChild(newDiv);
```

```
<div>Newly created div</div>
```

```
<div id="parentElement">
  <h1>Existing Element</h1>
</div>
```

```
var parentElement = document.getElementById("parentElement");
// Create a new text node
var newText = document.createTextNode(" This is some text.");
// Append the text node to the parent
parentElement.appendChild(newText);
```

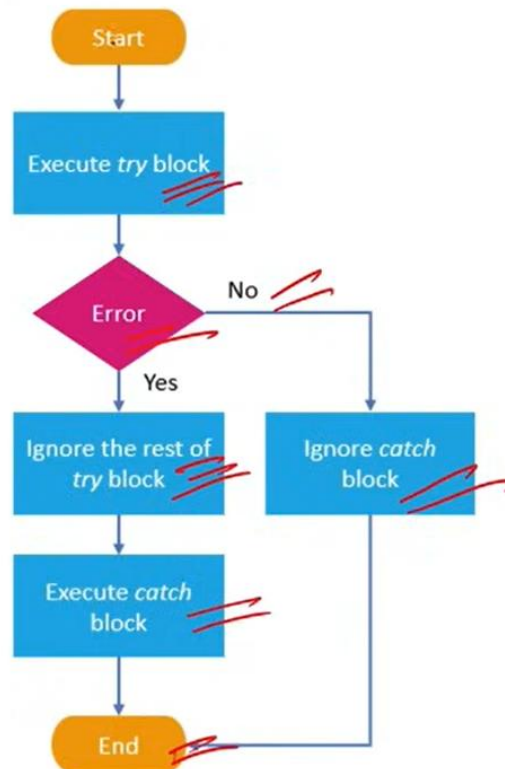
```
▼ <div id="parentElement">
  <h1>Existing Element</h1>
  " This is some text."
</div>
```



Q. What is Error Handling in JS? V. IMP.



- ❖ Error handling is the process of managing errors.



```
//try block contains the code that might throw an error
try {
  const result = someUndefinedVariable + 10;
  console.log(result);
}
//catch block is where the error is handled
catch (error) {
  console.log('An error occurred:', error.message);
}

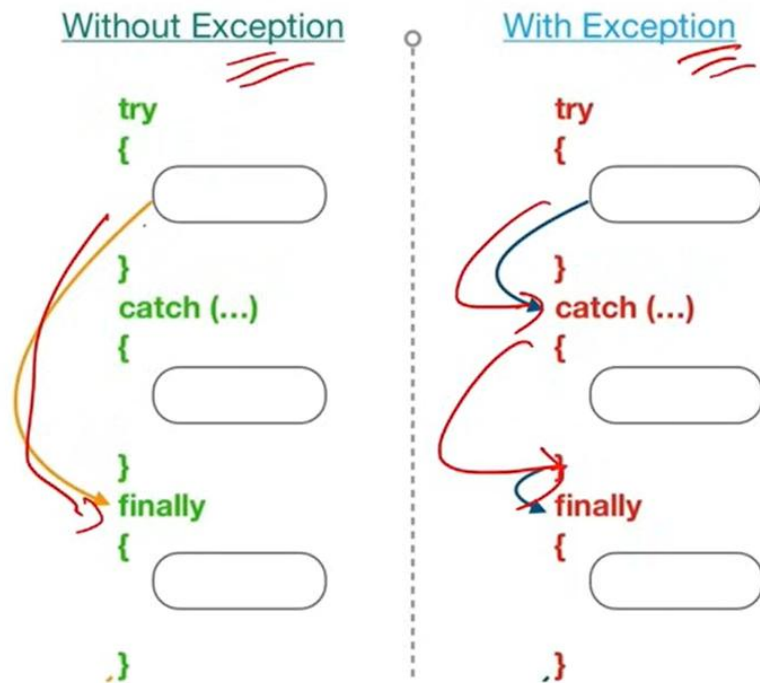
//Output
//An error occurred: someUndefinedVariable is not defined
```



Q. What is the role of **finally** block in JS?



- ❖ Finally, block is used to **execute some code irrespective of error.**



```
//try block contains the code that might throw an error
try {
  const result = someUndefinedVariable + 10;
  console.log(result);
}
//catch block is where the error is handled
catch (error) {
  console.log('An error occurred:', error.message);
}
//finally block to execute code regardless of whether
//an error occurred or not
finally{
  console.log("finally executed");
}

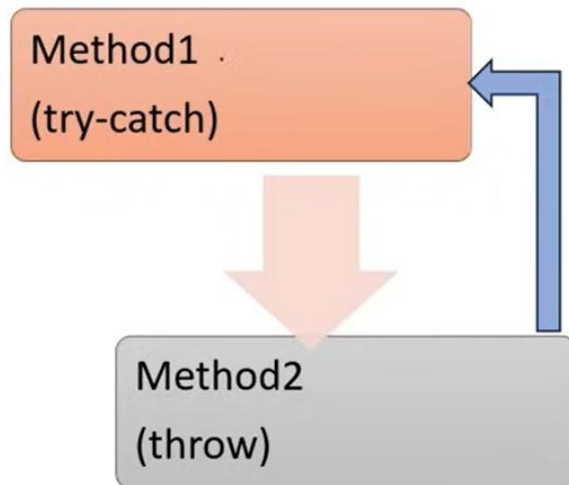
//Output
//An error occurred: someUndefinedVariable is not defined
//finally executed
```



Q. What is the purpose of the throw statement in JS?



- ❖ The throw statement stops the execution of the current function and **passes the error to the catch block of calling function.**



```
function UserData() {  
  try {  
    validateUserAge(25);  
    validateUserAge("invalid"); // This will throw  
    validateUserAge(15); // This will not execute  
  } catch (error) {  
    console.error("Error:", error.message);  
  }  
}  
  
function validateUserAge(age) {  
  if (typeof age !== "number") {  
    throw new Error("Age must be a number");  
  }  
  console.log("User age is valid");  
}
```



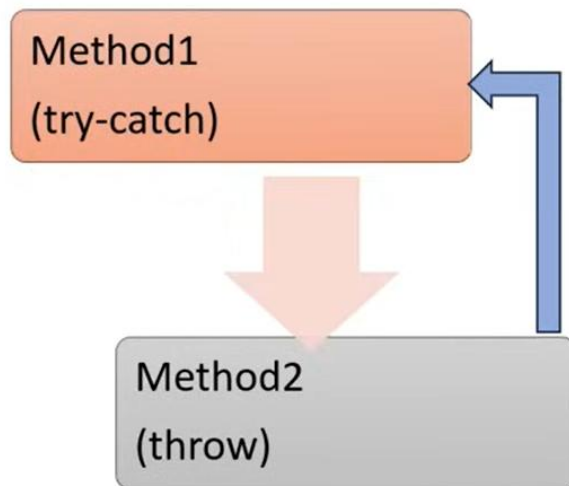
Q. What are the best practices for **error handling**?



Q. What is **Error propagation** in JS?



- ❖ Error propagation refers to the process of passing or propagating an error from **one part of the code to another** by using the **throw statement** with try catch.



```
function UserData() {  
  try {  
    validateUserAge(25);  
    validateUserAge("invalid"); // This will throw  
    validateUserAge(15); // This will not execute  
  } catch (error) {  
    console.error("Error:", error.message);  
  }  
}  
  
function validateUserAge(age) {  
  if (typeof age !== "number") {  
    throw new Error("Age must be a number");  
  }  
  console.log("User age is valid");  
}
```



Q. What are the different **types of errors** In JS?



SUBSCRIBE



SUBSCRIBE

Q. What are the best practices for error handling?



```
// 1. Use Try Catch and Handle Errors Appropriately
try {
    // Code that may throw an error
} catch (error) {
    // Error handling and recovery actions
}
```

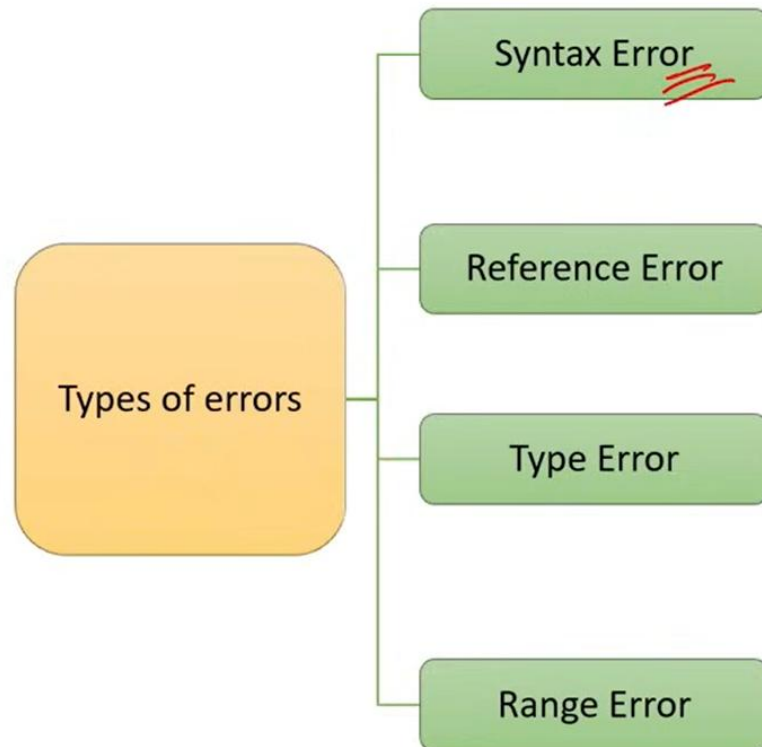
```
// 2. Use Descriptive Error Messages
throw new Error("Cannot divide by zero");
```

```
// 3. Avoid Swallowing Errors
try {
    // Code that may throw an error
} catch (error) {
    // Do not leave the catch blank
    empty
}
```

```
// 4. Log Errors
try {
    // Code that may throw an error
} catch (error) {
    console.error("An error occurred:", error);
    // Log the error with a logging library
}
```



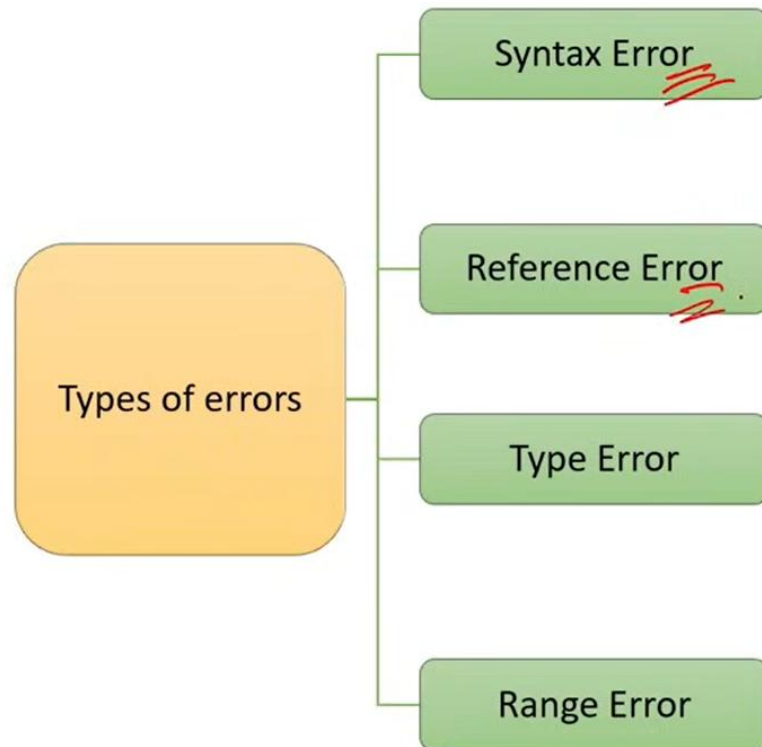
Q. What are the different types of errors In JS?



```
// Syntax Error  
console.log("Hello, World!"  
// Missing closing parenthesis );
```



Q. What are the different types of errors In JS?

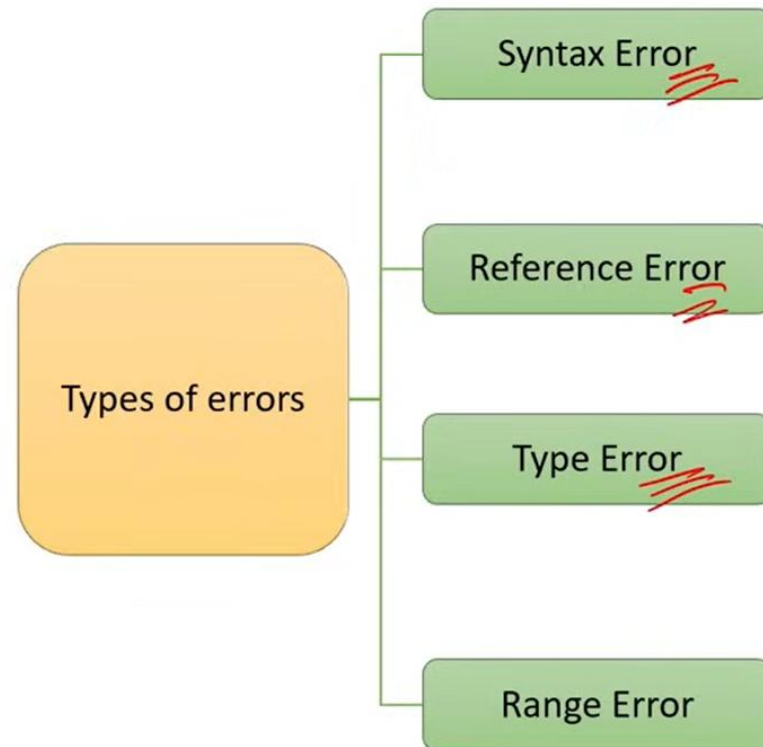


```
// Syntax Error  
console.log("Hello, World!"  
// Missing closing parenthesis );
```

```
// Reference Error  
console.log(myVariable);  
// myVariable is not defined
```



Q. What are the different types of errors In JS?



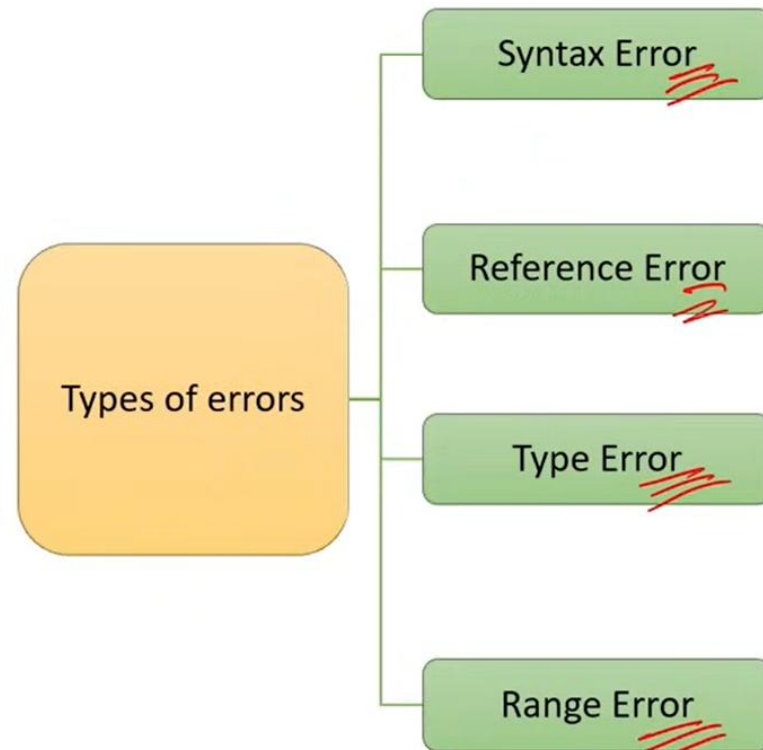
```
// Syntax Error  
console.log("Hello, World!"  
// Missing closing parenthesis );
```

```
// Reference Error  
console.log(myVariable);  
// myVariable is not defined
```

```
// Type Error  
const number = 42;  
console.log(number.toUpperCase());  
// number.toUpperCase is not a function
```



Q. What are the different types of errors In JS?



```
// Syntax Error  
console.log("Hello, World!"  
// Missing closing parenthesis );
```

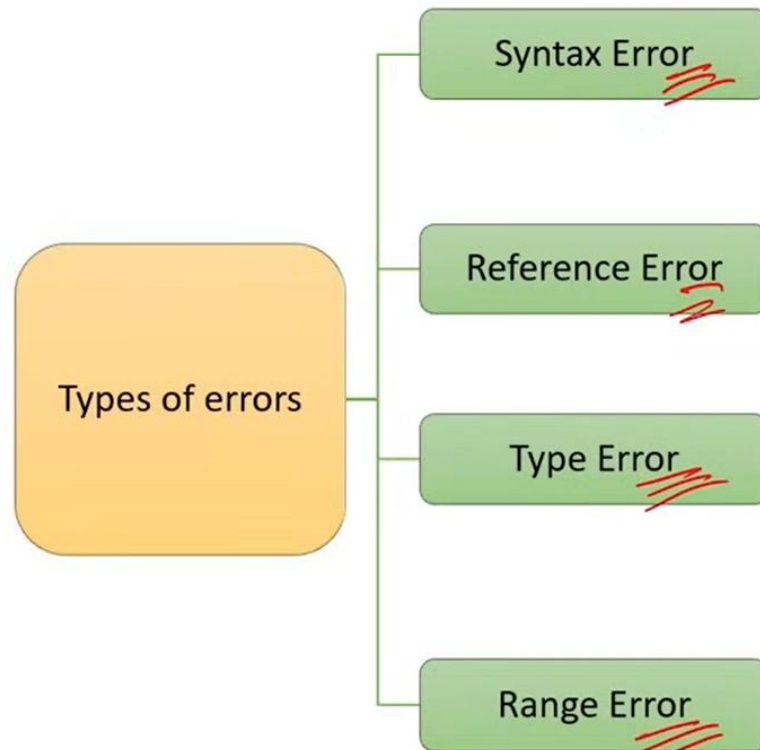
```
// Reference Error  
console.log(myVariable);  
// myVariable is not defined
```

```
// Type Error  
const number = 42;  
console.log(number.toUpperCase());  
// number.toUpperCase is not a function
```

```
// Range Error  
const array = [1, 2, 3];  
console.log(array[10]);  
// Index 10 is out of bounds
```



Q. What are the different types of errors In JS?



```
// Syntax Error  
console.log("Hello, World!"  
// Missing closing parenthesis );
```

```
// Reference Error  
console.log(myVariable);  
// myVariable is not defined
```

```
// Type Error  
const number = 42;  
console.log(number.toUpperCase());  
// number.toUpperCase is not a function
```

```
// Range Error  
const array = [1, 2, 3];  
console.log(array[10]);  
// Index 10 is out of bounds
```



Chapter 10: Objects



basics

Q. What are Objects in JS?

Q. In how many ways we can create an object?

Q. What is the difference between an array and an object?

Q. What is the difference between array and objects?

Q. How do you add or modify or delete properties of an object?

Q. Explain the difference between dot notation and bracket notation?

Q. What are some common methods to iterate over the properties of an object?

Q. How do you check if a property exists in an object?

Q. How do you clone or copy an object?

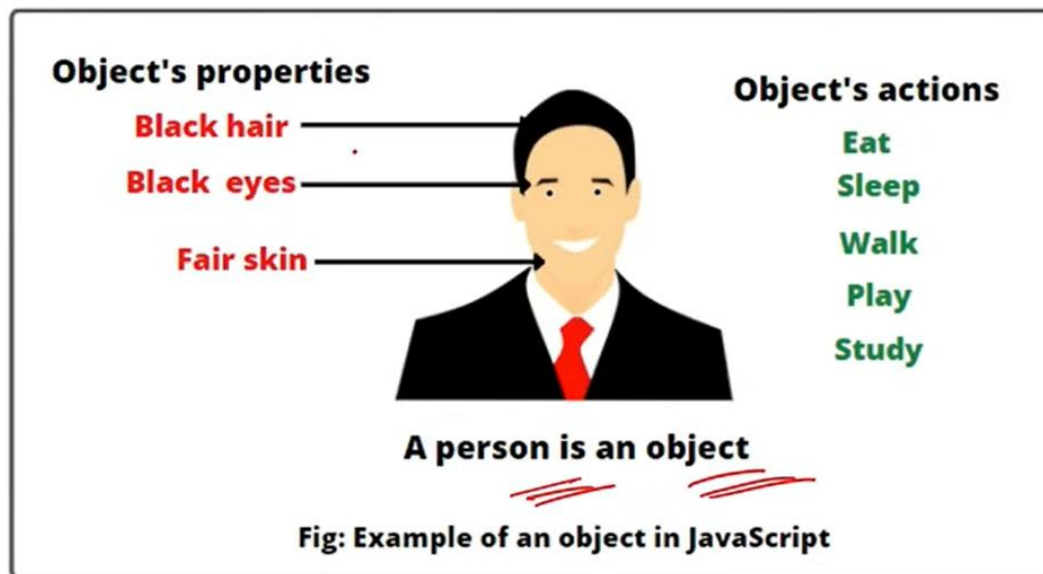
Q. What is the difference between deep copy and shallow copy in JS?



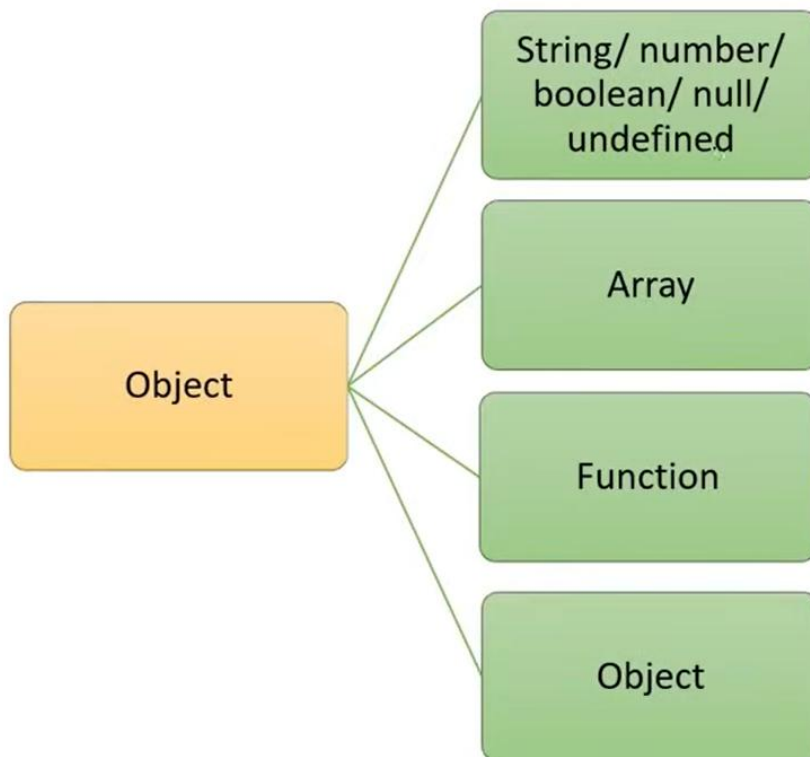
Q. What are **Objects** in JS? **V. IMP.**



Object → Real



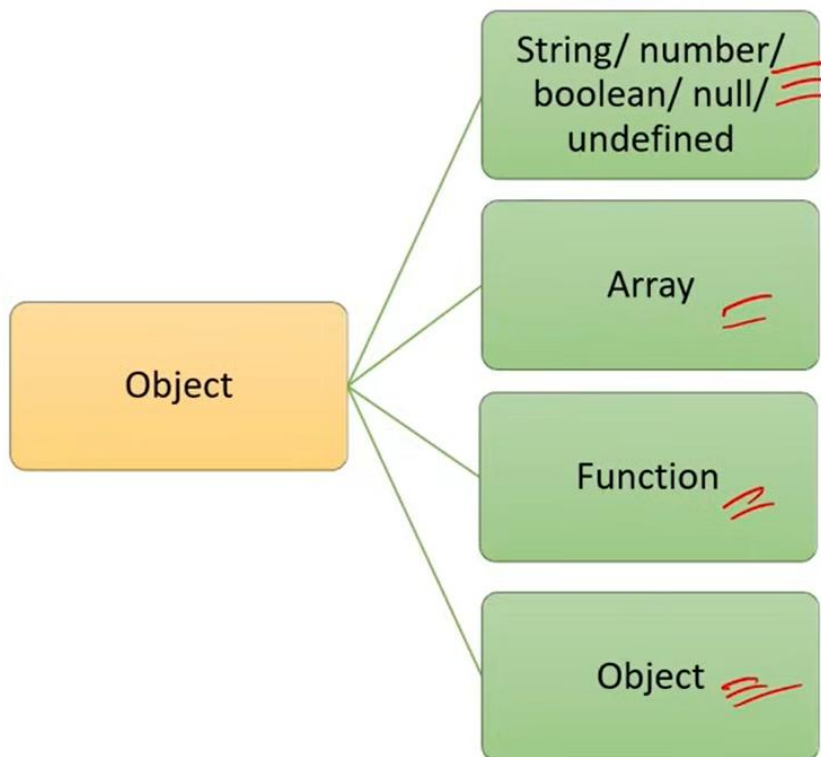
Q. What are **Objects** in JS? **V. IMP.**



```
//Object Example
let person = {
  name: "Happy",
  hobbies: ["Teaching", "Football", "Coding"],
  greet: function () {
    console.log("Name: " + this.name);
  },
};
```



Q. What are **Objects** in JS? **V. IMP.**



```
//Object Example
let person = {
  name: "Happy",
  hobbies: ["Teaching", "Football", "Coding"],
  greet: function () {
    console.log("Name: " + this.name);
  },
};

console.log(person.name);
// Output: "Happy"

console.log(person.hobbies[1]);
// Output: "Football"

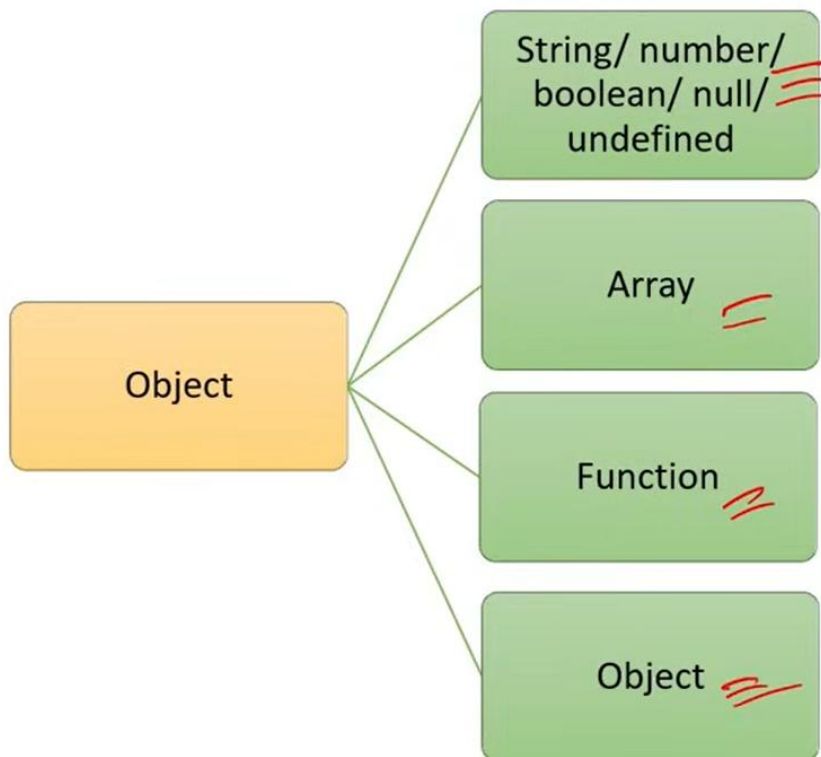
person.greet();
// Output: "Name: Happy"
```



Q. What are **Objects** in JS? **V. IMP.**



- ❖ An object is a data type that allows you to store **key-value** pairs.



```
//Object Example
let person = {
  name: "Happy",
  hobbies: ["Teaching", "Football", "Coding"],
  greet: function () {
    console.log("Name: " + this.name);
  },
};

console.log(person.name);
// Output: "Happy"

console.log(person.hobbies[1]);
// Output: "Football"

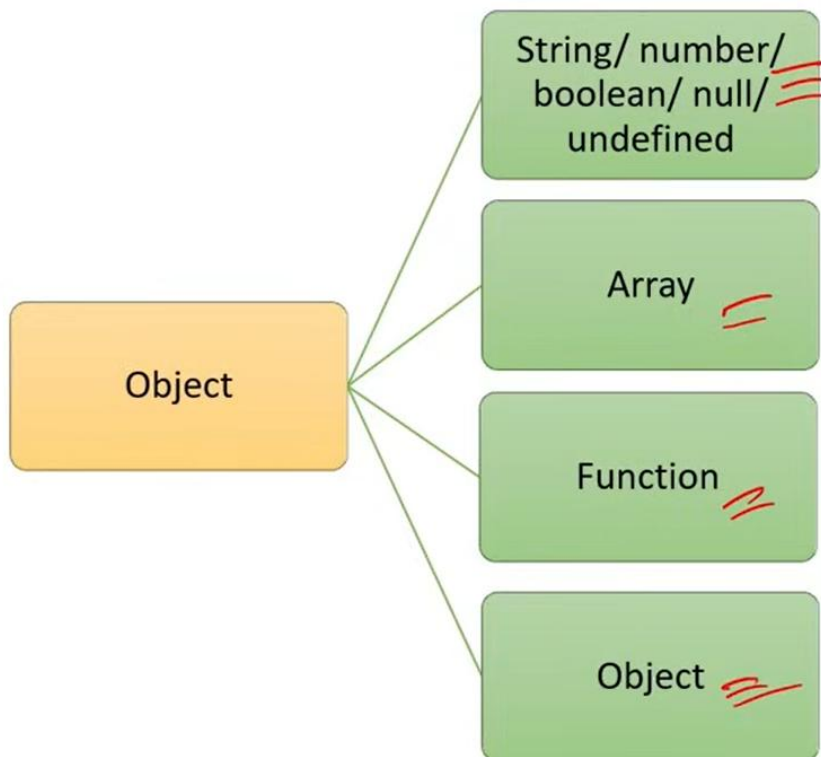
person.greet();
// Output: "Name: Happy"
```



Q. What are **Objects** in JS? **V. IMP.**



- ❖ An object is a data type that allows you to store **key-value** pairs.



```
//Object Example
let person = {
  name: "Happy",
  hobbies: ["Teaching", "Football", "Coding"],
  greet: function () {
    console.log("Name: " + this.name);
  },
};

console.log(person.name);
// Output: "Happy"

console.log(person.hobbies[1]);
// Output: "Football"

person.greet();
// Output: "Name: Happy"
```



Q. In how many ways we can create an object?



Ways to create and initialize an object

Object Literal

Object Constructor

Object.create() Method

```
// Object literal
var person = {
  name: "Happy",
  age: 38,
  role: "Trainer"
};
console.log(person);
```



Q. In how many ways we can create an object?



Ways to create and initialize an object

Object Literal

```
// Object literal
var person = {
  name: "Happy",
  age: 38,
  role: "Trainer"
};
console.log(person);
```

```
▼ {name: 'Happy', age: 38, role: 'Trainer'}
  age: 38
  name: "Happy"
  role: "Trainer"
  ► [[Prototype]]: Object
```

Object Constructor

```
// Object Constructor
var person = new Object();
person.name = "Happy";
person.age = 38;
person.role = "Trainer";
console.log(person);
```

Object.create() Method

```
// Object.create() Method
var person = {
  name: "",
  age: 0,
  role: ""
};

var men = Object.create(person);
men.name = "Happy";
men.age = 38;
men.role = "Trainer";
```



Q. What is the difference between an array and an object?



Arrays	Objects
1. Arrays are collection of <u>values</u> .	Objects are collections of <u>key-value pairs</u> .
2. Arrays are denoted by square brackets <u>[]</u> .	Objects are denoted by curly braces <u>{}</u> .
3. Elements in array are <u>ordered</u> .	Properties in objects are <u>unordered</u> .

```
// Array  
var fruits = ["apple", "banana", "orange"];
```

```
// Object  
var person = {  
  name: "Amit",  
  age: 25,  
  city: "Delhi"  
};
```



Q. How do you add or modify or delete properties of an object?



```
//Blank object
```

```
var person = {};
```

```
// Adding Properties
```

```
person.name = "Happy";
```

```
person.age = 35;
```

```
person.country = "India"
```

```
// Modifying Properties
```

```
person.age = 30;
```

```
// Deleting Properties
```

```
delete person.age;
```



Q. Explain the difference between dot notation and bracket notation?



- ❖ Both dot notation and bracket notation are used to access properties or methods of an object.
- ❖ Dot notation is more popular and used due to its simplicity.

```
const person = {  
  name: 'Happy',  
  age: 35,  
};
```

```
// Dot notation:  
console.log(person.name);  
// Output: 'Happy'
```

```
// Bracket notation:  
console.log(person['name']);  
// Output: 'Happy'
```

- ❖ Limitation of dot notation - In some scenarios bracket notation is the only option, such as when accessing properties when the property name is stored in a variable.

```
// Dynamically assign property name  
// to a variable  
var propertyName = 'age';  
  
console.log(person[propertyName]);  
// Output: '35'  
  
console.log(person.propertyName);  
// Output: undefined
```



Q. Explain the difference between dot notation and bracket notation?



- ❖ Both dot notation and bracket notation are used to access properties or methods of an object.
- ❖ Dot notation is more popular and used due to its simplicity.

```
const person = {  
  name: 'Happy',  
  age: 35,  
};
```

```
// Dot notation:  
console.log(person.name);  
// Output: 'Happy'
```

```
// Bracket notation:  
console.log(person['name']);  
// Output: 'Happy'
```

- ❖ Limitation of dot notation - In some scenarios bracket notation is the only option, such as when accessing properties when the **property name is stored in a variable**.

```
// Dynamically assign property name  
// to a variable  
var propertyName = 'age';  
  
console.log(person[propertyName]);  
// Output: '35'  
  
console.log(person.propertyName);  
// Output: undefined
```



Q. What are some common methods to iterate over the properties of an object?



```
const person = {  
  name: "John",  
  age: 30,  
};
```

1. for...in loop

```
// 1. Using for...in loop  
for (let prop in person) {  
  console.log(prop + ": " + person[prop]);  
}  
// Output: name: John age: 30
```

2. Object.keys() & forEach()

```
// 2. Using Object.keys() and forEach()  
Object.keys(person).forEach((prop) => {  
  console.log(prop + ": " + person[prop]);  
});  
// Output: name: John age: 30
```

3. Object.values() & forEach()

```
// 3. Using Object.values() and forEach()  
Object.values(person).forEach((value) => {  
  console.log(value);  
});  
// Output: John 30
```

4 Ways to iterate over
the properties of an
object



Q. How do you check if a property exists in an object?



```
var person = {  
  name: "Alice",  
  age: 25  
};
```

// 1. Using the in Operator

```
console.log("name" in person); // Output: true  
console.log("city" in person); // Output: false
```

// 2. Using the hasOwnProperty() Method

```
console.log(person.hasOwnProperty("name")); // Output: true  
console.log(person.hasOwnProperty("city")); // Output: false
```

// 3. Comparing with undefined

```
console.log(person.name !== undefined); // Output: true  
console.log(person.city !== undefined); // Output: false
```



Q. How do you clone or copy an object?



```
// Original object
const originalObject = {
  name: 'Happy',
  age: 35,
  city: 'Delhi',
};
```

```
// Method 1: Spread syntax (shallow copy)
const clonedObjectSpread = { ...originalObject };
```

```
// Method 2: Object.assign() (shallow copy)
// Parameters: target, source
const clonedObjectAssign = Object.assign({}, originalObject);
```

```
// Method 3: JSON.parse() and JSON.stringify() (deep copy)
const clonedObjectJSON = JSON.parse(JSON.stringify(originalObject));
```

Ways to clone
or copy an
object

1. Spread Syntax
(...)

2. Object.assign()

3. JSON.parse() &
JSON.stringify()



Q. What is the difference between deep copy and shallow copy in JS?



```
// Original object
const person = {
  name: 'Happy',
  age: 30,
  address: {
    city: 'Delhi',
    country: 'India'
  }
};
```

```
// Shallow copy using Object.assign()
const shallowCopy = Object.assign({}, person);

shallowCopy.address.city = 'Mumbai';

console.log(person.address.city); // Output: "Mumbai"
console.log(shallowCopy.address.city); // Output: "Mumbai"
```

```
// Deep copy using JSON.parse() and JSON.stringify()
const deepCopy = JSON.parse(JSON.stringify(person));

deepCopy.address.city = 'Bangalore';

console.log(person.address.city); // Output: "Delhi"
console.log(deepCopy.address.city); // Output: "Bangalore"
```



Q. What is the difference between deep copy and shallow copy in JS?



- ❖ Shallow copy in **nested objects case** will modify the parent object property value, if cloned object property value is changed. But deep copy will not modify the parent object property value.

```
// Original object
const person = {
  name: 'Happy',
  age: 30,
  address: {
    city: 'Delhi',
    country: 'India'
  }
};
```

```
// Shallow copy using Object.assign()
const shallowCopy = Object.assign({}, person);

shallowCopy.address.city = 'Mumbai';

console.log(person.address.city); // Output: "Mumbai"
console.log(shallowCopy.address.city); // Output: "Mumbai"
```

```
// Deep copy using JSON.parse() and JSON.stringify()
const deepCopy = JSON.parse(JSON.stringify(person));

deepCopy.address.city = 'Bangalore';

console.log(person.address.city); // Output: "Delhi"
console.log(deepCopy.address.city); // Output: "Bangalore"
```



Q. What is Set Object in JS?



- ❖ The Set object is a collection of unique values, meaning that duplicate values are not allowed.
- ❖ Set provides methods for **adding, deleting, and checking the existence** of values in the set.

```
// Creating a Set to store unique numbers
const uniqueNumbers = new Set();
uniqueNumbers.add(5);
uniqueNumbers.add(10);
uniqueNumbers.add(5); //Ignore duplicate values

console.log(uniqueNumbers);
// Output: {5, 10} unique

// Check size
console.log(uniqueNumbers.size);
// Output: 2

// Check element existence
console.log(uniqueNumbers.has(10));
// Output: true

// Delete element
uniqueNumbers.delete(10);
console.log(uniqueNumbers.size);
// Output: 1
```



Q. What is Set Object in JS?



- ❖ The Set object is a collection of unique values, meaning that duplicate values are not allowed.
- ❖ Set provides methods for adding, deleting, and checking the existence of values in the set.
- ❖ Set can be used to remove duplicate values from arrays.

```
// Creating a Set to store unique numbers
const uniqueNumbers = new Set();
uniqueNumbers.add(5);
uniqueNumbers.add(10);
uniqueNumbers.add(5); //Ignore duplicate values

console.log(uniqueNumbers);
// Output: {5, 10} Unique

// Check size
console.log(uniqueNumbers.size);
// Output: 2

// Check element existence
console.log(uniqueNumbers.has(10));
// Output: true

// Delete element
uniqueNumbers.delete(10);
console.log(uniqueNumbers.size);
// Output: 1
```



Q. What is Set Object in JS?



- ❖ The Set object is a collection of unique values, meaning that duplicate values are not allowed.
- ❖ Set provides methods for adding, deleting, and checking the existence of values in the set.
- ❖ Set can be used to remove duplicate values from arrays.

```
// Set can be used to remove
// duplicate values from arrays
let myArr = [1, 4, 3, 4];
let mySet = new Set(myArr);

let uniqueArray = [...mySet];
console.log(uniqueArray);
// Output: [1, 4, 3]
```

```
// Creating a Set to store unique numbers
const uniqueNumbers = new Set();
uniqueNumbers.add(5);
uniqueNumbers.add(10);
uniqueNumbers.add(5); // Ignore duplicate values

console.log(uniqueNumbers);
// Output: {5, 10} Unique

// Check size
console.log(uniqueNumbers.size);
// Output: 2

// Check element existence
console.log(uniqueNumbers.has(10));
// Output: true

// Delete element
uniqueNumbers.delete(10);
console.log(uniqueNumbers.size);
// Output: 1
```



Q. What is Map Object in JS?



- ❖ The Map object is a collection of key-value pairs where each key can be of any type, and each value can also be of any type.
- ❖ A Map maintains the order of key-value pairs as they were inserted.

```
// Creating a Map to store person details
const personDetails = new Map();
personDetails.set("name", "Alice");
personDetails.set("age", 30);

console.log(personDetails.get("name"));
// Output: "Alice"

console.log(personDetails.has("age"));
// Output: true

personDetails.delete("age");
console.log(personDetails.size);
// Output: 1
```



Q. What is the difference between Map and Object in JS?



Map	Javascript Object
1. Keys in a Map can be of <u>any data type</u> , including <u>strings</u> , <u>numbers</u> , <u>objects</u> , <u>functions</u> etc.	Keys in a regular JavaScript object are limited to <u>strings</u> and <u>symbols</u> .
2. A Map maintains the <u>order of key-value pairs</u> as they were <u>inserted</u> .	In a regular object, there is <u>no guaranteed order of keys</u> .
3. Useful when keys are of <u>different types</u> , <u>insertion order</u> is important.	Useful when keys are <u>strings or symbols</u> and there are <u>simple set of properties</u> .





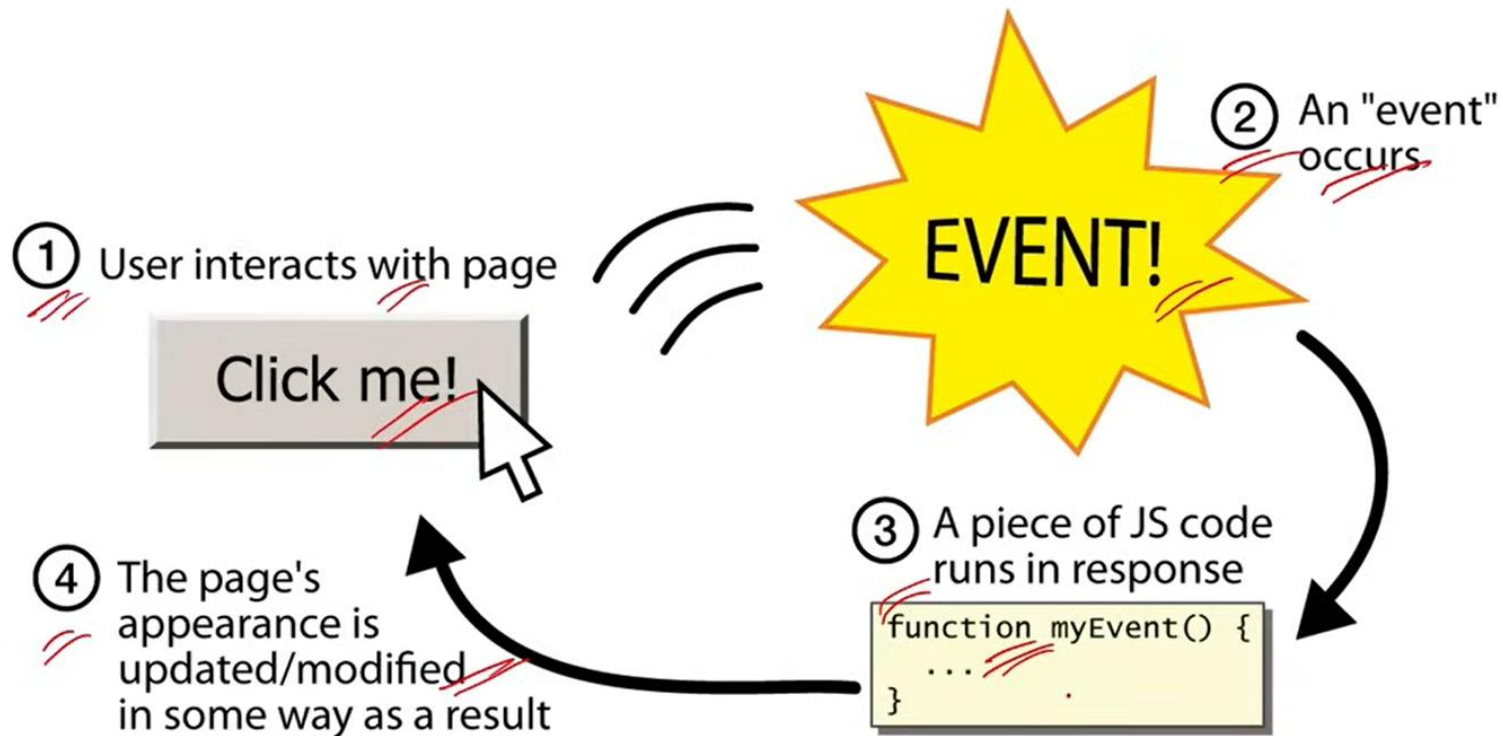
Chapter 11: Events



- Q. What are Events? How are events triggered?
- Q. What are the types of events in JS?
- Q. What is Event Object in JS?
- Q. What is Event Delegation in JS?
- Q. What is Event Bubbling In JS?
- Q. How can you stop event propagation or event bubbling in JS?
- Q. What is Event Capturing in JS?
- Q. What is the purpose of the event.preventDefault() method in JS?
- Q. What is the use of "this" keyword in the context of event handling in JS?
- Q. How to remove an event handler from an element in JS?



Q. What are Events? How are events triggered? V. IMP.



Q. What are **Events**? How are events **triggered**? **V. IMP.**



- ❖ Events are **actions** that happen in the browser, such as a button click, mouse movement, or keyboard input.

```
<button id="myButton">Click Me</button>
```

```
// Get the reference of button in a variable
var button = document.getElementById("myButton");

// Attach an event handler to the button
button.addEventListener("click", handleClick);

// Event handler function
function handleClick(){
    alert("button clicked");
}
```

Event

Event handler/
Callback function



Q. What are the types of events in JS? **V. IMP.**



```
<button id="myButton">Click Me</button>
```

```
// Get the reference of button in a variable
var button = document.getElementById("myButton");

// Attach an event handler to the button
button.addEventListener("click", handleClick);

// Event handler function
function handleClick(){
    alert("button clicked");
}
```

Event

Click Event: addEventListener('click', handler)

Mouseover Event: addEventListener('mouseover', handler)

Keydown Event: addEventListener('keydown', handler)

Keyup Event: addEventListener('keyup', handler)

Submit Event: addEventListener('submit', handler)

Focus Event: addEventListener('focus', handler)

Blur Event: addEventListener('blur', handler)

Change Event: addEventListener('change', handler)

Load Event: addEventListener('load', handler)

Resize Event: addEventListener('resize', handler)



Q. What is Event Object in JS?



- ❖ Whenever any event is triggered, the browser automatically creates an event object and **passes it as an argument** to the event handler function.

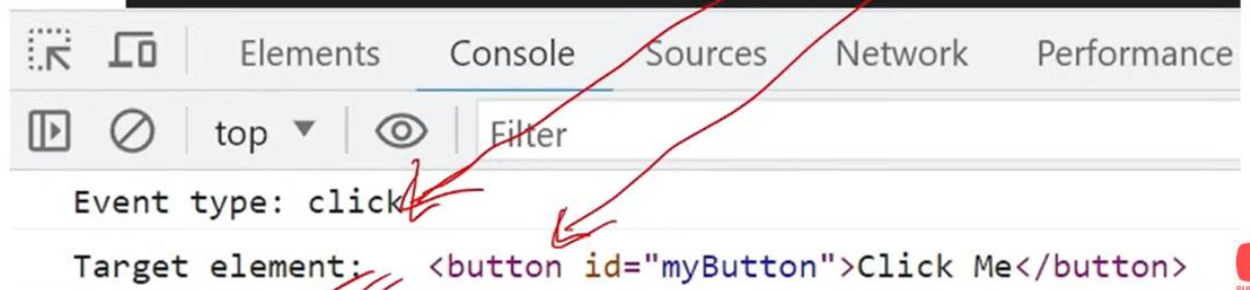
- ❖ The event object contains various properties and methods that provide **information about the event**, such as the type of event, the element that triggered the event etc.

```
<button id="myButton">Click Me</button>
```

```
// Get the button element
var button = document.getElementById("myButton");

// Attach event listener to the button element
button.addEventListener("click", handleClick);

// Event handler function
function handleClick(event) {
  // Accessing properties of the event object
  console.log("Event type:", event.type);
  console.log("Target element:", event.target);
}
```



Q. What is Event Object in JS?



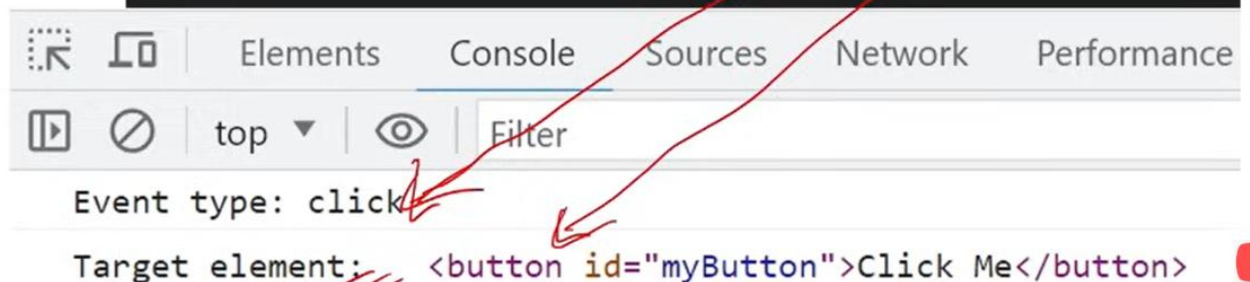
- ❖ Whenever any event is triggered, the browser automatically creates an event object and passes it as an argument to the event handler function.
- ❖ The event object contains various properties and methods that provide information about the event, such as the type of event, the element that triggered the event etc.

```
<button id="myButton">Click Me</button>
```

```
// Get the button element
var button = document.getElementById("myButton");

// Attach event listener to the button element
button.addEventListener("click", handleClick);

// Event handler function
function handleClick(event) {
  // Accessing properties of the event object
  console.log("Event type:", event.type);
  console.log("Target element:", event.target);
}
```



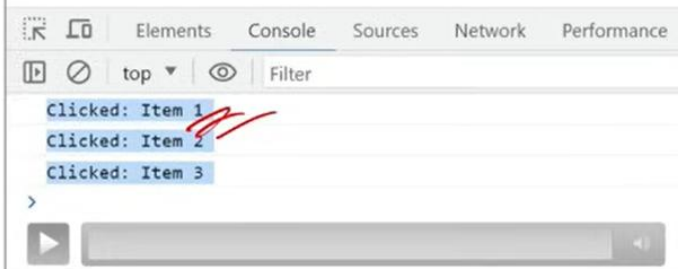
Q. What is Event Delegation in JS? V. IMP.



- ❖ Event delegation in JavaScript is a technique where you attach a **single event handler to a parent element** to handle events on its **child elements**.

```
<ul id="myList">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

- Item 1
- Item 2
- Item 3

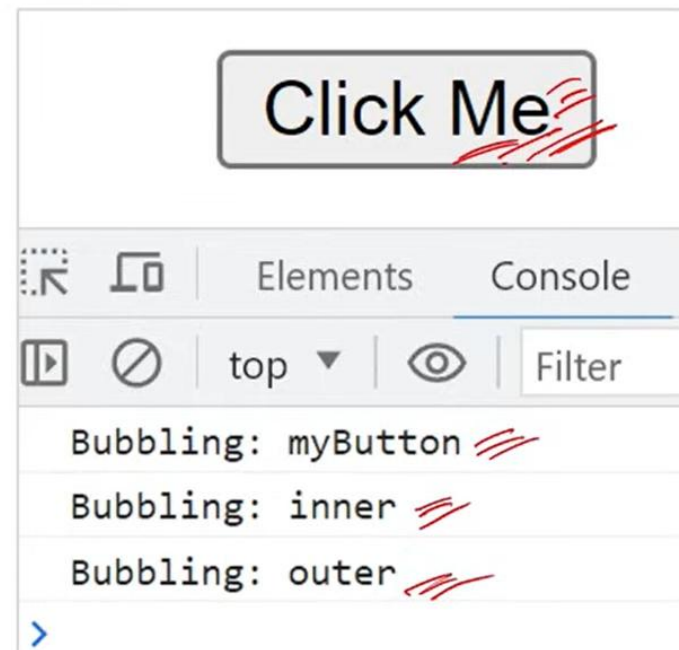
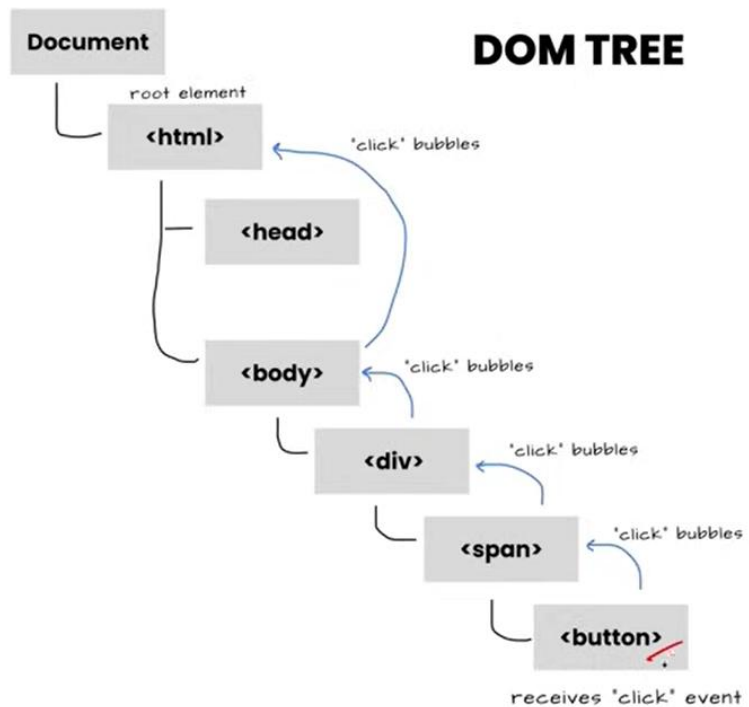


```
var parentList = document.getElementById("myList");
// Attach event handler to parent element
parentList.addEventListener("click", handleItemClick);
// Event handler function
function handleItemClick(event) {
  var target = event.target;
  console.log("Clicked:", target.textContent);
}
```

Q. What is Event Bubbling In JS? **V. IMP.**

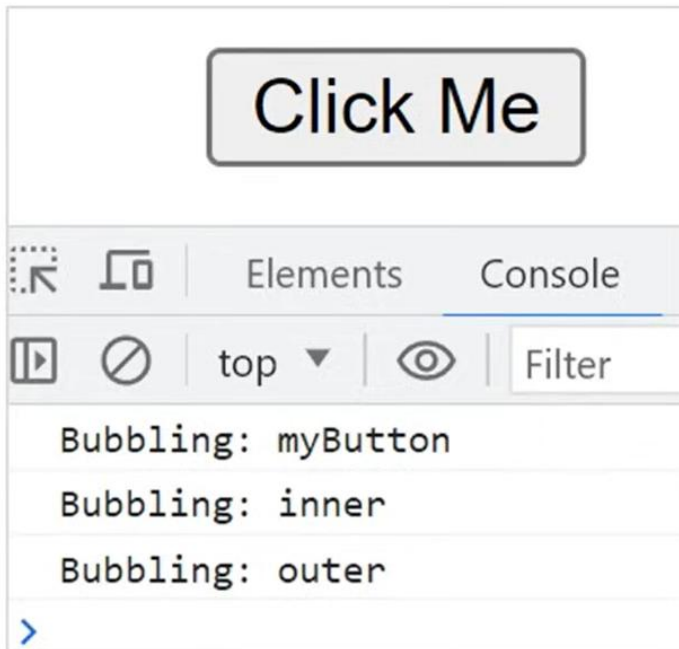


```
<div id="outer">
  <div id="inner">
    <button id="myButton">Click Me</button>
  </div>
</div>
```



Q. What is **Event Bubbling** In JS?

V. IMP.



```
<div id="outer">
  <div id="inner">
    <button id="myButton">Click Me</button>
  </div>
</div>
```

```
// Get the reference of elements
var outer = document.getElementById("outer");
var inner = document.getElementById("inner");
var button = document.getElementById("myButton");

// Attach event handlers with elements
outer.addEventListener("click", handleBubbling);
inner.addEventListener("click", handleBubbling);
button.addEventListener("click", handleBubbling);

function handleBubbling(event) {
  console.log("Bubbling: " + this.id);
}
```

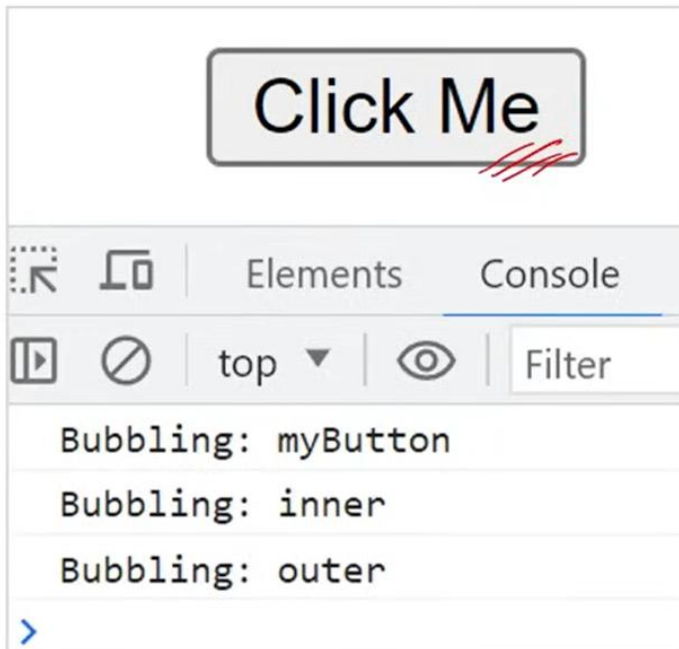


Q. What is Event Bubbling In JS?

V. IMP.



- ❖ Event bubbling is the process in JavaScript where an event triggered on a child element **propagates up the DOM tree**, triggering event handlers on its parent elements.



```
<div id="outer">
  <div id="inner">
    <button id="myButton">Click Me</button>
  </div>
</div>
```

```
// Get the reference of elements
var outer = document.getElementById("outer");
var inner = document.getElementById("inner");
var button = document.getElementById("myButton");

// Attach event handlers with elements
outer.addEventListener("click", handleBubbling);
inner.addEventListener("click", handleBubbling);
button.addEventListener("click", handleBubbling);

function handleBubbling(event) {
  console.log("Bubbling: " + this.id);
}
```

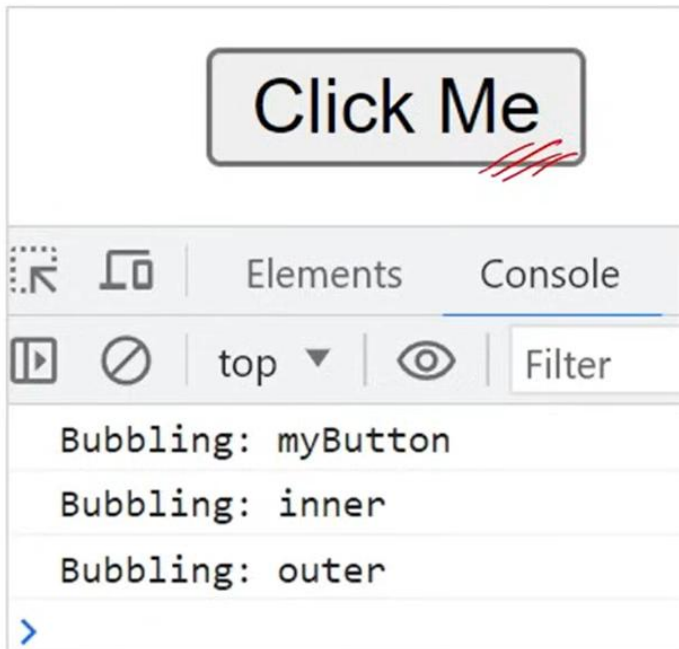


Q. What is Event Bubbling In JS?

V. IMP.



- ❖ Event bubbling is the process in JavaScript where an event triggered on a child element **propagates up the DOM tree**, triggering event handlers on its parent elements.



```
<div id="outer">
  <div id="inner">
    <button id="myButton">Click Me</button>
  </div>
</div>
```

```
// Get the reference of elements
var outer = document.getElementById("outer");
var inner = document.getElementById("inner");
var button = document.getElementById("myButton");

// Attach event handlers with elements
outer.addEventListener("click", handleBubbling);
inner.addEventListener("click", handleBubbling);
button.addEventListener("click", handleBubbling);

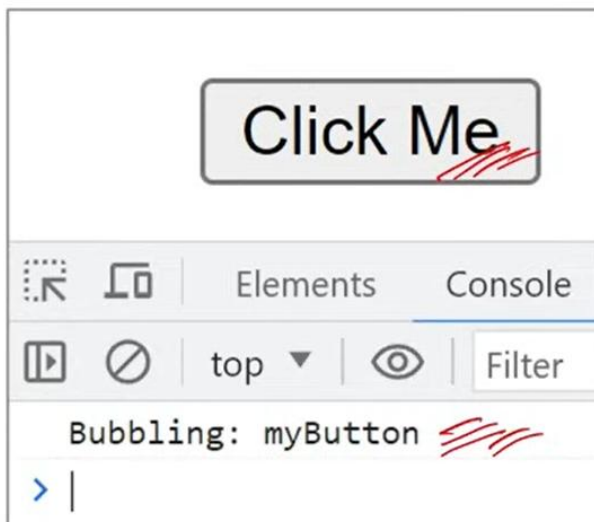
function handleBubbling(event) {
  console.log("Bubbling: " + this.id);
}
```



Q. How can you stop event propagation or event bubbling in JS?



- ❖ Event bubbling can be stopped by calling **stopPropagation()** method on event.



```
<div id="outer">
  <div id="inner">
    <button id="myButton">Click Me</button>
  </div>
</div>
```

```
// Get the reference of elements
var outer = document.getElementById("outer");
var inner = document.getElementById("inner");
var button = document.getElementById("myButton");

// Attach event handlers with elements
outer.addEventListener("click", handleBubbling);
inner.addEventListener("click", handleBubbling);
button.addEventListener("click", handleBubbling);
```

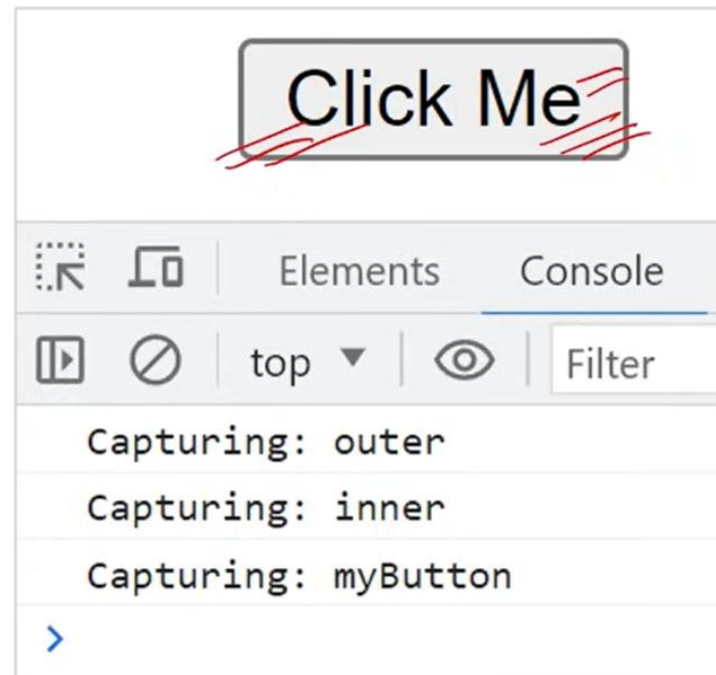
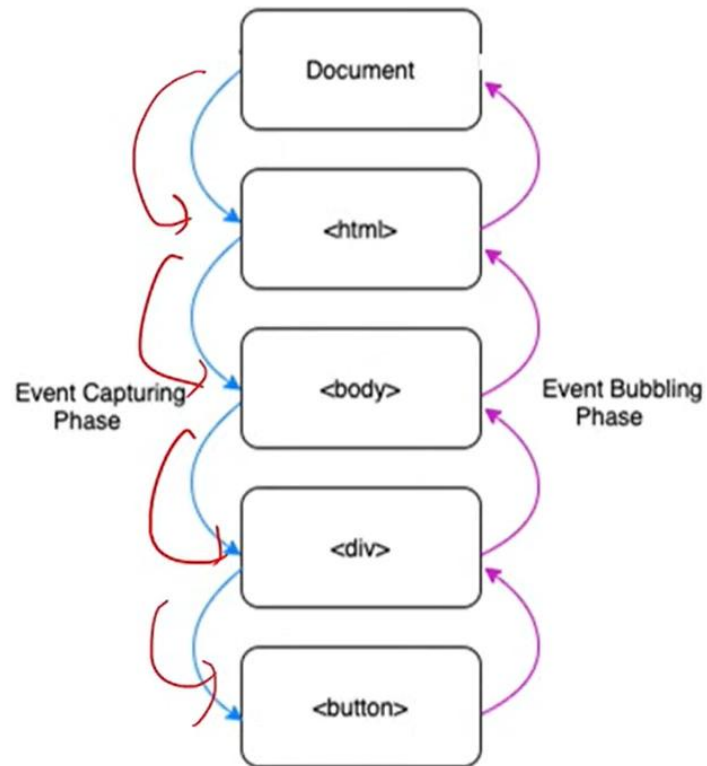
```
function handleBubbling(event) {
  console.log("Bubbling: " + this.id);
  event.stopPropagation(); // Stop event propagation
}
```



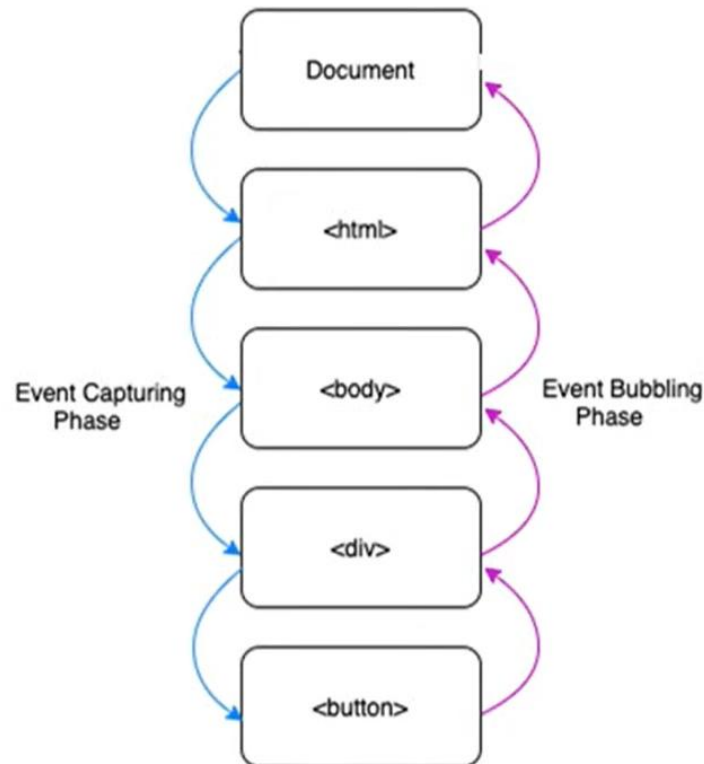
Q. What is Event Capturing in JS?



```
<div id="outer">  
  <div id="inner">  
    <button id="myButton">Click Me</button>  
  </div>  
</div>
```



Q. What is Event Capturing in JS?



```
<div id="outer">
  <div id="inner">
    <button id="myButton">Click Me</button>
  </div>
</div>
```

```
// Get the reference of elements
var outer = document.getElementById("outer");
var inner = document.getElementById("inner");
var button = document.getElementById("myButton");
```

```
// Attach event handlers with elements
outer.addEventListener('click', handleCapture, true);
inner.addEventListener('click', handleCapture, true);
button.addEventListener('click', handleCapture, true);
```

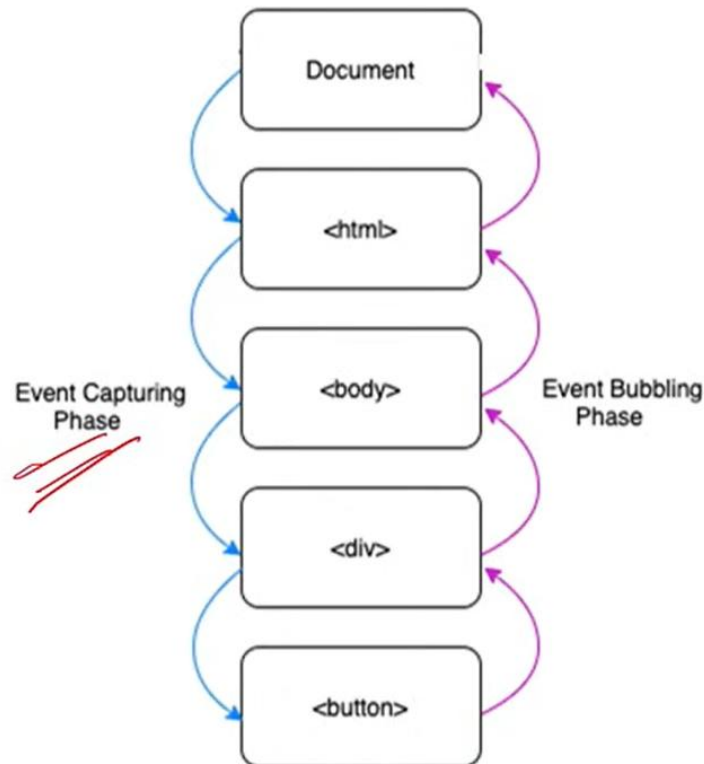
```
function handleCapture(event) {
  console.log("Capturing: " + this.id);
}
```



Q. What is Event Capturing in JS?



- ❖ Event capturing is the process in JavaScript where an event is handled starting from the highest-level ancestor (the root of the DOM tree) and **moving down to the target element**.



```
<div id="outer">
  <div id="inner">
    <button id="myButton">Click Me</button>
  </div>
</div>
```

```
// Get the reference of elements
var outer = document.getElementById("outer");
var inner = document.getElementById("inner");
var button = document.getElementById("myButton");
```

```
// Attach event handlers with elements
outer.addEventListener('click', handleCapture, true);
inner.addEventListener('click', handleCapture, true);
button.addEventListener('click', handleCapture, true);
```

```
function handleCapture(event) {
  console.log("Capturing: " + this.id);
}
```



Q. What is the purpose of the `event.preventDefault()` method in JS?



- ❖ The `event.preventDefault()` method is used to **prevent the default behavior** of an event and the link click will be prevented.

```
<a href="https://example.com" id="myLink">Click Me</a>
```

```
var link = document.getElementById('myLink');  
link.addEventListener('click', handler);  
function handler(event)  
{  
    event.preventDefault(); //Prevent default action  
    // Perform custom behavior  
    console.log('Clicked, default action prevented');  
}
```



Q. What is the use of "this" keyword in the context of event handling in JS?



- ❖ "this" keyword refers to the element that the event handler is attached to.

```
<button id="myButton">Click Me</button>
```

```
var button = document.getElementById("myButton");  
button.addEventListener("click", handler);  
  
function handler(event) {  
  console.log("Clicked:", this.id);  
  this.disabled = true;  
}
```



Q. How to remove an event handler from an element in JS?



❖ **removeEventListener()** method is used to remove event handler from element.

```
<button id="myButton">Click Me</button>
```

```
var button = document.getElementById("myButton");  
  
// Attach the event handler  
button.addEventListener("click", handleClick);  
  
function handleClick() {  
  console.log("Button clicked!");  
}
```

```
// Remove the event handler  
button.removeEventListener("click", handleClick);
```

